Week 6 · The Bow Thruster: Position Control and Berthing

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-10-27

ℹ Recordings of this semester's own classes

Every session of this course is recorded as it is given and posted to the playlist below. The written material here is the reference; the recording is the lecture, at the pace it was actually delivered. Use it to review a derivation, or to catch up on a missed week before the laboratory.

	Where
[충남대] 2026 센서신호처리 — this course, recorded live, 2026	playlist

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	**Control Engineering** (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	**Control System Design** (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	**Advanced Control Engineering** (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	**Sensor Signal Processing and Fusion** (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	**Capstone Design** (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

✎ Getting the course files, and keeping them current (Windows)

The notes, models and scripts are kept in one Git repository that is updated through the semester. Clone it once; before every class, pull. A pull downloads only what has changed since the last one.

When	Where to run it	Command
once	PowerShell — installs Git for Windows	`winget install --id Git.Git -e`
once	the folder that will hold the course, e.g. `Documents`	`git clone https://github.com/wkyouncnu/Sensor-Signal-Processing-and-Fusion.git`
before every class	inside the cloned folder `Sensor-Signal-Processing-and-Fusion`	`git pull`

- `git pull` prints `Already up to date.` when nothing has changed, and otherwise lists the files it updated.
- The repository is private. When Git asks, sign in with the GitHub account the instructor has given access.
- Experiment on copies, not on the cloned files: copy a week's `WXX_simulink` folder elsewhere first, and a pull can never collide with local edits. If it already has, `git stash`, then `git pull`, then `git stash pop` sets the edits aside, updates, and puts them back.
- The MSS toolbox is not part of the repository. The weeks that simulate the Otter need it at `Tools\MSS` inside the cloned folder, and `_tools\mss_path.m` finds it there in either release layout — `Tools\MSS\VESSELS\otter.m` in the older releases, or `Tools\MSS\CRAFT\USV\models\otter.m` in the current master.
- **The two releases are not the same vessel.** The figures printed in these notes were measured with the older layout. Terminal surge speed agrees at 1.0286 m/s, but the current master turns 25.5 % faster under differential thrust, 0.1276 rad/s against 0.1017 rad/s. `mss_path` therefore prefers the older layout when both are present, and warns when only the newer one is available, so a reading that cannot reproduce is announced rather than silent.
- **A 118 MB folder disappears on the first pull after 2026-10-27.** `lectures\MSS-master` held a copy of MSS until then. It sat outside `Tools\MSS`, so no course script could reach it, and it only made cloning slower. Its removal is expected and is not a fault. Any file kept inside that folder must be moved elsewhere before pulling.

- **Course**: USV Guidance, Navigation and Control (Graduate)
- **Department**: Autonomous Vehicle System Engineering, Chungnam National University
- **This week**: ① a third thruster, and the words under-, fully- and over-actuated ② allocation as one matrix inverse, and the ceiling that binds first ③ three nested loops that hold a point ④ stopping, and who declares arrival ⑤ berthing as four latched phases ⑥ alongside against bow-first ⑦ the same controller written in two frames

★ Before starting

- Week 5 must run: the lookahead guidance law produces a heading reference, and the heading autopilot follows it. The vessel can follow a path.
- Verify in one command: `cd lectures/W05_simulink; W05_0_setup` opens the model and prints the gain banner without error.
- What Week 5 could not do: hold a position. A path follower needs forward speed to steer; at zero speed the rudder-equivalent of a differential-thrust hull has no authority in sway at all.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. State the rank test that separates an under-actuated, a fully actuated and an over-actuated vessel, and apply it to the three Otter configurations defined in `otter_config.m`.
2. Build the control effectiveness matrix $\boldsymbol{B}$ for a three-thruster Otter from the column rule alone, and confirm the result against `otter_B.m` to machine precision.

3. Invert $\boldsymbol{B}$, read its three rows as three statements about the vessel, explain why the third row of $\boldsymbol{B}^{-1}$ is $[0\ \ 1\ \ 0]$, and predict which actuator ceiling binds first for a pure sway demand.
4. Predict, from the clipped allocation alone, the surge force and yaw moment that leak out when a sway demand exceeds the attainable set, and verify the prediction to two decimal places against simulation.
5. Explain why a position controller is built as a cascade of a position P loop, a velocity PI loop and a heading P-D loop rather than as one gain from position error to thrust, and name the quantity each loop owns.
6. Derive the braking distance and the settling time constant of the outer position loop, and state the condition under which the cascade's time-scale separation fails.
7. Implement an arrival test as a conjunction of bounded conditions with a latch, and explain what the latch prevents.
8. Compare an alongside and a bow-first berthing on measured phase durations, and attribute the whole difference to one commanded quantity.
9. Place the single rotation that must sit between a NED position error and a body-fixed force, in both of the two possible positions, and state what each choice costs.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b or later, with Simulink
MSS toolbox	on the path; `mss_path()` locates it — do not add it by hand
Week 5	`W05_simulink` runs and its figures regenerate
Working folder	`lectures/W06_simulink`
First command	`W06_0_setup` — defines every constant; then open any of the seven models
Verification	`verify_w06_actuation` — 17 checks, all must print `OK`

Part 1 · Principle and experiment, section by section

Each section below is built the same way, and the three parts answer three different questions.

What is observed	the measurement that makes the section necessary — a number from a run, not an assertion
What follows from it	the derivation, one line at a time, ending in a prediction that can be measured
Experiment N-x	the run that measures it, with the output reproduced exactly as the script prints it

6-0. Setting up (10 min)

Every constant this week lives in one file. Nothing is typed into a block dialogue.

```
cd lectures/W06_simulink
W06_0_setup
```

Expected output:

```
  W06_0_setup
    actuation   3 thrusters, rank B = 3, det B = 0.790  ->  fully actuated
    thrust      stern -66.71 .. +119.68 N per propeller,  bow +-40 N
    lever       l_f/(2 y_pont) = 1.8987
    sway limit  35.13 N from the stern reverse limit, 40.00 N from the bow unit
    berth       (40.0, 1.2) via (40.0, 8.2), final heading +0 deg
```

The banner is the whole week in five lines. Each of them is derived in a section below, and the figure on the right of each experiment is the measurement that confirms it.

Seven models are used — one for each rung of the ladder, and one script for each model. They share one hull, one allocation block and one measurement block; what differs is how much of the control chain is present and what the canvas asks for.

Model	Section	What is closed	What its canvas commands
`W06_3_thrust.slx`	6-1	nothing — generalised force is commanded directly	X 40 N, Y 15 N, N 10 N·m
`W06_2_sway.slx`	6-2	position, velocity and heading, all in the body frame	$(N_d, E_d) = (0, 13)$ m
`W06_1_point.slx`	6-3	the same	$(10, 10)$ m
`W06_4_stop.slx`	6-4	the same	$(12, 25)$ m
`W06_5_dock.slx`	6-5, 6-6	the same, with a four-phase mission block setting the target	$\psi_{\text{berth}} = 0$
`W06_7_front.slx`	6-6	the same	$\psi_{\text{berth}} = -\pi/2$
`W06_8_ned.slx`	6-7	the same, with the outer loop written in NED instead	$(10, 10)$ m

★ The demand is on the canvas, the coefficient is in the file

What a rung is **told to do** is a number on its own canvas: double-clicking the Constant shows it, and the diagram therefore states its own task. What a rung is **built from** — gains, saturation limits, hull constants — is in `W06_0_setup`, shared by all seven. Editing a demand in `W06_0_setup` would change nothing, which is why no demand is there to edit.

★ Run the verification before the experiments

`verify_w06_actuation` re-derives B by hand, re-solves the allocation for 200 random demands, re-computes every ceiling quoted in this document, and compares `W06_vars` against `W06_0_setup` field by field. Its 17 checks are the reason the numbers in this week's tables can be quoted without qualification.

```
verify_w06_actuation

  1  W06_vars = W06_0_setup                                OK
       52 fields compared, none differ
  2  B = the column rule, written by hand                  OK
       max |B - Bhand| = 0.0e+00
  3  det B = 0.790000, rank B = 3                          OK
       det = 0.790000, rank = 3, size = 3x3
  4  inv(B) rows, the third one [0 1 0]                    OK
       lever a = 1.898734, 1/(2 y_pont) = 1.265823
  5  the hand-solved law = B \ tau  (200 demands)          OK
       max error = 3.553e-14 N
  6  pure sway unit column = [-a; a; 1]                    OK
       [-1.898734; 1.898734; 1.000000]
  7  T_max 119.6820, T_min -66.7080, N_max 52.6993         OK
       119.6820 N, -66.7080 N, 52.6993 N m
  8  sway limits 35.1329 / 63.0325 / 40, the first binds OK
       35.1329 (stern astern) / 63.0325 (stern ahead) / 40.0000 (bow) -> Y_max = 35.1329 N
  9  the leak is 0 up to Y_max and not above it            OK
       at Y = 50 N:  X leaks +28.23 N, N leaks -3.85 N m
  10 ranks 2 (under) / 3 square (fully) / 3 of 8 (over)  OK
       base 3x2 r2, bow 3x3 r3, quad_tilt 3x8 r3
  11 M(2,6) = 12.25 gives rdot = -1.553 deg/s^2           OK
       M(2,2) = 162.5000, M(2,6) = 12.2500, M(6,6) = 42.6515
  12 superposition residual < 1e-12                       OK
       0.000e+00
  13 R' R = I, det R = 1, and the 30 deg example          OK
       cos 30 = 0.866, sin 30 = 0.500, x_e = 1.866, y_e = 1.232
  14 app = [berth_N; berth_E + stand_off]                 OK
       berth (40.0, 1.2) + 7.0 m  ->  (40.0, 8.2)
  15 body and NED ask the same velocity, before the cap  OK
       max difference over 24 headings = 2.001e-16 m/s
  16 per-axis bends the direction, magnitude keeps it    OK
       diagonal 0.8732, two 0.35 caps 0.4950, magnitude 0.3500 m/s
  17 W06_read and W06_plot read the same 19 columns      OK
       19 and 19 names, identical

ALL CHECKS PASSED
```

6-1. From two thrusters to three

This section answers: what changes in the equation of motion when a third thruster is fitted at the bow, what changes in the matrix that turns thrust into generalised force, and what are the words for what the third thruster adds.

> **Where this section comes from, and three corrections made on the way**
>
> The actuator model derived here is the one presented in this course's own 2025 slides on USV controller design, pages 125–136. Those pages are reproduced below as they stand, and every equation on them is re-typeset in LaTeX so that it can be read against the code that implements it.
>
> Three of the transcribed equations are **corrected rather than copied**, and each correction is marked where it occurs: the sign convention on the stern lever arms (derivation step 11), the order of the two inverted matrices (step 13), and a missing absolute value inside a square root (step 14). The slides' own MATLAB listing is correct in the first case; the other two are errors that a student who copies the printed formula will meet as a wrong heading and as a complex shaft speed respectively.

What is observed. Week 5 closed a path-following loop on the Otter with two stern propellers and it worked. The same vessel, asked to hold a fixed point, fails: it arrives on the line of its own heading and then stops 14.8 m short, with no gain anywhere in the loop saturated and no disturbance acting. The failure is not one of tuning. It is a property of the actuator set.

Fitting a third thruster changes two things, in two different places, and neither is of any use without the other. The **motion model** gains a non-zero second row in the generalised force it builds for itself, so that a sideways force exists at all. The **allocation** gains a third column, so that a sideways force can be asked for. This section derives both, before any loop is closed around either.

The result. The generalised force a vessel can produce is the image of its actuator vector under one matrix,

$$\boldsymbol{\tau} = B\mathbf{f}, \qquad \boldsymbol{\tau} = \begin{bmatrix} X \\ Y \\ N \end{bmatrix}, \qquad \mathbf{f} = \begin{bmatrix} T_L \\ T_R \\ F_B \end{bmatrix}$$

and the three words that classify a vessel are three statements about that image.

Term	Condition on $B \in \mathbb{R}^{3\times m}$	What it means for control	Otter configuration
under-actuated	$\operatorname{rank} B < 3$	some generalised forces cannot be produced at all	`base` — two stern propellers, $B \in \mathbb{R}^{3\times 2}$, rank 2
fully actuated	$\operatorname{rank} B = 3$, $m = 3$	every $\boldsymbol{\tau}$ is attainable, by exactly one $\mathbf{f}$	`bow_thruster` — $B \in \mathbb{R}^{3\times 3}$, rank 3, $\det B = 0.79$
over-actuated	$\operatorname{rank} B = 3$, $m > 3$	every $\boldsymbol{\tau}$ is attainable, by infinitely many $\mathbf{f}$	`quad_tilt` — four tilting units, $B \in \mathbb{R}^{3\times 8}$, rank 3

The vector $\mathbf{f}$ is not itself a command. What is commanded is a **shaft speed**, and three stages stand between it and the hull:

$$\underbrace{n_i \longmapsto u_i = n_i|n_i|}_{\text{1. signed square}} \qquad \underbrace{\mathbf{u} \longmapsto \mathbf{f} = K\mathbf{u}}_{\text{2. thrust curve}} \qquad \underbrace{\mathbf{f} \longmapsto \boldsymbol{\tau} = B\mathbf{f}}_{\text{3. geometry}}$$

$$\mathbf{n} = \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix}, \qquad \mathbf{u} = \begin{bmatrix} n_1|n_1| \\ n_2|n_2| \\ n_3|n_3| \end{bmatrix}, \qquad K = \begin{bmatrix} k_1 & 0 & 0 \\ 0 & k_2 & 0 \\ 0 & 0 & k_3 \end{bmatrix}, \qquad \boldsymbol{\tau} = BK\mathbf{u}.$$

Only the third stage is a matter of where the thrusters sit; the first two are a matter of what a propeller does. The slides of this course write the geometry matrix as T and call it the **actuator configuration matrix**; it is the same object as the B of this week, assembled by the same rule, and the correspondence is tabulated in the next sub-section.

Symbol	Quantity	Value, source
X	surge force, body frame	N
Y	sway force, body frame	N
N	yaw moment about z_b	N·m
T_L, T_R	thrust of the port and starboard stern propellers	N, $-66.708 \ldots + 119.682$
F_B	bow thruster force, positive to starboard	N, $\lvert F_B \rvert \le 40$
y_{pont}	half the pontoon separation	0.395 m, `otter.m` line 57
l_f	bow thruster arm forward of the origin	1.5 m, `otter_config.m`
m	number of actuators	2, 3 or 8 by configuration
n_i	shaft speed of thruster i	rad/s; stern $-101.74 \ldots + 103.93$, bow ± 60.08
u_i	the signed square $n_i\lvert n_i \rvert$	$(\text{rad/s})^2$
K	diagonal matrix of thrust coefficients	$\operatorname{diag}(k_1, k_2, k_3)$
k_{pos}, k_{neg}	stern thrust coefficients, ahead and astern	0.011080, 0.006445 N/(rad/s)^2, `otter.m`
k_3	bow thrust coefficient, the same in both directions	0.011080 N/(rad/s)^2, `otter_config.m`
T	the slides' name for the actuator configuration matrix	the same object as B

Derivation, one line at a time.

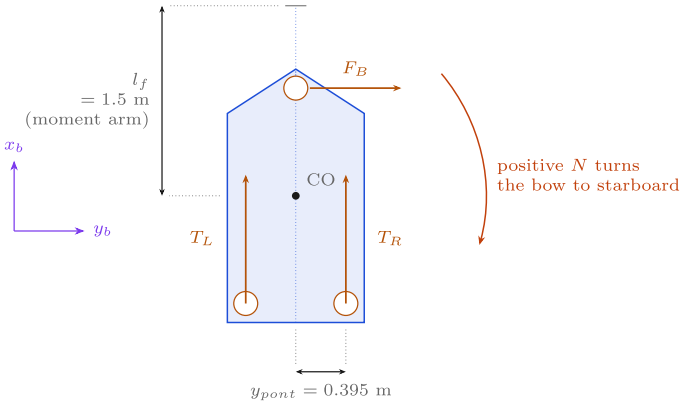
1. Fix the body frame at the origin CO used by `otter.m`: x_b forward, y_b to starboard, z_b down. A force $\mathbf{F} = [F_x \;\; F_y]^{\mathsf{T}}$ applied at the body-frame point (x_t, y_t) produces the generalised force

$$\begin{bmatrix} X \\ Y \\ N \end{bmatrix} = \begin{bmatrix} F_x \\ F_y \\ x_t F_y - y_t F_x \end{bmatrix}.$$

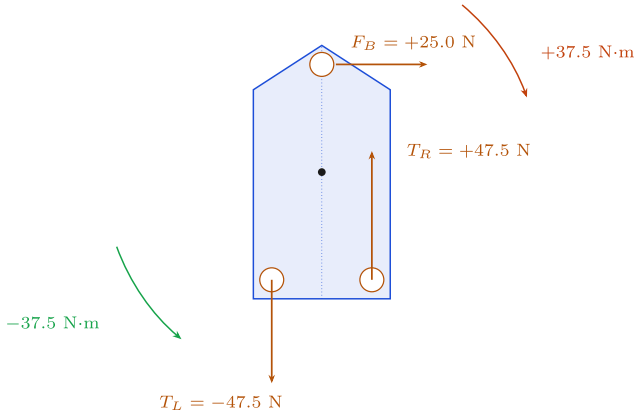
2. The k-th **column** of B is therefore nothing but the generalised force produced by one unit of actuator k. There is no matrix to memorise: each column is read off the geometry of one actuator.
3. The port stern propeller pushes along $+x_b$ and sits at $y_t = -y_{pont}$, so its column is $[1 \;\; 0 \;\; +y_{pont}]^{\mathsf{T}}$. The starboard propeller is the mirror image, $[1 \;\; 0 \;\; -y_{pont}]^{\mathsf{T}}$. The bow thruster pushes along $+y_b$ and sits at $x_t = +l_f$, so its column is $[0 \;\; 1 \;\; +l_f]^{\mathsf{T}}$.
4. Assembling the three columns,

$$B = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ y_{pont} & -y_{pont} & l_f \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 0.395 & -0.395 & 1.5 \end{bmatrix}.$$

(a) where the three thrusters sit



(b) pure sway — all three thrusters work



$X = T_L + T_R = 0$ — the stern pair cancel in surge

Read the arms, not the hull stations. The lateral pair push along x_b, so their fore-aft station never enters B — their only moment arm is the offset y_{pont} = 0.395 m, and panel (a) draws them at the transom only because that is where they are. The bow unit is drawn in its tunnel, inside the hull, where such a unit is fitted. The arm it enters B through is longer than that station: `otter_config` sets l_f = 1.5 m on a hull 2.0 m long, so the dimension runs past the bow to the tick, and the dotted line joins the two. That gap is an idealisation of the model, not a second thruster. The arm, not a hull station, is what B is built from, and every Week 6 number follows from it.

Sections 6-1 and 6-2. The three thrusters of the `bow_thruster` configuration, drawn to scale from `otter_config.m`: the hull is 2.0 m long and 1.08 m in beam. Panel (a) fixes the two moment arms and the sign of N; panel (b) solves $B\,\mathbf{u} = (0,\ 25,\ 0)$ and shows why a pure sideways push needs all three — the bow thruster alone would turn the hull.

In the figure	Meaning
panel (a), two arrows at the transom	the stern pair, each pushing along $+x_b$; their arms are the lateral offsets $\pm y_{pont} = \pm 0.395$ m
panel (a), one arrow ahead of the bow	the bow thruster, pushing along y_b at the arm $l_f = 1.5$ m
the arc labelled *positive N turns the bow to starboard*	the sign convention: z_b points down, so a positive moment about it is clockwise seen from above
`CO`	the body-frame origin B is written about — not the centre of gravity
panel (b), $F_B = +25.0$ N	the bow force needed for a pure sway demand $Y = 25$ N: exactly Y, by the second row of B
panel (b), $T_L = -47.5$, $T_R = +47.5$ N	the stern pair **opposing** each other, to cancel the bow unit's moment
the two moment labels ± 37.5 N·m	$l_f F_B = 1.5 \times 25 = 37.5$ N·m produced by the bow unit, and the equal and opposite $2 y_{pont} \times 47.5 = 37.5$ N·m from the stern pair
the caption *the stern pair cancel in surge*	$X = T_L + T_R = 0$: the hull is pushed sideways and not forward

Panel (a) is the matrix above, drawn. Panel (b) is §6-2 in advance: a pure sideways force on this hull costs **three** thrusters working, two of them against each other, and the magnitudes are set by the ratio of the arms.

5. Expanding the determinant along the second row, whose only non-zero entry is $B_{23} = 1$,

$$\det B = (-1)^{2+3} \cdot 1 \cdot \det \begin{bmatrix} 1 & 1 \\ 0.395 & -0.395 \end{bmatrix} = -(-0.395 - 0.395) = 0.790.$$

6. The determinant is non-zero, so B is invertible and the Otter with a bow thruster is **fully actuated**: there is exactly one $\boldsymbol{f}$ for every $\boldsymbol{\tau}$, and no choice to make. Delete the third column and the remaining 3×2 matrix has rank 2; its image is the plane $Y = 0$, which is why the two-thruster Otter can never produce a pure sideways force. Add more actuators and the rank stays 3 while the null space grows, which is what makes allocation an optimisation problem rather than a solve — the subject of Week 7.
7. Nothing in the derivation used a hull station. What enters B is the **moment arm**, which for the stern pair is the lateral offset y_{pont} and for the bow unit is the longitudinal arm l_f. The fore-aft position of the stern propellers never appears, because they push along x_b and a force through a point parallel to x_b has no moment arm except in y.
8. A thruster is commanded by shaft speed, not by force. Bollard thrust grows with the **square** of the shaft speed, and the sign must follow the direction of rotation, so the quantity that is linear in thrust is the signed square $u_i = n_i |n_i|$ rather than n_i^2. The coefficient is a property of the hardware:

$$f_i = k_i \, u_i = k_i \, n_i |n_i|, \qquad k_i = \begin{cases} k_{pos} & n_i > 0 \\ k_{neg} & \text{otherwise} \end{cases} \quad \text{(screw propeller)}, \qquad k_3 = \text{const} \quad \text{(tunnel thruster)}.$$

A screw propeller is asymmetric because its blades are shaped for one direction: $k_{neg}/k_{pos} = 0.58$ on this hull. A tunnel thruster is a duct open at both ends with a symmetric impeller, so one coefficient serves both directions. The three ratings follow from the three shaft-speed limits and nothing else:

Thruster	Limit	Coefficient	Rating
stern, ahead	$n_{max} = +103.93$ rad/s	$k_{pos} = 0.011080$	$T_{max} = k_{pos} n_{max}^2 = +119.682$ N
stern, astern	$n_{min} = -101.74$ rad/s	$k_{neg} = 0.006445$	$T_{min} = -k_{neg} n_{min}^2 = -66.708$ N
bow, either way	$\pm n_{3,max} = \pm 60.08$ rad/s	$k_3 = 0.011080$	$F_{B,max} = k_3 n_{3,max}^2 = 40.000$ N

9. With two stern propellers only, the generalised force the motion model builds for itself is the one line of `otter.m`,

$$\boldsymbol{\tau}_{\text{two}} = \begin{bmatrix} T_1 + T_2 \\ 0 \\ 0 \\ 0 \\ 0 \\ -l_1 T_1 - l_2 T_2 \end{bmatrix}, \qquad l_1 = -y_{pont}, \quad l_2 = +y_{pont},$$

and the **second row is identically zero**. That zero is the formal statement of under-actuation: it is not that the two-thruster Otter is bad at going sideways, it is that the quantity does not exist in its model. No gain placed anywhere outside this equation can create it.

10. Fitting the bow unit at $(l_3, 0)$ pushing along $+y_b$ puts its thrust into the second row and its moment into the sixth, and changes nothing else:

$$\boldsymbol{\tau}_{\text{three}} = \begin{bmatrix} T_1 + T_2 \\ T_3 \\ 0 \\ 0 \\ 0 \\ -l_1 T_1 - l_2 T_2 + l_3 T_3 \end{bmatrix} = \begin{bmatrix} \tau_x \\ \tau_y \\ 0 \\ 0 \\ 0 \\ \tau_N \end{bmatrix}, \qquad \begin{aligned} \tau_x &= k_1 u_1 + k_2 u_2, \\ \tau_y &= k_3 u_3, \\ \tau_N &= y_{pont} k_1 u_1 - y_{pont} k_2 u_2 + l_3 k_3 u_3. \end{aligned}$$

Rows 3, 4 and 5 — heave, roll and pitch — stay zero because no thruster on this vessel pushes vertically or about a horizontal axis. They are carried so that the vector matches the six-degree-of-freedom hull it is added to.

11. **Correction 1 — the sign of the stern arms.** The slides write the third row of the configuration matrix as $\begin{bmatrix} l_1 & -l_2 & l_3 \end{bmatrix}$ and, separately, $l_1 = -l_2 = -Y_{pont}$. Substituting the second statement into the first gives

$$\tau_N = -y_{pont} T_1 - y_{pont} T_2 + l_3 T_3 \qquad \text{(as printed — and wrong)},$$

in which both stern propellers turn the hull the **same** way: driving the pair equally ahead would spin the vessel, which it does not. The slides' own MATLAB listing on page 135 writes $-l_1 T_1 - l_2 T_2$, which with the signed arms gives $+y_{pont} T_1 - y_{pont} T_2$ and is correct. The code is right and the printed matrix row is wrong. Writing the arms as **positive distances**, as this course does, removes the ambiguity entirely: the third row is $\begin{bmatrix} y_{pont} & -y_{pont} & l_f \end{bmatrix}$, and every sign in it comes from the column rule of step 2 rather than from a separate convention that has to be remembered.

12. Where the result enters. The hull's equation of motion is unchanged in every term but one,

$$M\dot{\boldsymbol{\nu}} + C(\boldsymbol{\nu}_r)\boldsymbol{\nu}_r + D(\boldsymbol{\nu}_r)\boldsymbol{\nu}_r + \mathbf{g}(\boldsymbol{\eta}) = \boldsymbol{\tau},$$

with M, C, D and $\mathbf{g}$ identical to `otter.m` line for line. Only $\boldsymbol{\tau}$ differs, and only in rows 2 and 6. This is why `otter_bow.m` is a copy of `otter.m` with four marked changes rather than a new model: the hull is the same hull, and the file has to show that.

13. **Correction 2 — the order of the two inverses.** Allocation runs the chain of step 8 backwards. From $\boldsymbol{\tau} = BK\mathbf{u}$,

$$\mathbf{u} = (BK)^{-1}\boldsymbol{\tau} = K^{-1}B^{-1}\boldsymbol{\tau} \qquad \text{(correct)},$$

whereas the slides write $\mathbf{u} = B^{-1}K^{-1}\boldsymbol{\tau}$. The two agree only when K commutes with B^{-1}, which for a diagonal K means only when all three coefficients are equal. They are not: the moment one stern propeller is asked to reverse, its coefficient becomes k_{neg}. Dimensions settle it before numbers do — K^{-1} divides by a coefficient of N/(rad/s)², so it must act on a vector of **forces**, and $B^{-1}\boldsymbol{\tau}$ is that vector, while $\boldsymbol{\tau}$ itself has a moment in its third entry. On the pure sway demand $\boldsymbol{\tau} = [0,\ 25,\ 0]^{\mathsf{T}}$ the difference is measurable:

	port n_1	starboard n_2	bow n_3	delivered X	delivered Y	delivered N
$K^{-1}B^{-1}\boldsymbol{\tau}$	-85.820	$+65.453$	$+47.501$	0.000 N	25.000 N	0.000 N·m
$B^{-1}K^{-1}\boldsymbol{\tau}$	-65.453	$+65.453$	$+47.501$	$+19.857$ N	25.000 N	$+7.844$ N·m

The wrong order spins the port propeller symmetrically with the starboard one, which at k_{neg} delivers only -27.611 N against the -47.468 N required. The sway still comes out; the surge and the yaw that were asked to be zero do not.

14. **Correction 3 — the absolute value.** Inverting the signed square of step 8 gives

$$n_i = \operatorname{sign}(u_i)\sqrt{|u_i|}\,,$$

where the slides write $\operatorname{sign}(u_i)\sqrt{u_i}$. Every astern command has $u_i < 0$: on the demand of step 13, $u_1 = -7365.14$ and $\sqrt{u_1} = 85.820\,i$. The absolute value is not decoration, and in MATLAB its omission does not raise an error — it returns a complex shaft speed, which propagates silently.

15. The derivation makes a prediction that costs one run to check. Commanded a pure sway force of 25 N from rest, the three shafts must turn at -85.82, $+65.45$ and $+47.50$ rad/s, the port propeller faster than the starboard one by the ratio $\sqrt{k_{pos}/k_{neg}} = 1.311$ although the two thrusts are equal and opposite. Experiment 6-1 commands the three axes one at a time and reads the shaft speeds back.

The bow thruster is, deliberately, the weakest of the three. The stern pair reach $+119.682$ N ahead and -66.708 N astern; the bow unit is rated at $|F_B| \le 40$ N, a third of the ahead figure. That ratio is not an arbitrary choice of this course but the ordinary shape of the installation:

Why the bow unit is small	Consequence on this hull
a tunnel thruster's thrust grows with the tunnel's cross-section, and the tunnel must fit inside the submerged part of the hull	the bow of a 2 m Otter is narrow and shallow, so the tunnel is small where the hull is smallest
tunnel thrust falls off as the vessel gains forward speed, the inflow sweeping the jet astern	the unit is useful at the speeds of §6-4 and §6-5 — under 0.8 m/s — and of little use at transit speed
the stern propellers are sized for transit, which is the duty that sets installed power	surge has the power of the whole propulsion plant behind it; sway has a manoeuvring aid

The weak member is therefore the one that would be expected to bind first. Section 6-2 measures which one actually does, and the answer is not this one — the ceiling on sway turns out to be set by the **geometry** of the lever $l_f/2y_{pont}$ acting on the stern astern limit, not by the bow unit's rating. That the rating is not the binding constraint is a result, not an assumption, and it is the reason §6-2 is a section rather than a footnote.

Counting the thrusters is enough to place any hull in the three classes, and this course works through them in order:

Thrusters	B	Rank	Class	What becomes possible	Where in this course
2 stern propellers	$\mathbb{R}^{3\times2}$	2	under-actuated	surge and yaw independently; sway only as a by-product of turning	Weeks 1–5
2 stern + 1 bow tunnel	$\mathbb{R}^{3\times3}$	3	fully actuated	all three of surge, sway and yaw at once — a point held at an attitude, and a berth approached sideways	**Week 6**
4 tilting units	$\mathbb{R}^{3\times8}$	3	over-actuated	the same three, with a null space to spend on wear, power or a failed unit	Week 7

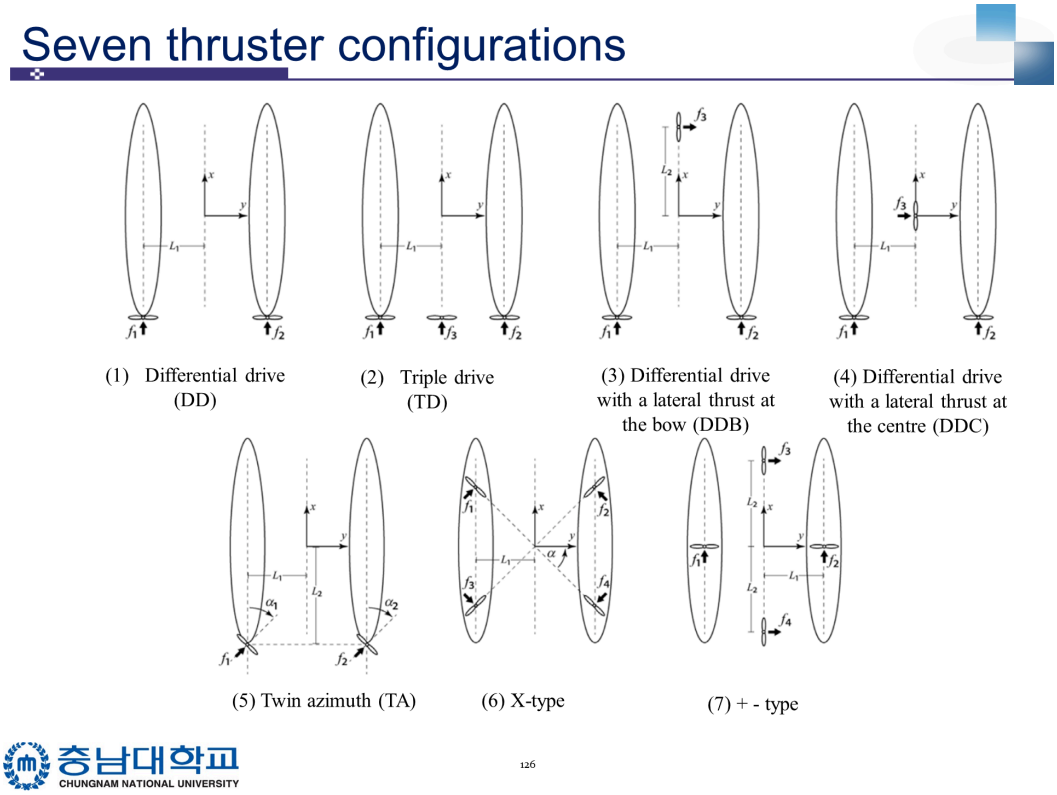
The middle row is the one that changes what can be *asked*. The bottom row changes only what can be *chosen* among the ways of granting a request that was already grantable.

The slides this derivation re-typesets

The pages reproduced here are from this course's 2025 slides on USV controller design. They are included as they stand so that a student who has those slides can place each equation of the derivation against the page it came from. The symbols differ in four places, and the correspondence is fixed once here rather than repeated under each figure.

On the slides	In this course	Note
T	B	the same object: the actuator configuration matrix, one column per thruster
$l_1,\ l_2$ with $l_1 = -l_2 = -Y_{pont}$	y_{pont}, a **positive distance**, with the signs carried in the matrix row	derivation step 11 — the slides' own listing agrees with this, their printed matrix row does not
l_3	l_f	1.5 m, the bow unit's arm forward of the CO
k_1, k_2	k_{pos} or k_{neg}, selected by the sign of u_i	the slides state the selection separately, on page 135
k_3	k_{bow}	0.011080 N/(rad/s)^2, the same in both directions
$\tau_x,\ \tau_y,\ \tau_N$	$X,\ Y,\ N$	body-frame surge force, sway force and yaw moment
f_i in the configuration sketches	T_L, T_R, F_B	thrust of one unit, in newtons

Seven thruster configurations. The arrangement of this week is number (3), differential drive with a lateral thrust at the bow.



On the slide	In this course
(1) Differential drive (DD)	the Otter of Weeks 1–5: two stern units, $\operatorname{rank} B = 2$
(2) Triple drive (TD)	three units all pushing along x_b — a third column $[1\ \ 0\ \ 0]^{\mathsf{T}}$, which adds nothing to the rank
(3) DD with a lateral thrust at the bow (DDB)	**this week**: $\operatorname{rank} B = 3$, square, one exact solution
(4) DD with a lateral thrust at the centre (DDC)	the same rank, but with the lateral unit at $x_t = 0$ its column is $[0\ \ 1\ \ 0]^{\mathsf{T}}$ and sway costs the stern pair nothing — §6-2's lever a vanishes
(5) Twin azimuth (TA)	two units free to rotate through α_1, α_2: rank 3 with a null space, which is the situation Week 7 meets with four units instead of two
(6) X-type, (7) +-type	four fixed units, rank 3, over-actuated — Week 7 gives each of its four a servo
L_1, L_2 in the sketches	the moment arms of step 2: L_1 the lateral offset of the stern pair, L_2 the longitudinal arm of a lateral unit

Before the bow unit is fitted. The page states $\tau = TK\mathbf{u}$ for the two-thruster vessel, and writes the sway row as zero in red.

Adding Bow thruster

➢ Before adding bow thruster

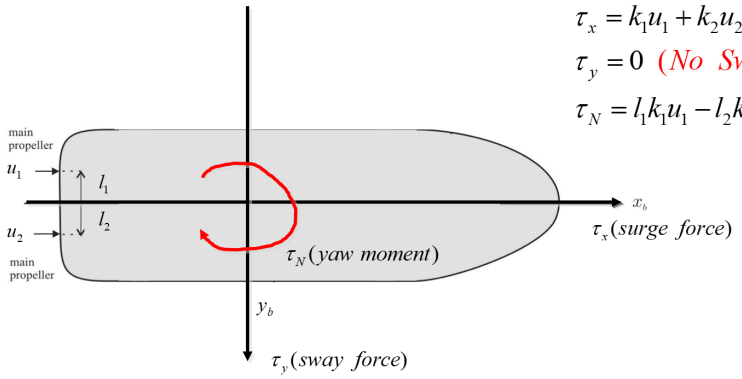
T: Actuator configuration matrix
K: Diagonal matrix of thrust coefficients

RPM Control variable given by

$$n=\begin{bmatrix} n_1 \\ n_2 \end{bmatrix} \qquad u=\begin{bmatrix} u_1 \\ u_2 \end{bmatrix}=\begin{bmatrix} n_1|n_1| \\ n_2|n_2| \end{bmatrix}$$

$$\tau=\begin{bmatrix} \tau_x \\ \tau_N \end{bmatrix}=\mathrm{TK}\,u=\begin{bmatrix} 1 & 1 \\ l_1 & -l_2 \end{bmatrix}\begin{bmatrix} k_1 & 0 \\ 0 & k_2 \end{bmatrix}\begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$\tau_x = k_1u_1 + k_2u_2$$
$$\tau_y = 0 \ \ (No\ \ Sway\ \ Force)$$
$$\tau_N = l_1k_1u_1 - l_2k_2u_2$$



충남대학교 CHUNGNAM NATIONAL UNIVERSITY

127

On the slide	Meaning
$\tau_y = 0$, boxed in red	derivation step 9 — the row that does not exist, and the whole reason for this week
the two-row τ beside the three-row one	sway is not a small quantity on this vessel, it is an absent one
T: *actuator configuration matrix*, K: *diagonal matrix of thrust coefficients*	the header note, and the two names used throughout the slides
x_b forward, y_b down the page	the same body frame as Fossen's, drawn from above with z_b into the page
u_1, u_2 at the stern with arms l_1, l_2	the two main propellers of Weeks 1–5, and the only two columns T has

$$\mathbf{n}=\begin{bmatrix} n_1 \\ n_2 \end{bmatrix}, \qquad \mathbf{u}=\begin{bmatrix} n_1|n_1| \\ n_2|n_2| \end{bmatrix}, \qquad \boldsymbol{\tau}=\begin{bmatrix} \tau_x \\ \tau_N \end{bmatrix}=TK\mathbf{u}=\begin{bmatrix} 1 & 1 \\ l_1 & -l_2 \end{bmatrix}\begin{bmatrix} k_1 & 0 \\ 0 & k_2 \end{bmatrix}\begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$\tau_x = k_1u_1 + k_2u_2, \qquad \boxed{\tau_y = 0 \quad \text{(no sway force)}}, \qquad \tau_N = l_1k_1u_1 - l_2k_2u_2$$

After the bow unit is fitted, in NED. A third column appears in T and a third diagonal entry in K.

Adding Bow thruster (NED)

➢ After adding bow thruster

RPM

$$n = \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix}$$

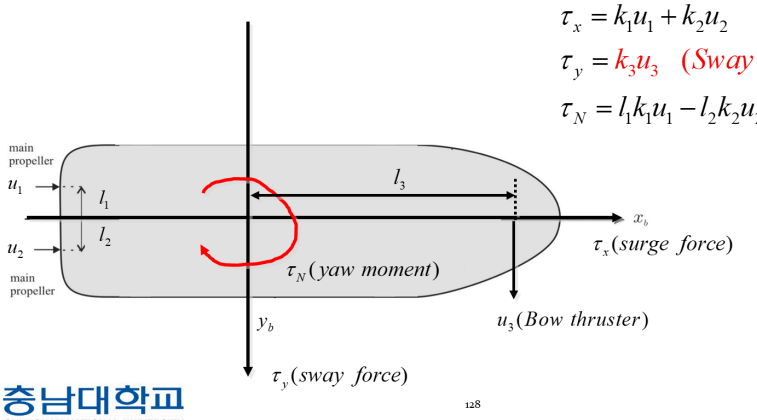
Control variable given by

$$u = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \begin{bmatrix} n_1|n_1| \\ n_2|n_2| \\ n_3|n_3| \end{bmatrix}$$

T: Actuator configuration matrix
K: Diagonal matrix of thrust coefficients

$$\tau = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_N \end{bmatrix} = \mathrm{TK}\,u = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ l_1 & -l_2 & l_3 \end{bmatrix} \begin{bmatrix} k_1 & 0 & 0 \\ 0 & k_2 & 0 \\ 0 & 0 & k_3 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}$$

$$\tau_x = k_1 u_1 + k_2 u_2$$
$$\tau_y = k_3 u_3 \quad (Sway\ Force)$$
$$\tau_N = l_1 k_1 u_1 - l_2 k_2 u_2 + l_3 k_3 u_3$$



충남대학교 CHUNGNAM NATIONAL UNIVERSITY

128

$$\mathbf{n} = \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix}, \qquad \boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_N \end{bmatrix} = TK\mathbf{u} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ l_1 & -l_2 & l_3 \end{bmatrix} \begin{bmatrix} k_1 & 0 & 0 \\ 0 & k_2 & 0 \\ 0 & 0 & k_3 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}$$

$$\tau_x = k_1 u_1 + k_2 u_2, \qquad \boxed{\tau_y = k_3 u_3 \quad (\text{sway force})}, \qquad \tau_N = l_1 k_1 u_1 - l_2 k_2 u_2 + l_3 k_3 u_3$$

On the slide	Meaning
the third column $[0\ \ 1\ \ l_3]^{\mathsf{T}}$	derivation step 3, read off the geometry of one actuator
$\tau_y = k_3 u_3$, boxed in red	the row that was zero on the previous page
l_3, the arm from the origin forward to the bow unit	$l_f = 1.5$ m, the longest arm on the vessel
u_3 drawn pointing down the page	along $+y_b$, to starboard: positive F_B pushes the bow to starboard
the row read with $l_1 = -l_2 = -Y_{pont}$	the one place to be careful — derivation step 11

The same vessel in ENU. The slides also give the form used when the axes are east-north-up rather than north-east-down, and mark the reversed bow column in red.

Adding Bow thruster (ENU)

➢ After adding bow thruster

RPM

$$n = \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix}$$

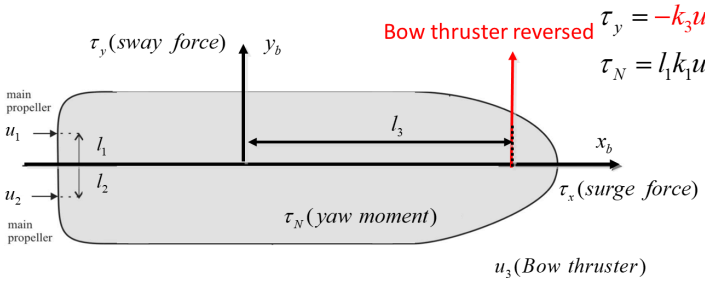
Control variable given by

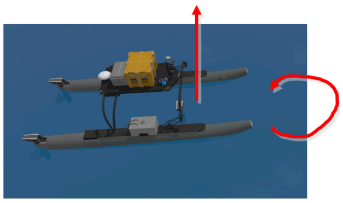
$$u = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \begin{bmatrix} n_1|n_1| \\ n_2|n_2| \\ n_3|n_3| \end{bmatrix}$$

T: Actuator configuration matrix
K: Diagonal matrix of thrust coefficients

$$\tau = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_N \end{bmatrix} = \mathrm{TK}\,u = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & -1 \\ l_1 & -l_2 & -l_3 \end{bmatrix} \begin{bmatrix} k_1 & 0 & 0 \\ 0 & k_2 & 0 \\ 0 & 0 & k_3 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}$$

$$\tau_x = k_1 u_1 + k_2 u_2$$
$$\tau_y = -k_3 u_3 \quad (Sway\ Force)$$
$$\tau_N = l_1 k_1 u_1 - l_2 k_2 u_2 - l_3 k_3 u_3$$





충남대학교 CHUNGNAM NATIONAL UNIVERSITY

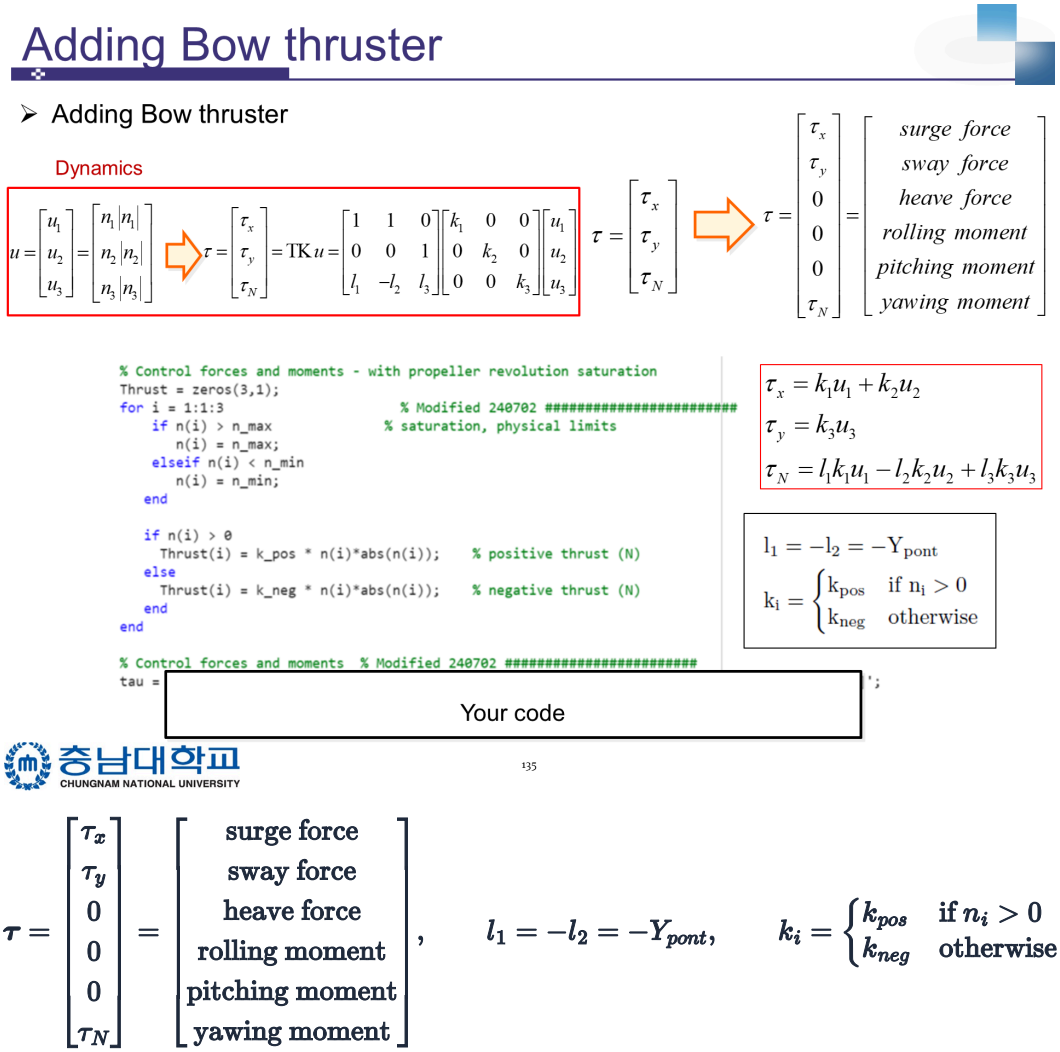
129

$$T_{\mathrm{ENU}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & -1 \\ l_1 & -l_2 & -l_3 \end{bmatrix}, \qquad \tau_y = -k_3 u_3, \qquad \tau_N = l_1 k_1 u_1 - l_2 k_2 u_2 - l_3 k_3 u_3$$

On the slide	Meaning
the third column negated, annotated *bow thruster reversed*	y points to port and z points up, so the same impeller rotation produces the opposite sway and the opposite moment
the photograph of the twin-pontoon USV	the vessel the slides' parameters were taken from
everything else unchanged	the stern columns are unaffected because they push along x, which both conventions agree on

> **⚠ This course uses NED throughout**
>
> Every model, script and figure of this week is north-east-down, so the T_{ENU} above is included for reading the slides and is used nowhere. Mixing the two silently reverses the sign of every sway command and every yaw moment, which presents as a berthing approach that drifts away from the quay instead of towards it.

The dynamics page, and the code that implements it. This is the page the week's motion model is written from.



$$\boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ 0 \\ 0 \\ 0 \\ \tau_N \end{bmatrix} = \begin{bmatrix} \text{surge force} \\ \text{sway force} \\ \text{heave force} \\ \text{rolling moment} \\ \text{pitching moment} \\ \text{yawing moment} \end{bmatrix}, \qquad l_1 = -l_2 = -Y_{pont}, \qquad k_i = \begin{cases} k_{pos} & \text{if } n_i > 0 \\ k_{neg} & \text{otherwise} \end{cases}$$

On the slide	In this course
the six-row $\boldsymbol{\tau}$ with rows 3–5 zero	derivation step 10 — the three-degree-of-freedom force, carried in a six-degree-of-freedom vector
`Thrust = zeros(3,1); for i = 1:1:3`	the loop of `otter_bow.m`, change 3 of 4: two iterations become three
`if n(i) > n_max … elseif n(i) < n_min`	the saturation inside the motion model, which is physical and independent of the one in the allocation
`if n(i) > 0 … Thrust(i) = k_pos * n(i)*abs(n(i))`	the signed square of step 8, with the coefficient chosen by sign
`% Modified 240702`	the two places the original `otter.m` was changed
the `Your code` box over the `tau =` line	the line left as an exercise on the slide; it is printed in full immediately below
$l_1 = -l_2 = -Y_{pont}$	derivation step 11 — this is the statement that conflicts with the printed matrix row

What the slide prints, and what its box hides. Lines 212–252 of `_tools/otter_bow.m`, exactly as the file has them. The loop is the part the slide shows; the vector after it is the part the `Your code` box covers.

```
% ---- CHANGED (3 of 4) ----------------------------------------------------
% Control forces and moments - with propeller revolution saturation
% 루프가 2 회에서 3 회로 늘고, 셋째 추진기만 제 계수와 제 한계를 쓴다.
% The loop runs three times; the third unit uses its own coefficient and limit.
Thrust = zeros(3,1);
for i = 1:1:3
    if i < 3                                % 선미 프로펠러 / stern propellers
        if n(i) > n_max                     % saturation, physical limits
            n(i) = n_max;
        elseif n(i) < n_min
            n(i) = n_min;
        end

        if n(i) > 0
            Thrust(i) = k_pos * n(i)*abs(n(i));   % positive thrust (N)
        else
            Thrust(i) = k_neg * n(i)*abs(n(i));   % negative thrust (N)
        end
    else                                    % 선수추진기 / the bow thruster
        if n(i) > n3_max
            n(i) = n3_max;
        elseif n(i) < -n3_max
            n(i) = -n3_max;
        end

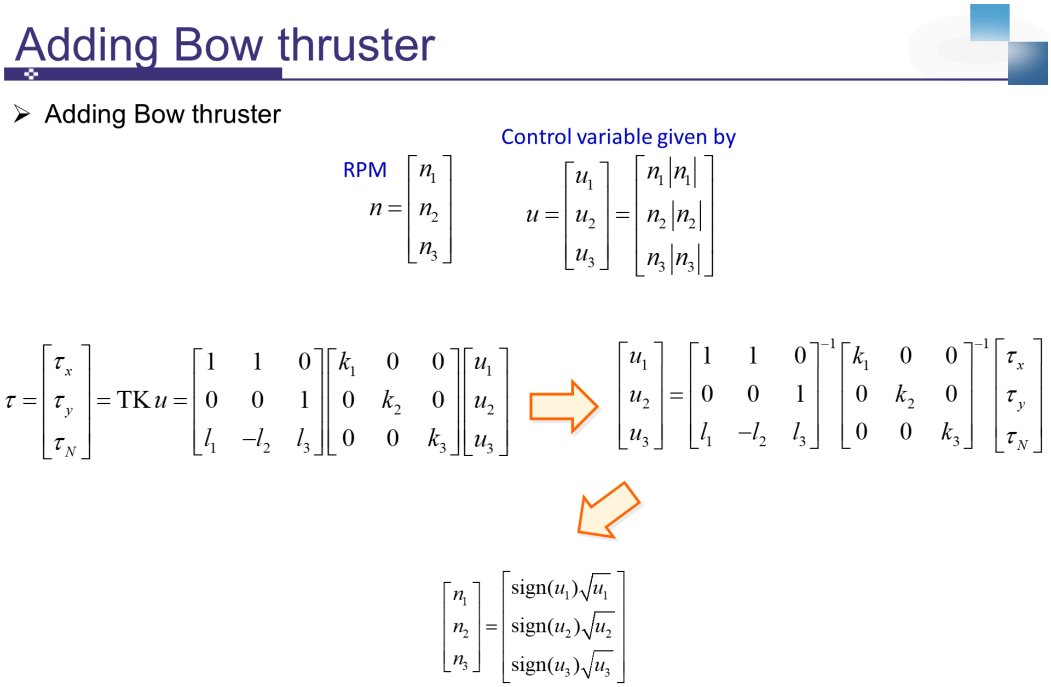
        Thrust(i) = k3 * n(i)*abs(n(i));          % symmetric, both ways (N)
    end
end
% --------------------------------------------------------------------------

% ---- CHANGED (4 of 4) ----------------------------------------------------
% Control forces and moments
% otter.m:  tau = [T1+T2  0  0 0 0  -l1*T1 - l2*T2]'      <- sway is zero
% here:     the bow thrust enters row 2, and its moment l3*T3 enters row 6.
tau = [Thrust(1) + Thrust(2)
       Thrust(3)
       0
       0
       0
       -l1 * Thrust(1) - l2 * Thrust(2) + l3 * Thrust(3)];
% --------------------------------------------------------------------------
```

In the code	In the figure	Note
`Thrust = zeros(3,1); for i = 1:1:3`	the `for i = 1:1:2` of the slide's listing, with the bound raised	the only structural change in the loop; the stern branch is the original, character for character
`if i < 3 … else … end`	the branch the slide does not have	the stern pair and the bow unit saturate against different limits and multiply by different coefficients, so one loop body cannot serve both
`n(i) > n_max … n(i) < n_min`	`n_max`, `n_min` on the slide	asymmetric, because a stern propeller pushes harder forward than back: $+103.93$ and -101.74 rad/s
`n(i) > n3_max … n(i) < -n3_max`	hidden behind the box	symmetric, because a tunnel thruster is the same impeller running either way
`k_pos` / `k_neg` chosen by `if n(i) > 0`	the two-branch `if` on the slide	derivation step 8 — the signed square $u_i = n_i\lvert n_i\rvert$ with a sign-dependent coefficient
`Thrust(i) = k3 * n(i)*abs(n(i))`	hidden	one coefficient both ways; $k_3 = k_{pos}$ here, from `otter_config`
the `tau = [...]` vector	the whole of the `Your code` box	the six-row $\boldsymbol{\tau}$ of the equation above it, now with rows 2 and 6 carrying the bow unit

The reading of that vector row by row, and the three checks that verify it, are in *What changed in the motion model* below.

The allocation page. Running the chain backwards, from a demanded $\boldsymbol{\tau}$ to three shaft speeds.

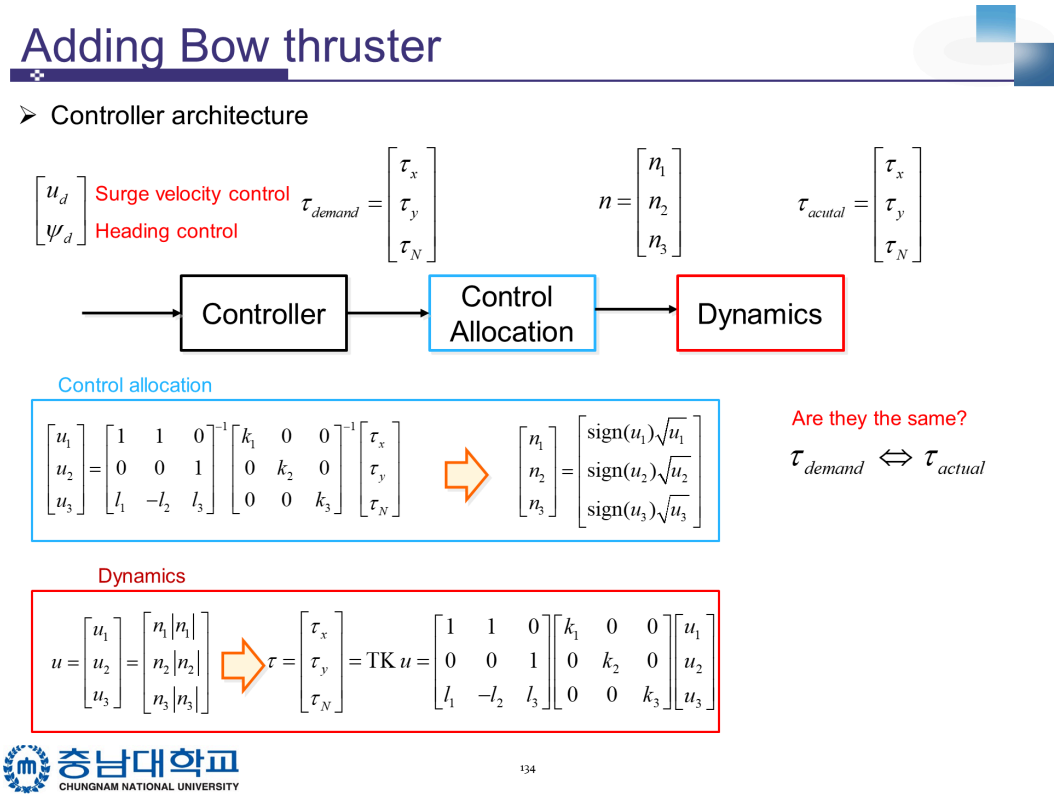


On the slide	In this course
$\boldsymbol{\tau} = T K \mathbf{u}$, the forward chain restated	derivation steps 8 and 10
$T^{-1}K^{-1}$	**corrected** to $K^{-1}B^{-1}$ — the order matters the moment one propeller reverses, derivation step 13
$\mathbf{sign}(u_i)\sqrt{u_i}$	**corrected** to $\mathbf{sign}(u_i)\sqrt{\lvert u_i\rvert}$ — derivation step 14
the inverse taken explicitly, not solved by hand	what the `allocate` block of this week does, listed in §6-2

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = T^{-1}K^{-1} \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_N \end{bmatrix} \quad \text{(as printed)}, \qquad \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix} = \begin{bmatrix} \text{sign}(u_1)\sqrt{u_1} \\ \text{sign}(u_2)\sqrt{u_2} \\ \text{sign}(u_3)\sqrt{u_3} \end{bmatrix} \quad \text{(as printed)}$$

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = K^{-1}B^{-1} \begin{bmatrix} X \\ Y \\ N \end{bmatrix} \quad \text{(corrected, step 13)}, \qquad n_i = \text{sign}(u_i)\sqrt{|u_i|} \quad \text{(corrected, step 14)}$$

The controller architecture. Three blocks in a row, and one question in red at the bottom.



On the slide	In this course
`Controller` taking $[u_d;\ \psi_d]$, labelled *surge velocity control* and *heading control*	§6-3's three loops, and the two subsystems of `velocity loop`
$\boldsymbol{\tau}_{demand} = [\tau_x\ \ \tau_y\ \ \tau_N]^\mathsf{T}$	the output of `position loop`, the input of `allocation`
`Control Allocation`, repeating $T^{-1}K^{-1}$ and $\text{sign}(u)\sqrt{u}$	the `allocate` block, with both corrections applied
$\mathbf{n} = [n_1\ n_2\ n_3]^\mathsf{T}$	the one signal that crosses from the allocation into the plant — a shaft speed, not a force
`Dynamics`, repeating $\mathbf{u} = n\|n\|$ then $\boldsymbol{\tau} = TK\mathbf{u}$	`otter_bow.m` rebuilding $\boldsymbol{\tau}$ for itself, inside the motion model
Are they the same? $\boldsymbol{\tau}_{demand} \Longleftrightarrow \boldsymbol{\tau}_{actual}$	the question §6-2 answers: identical below the ceiling, and not above it

> **★ The red question is the subject of §6-2**
>
> $\boldsymbol{\tau}_{actual} = \boldsymbol{\tau}_{demand}$ exactly, as long as no saturation acts anywhere — the allocation inverts what the dynamics then applies, and the round trip is the identity. The moment one shaft speed is clipped, the forward chain is evaluated at a different $\mathbf{n}$ from the one the inverse produced, and the two differ. §6-2 measures by how much, and shows that what leaks is not the demanded axis but the two that were asked to stay at zero.

What changed in the motion model

`_tools/otter_bow.m` is MSS `otter.m` copied and changed in four places, each marked in the file with `% ---- CHANGED (i of 4)`. Everything else — the rigid-body and added mass matrices, the Coriolis terms, the restoring forces, the damping,

`crossFlowDrag`, the kinematics — is character-for-character identical, because the hull is the same hull and the file has to show that.

	`otter.m`	`otter_bow.m`	Why
1 · input check	`if (length(n) ~= 2)`	`if (length(n) ~= 3)`	three shaft speeds in, and a seventh argument `bow = [l3 k3 n3_max]`
2 · thruster data	l_1, l_2, k_{pos}, k_{neg}, n_{max}, n_{min}	the same, plus `l3`, `k3`, `n3_max`	the bow unit's arm, coefficient and limit; one of each, because a tunnel thruster is symmetric
3 · thrust loop	`Thrust = zeros(2,1); for i = 1:1:2`	`Thrust = zeros(3,1); for i = 1:1:3`, with the third branch using `k3` and ± `n3_max`	derivation step 8
4 · the force vector	`tau = [T1+T2 0 0 0 0 -l1*T1-l2*T2]'`	`tau = [T1+T2; T3; 0; 0; 0; -l1*T1-l2*T2 + l3*T3]`	derivation step 10 — rows 2 and 6

The fourth change is the one the slide left as `Your code`. Its code is printed in §6-1 beside that slide; read row by row, it says this:

In the code	In the derivation	Note
`Thrust(1) + Thrust(2)`	$\tau_x = k_1 u_1 + k_2 u_2$	the stern pair in surge; the bow unit contributes nothing, its first column entry being zero
`Thrust(3)`	$\tau_y = k_3 u_3$	step 10 — the row that was zero in `otter.m`
`-l1 * Thrust(1) - l2 * Thrust(2)`	$+y_{pont} T_1 - y_{pont} T_2$	with the signed arms $l_1 = -y_{pont}$, $l_2 = +y_{pont}$ of `otter.m`: step 11, the form the slides' own listing uses
`+ l3 * Thrust(3)`	$+l_f F_B$	the moment the bow unit makes whether it was wanted or not — §6-2's N_s exists to subtract it
rows 3, 4, 5 left at zero	heave, roll, pitch	no thruster on this vessel acts in them

Three statements are verified rather than asserted, by `_tools/verify_otter_bow.m`:

Check	Why it must hold
with $n_3 = 0$, `otter_bow` equals `otter(x, n(1:2), …)` to rounding	the copy introduced no change outside the four marked ones
with $n_3 \neq 0$, it equals the previous week's superposition $\dot{\mathbf{x}} + M^{-1}[0,\ T_3,\ 0,\ 0,\ 0,\ l_3 T_3]^{\mathsf{T}}$	acceleration is affine in $\boldsymbol{\tau}$, so adding the force outside the call was exact — the residual is -8.9×10^{-16}
the saturation inside the model never binds first in normal running	the allocation clips the same quantity at the same value, so the model's own clip is a backstop and not a second controller

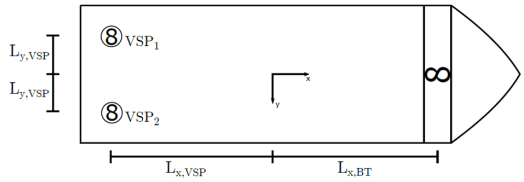
The second check is worth stating plainly, because it is easy to mistake this week's change for a bug fix. The superposition used until now was **not wrong**; it produced the same accelerations to machine precision. What it could not do is describe the vessel honestly. A motion model whose τ_y is identically zero is a model of an under-actuated vessel, however the sway force is added afterwards, and a student reading it would conclude that the Otter cannot go sideways.

Beyond three thrusters, in one page

The slides continue past this week's configuration to one this course does not build: two azimuth units and a bow tunnel thruster, from the CSEI model vessel. Week 7 asks the same question of four tilting units instead.

Another thruster configurations

➢ Azimuth thrusters + Bow thruster





$$\boldsymbol{\tau} = \mathbf{B}\mathbf{f}, \qquad \mathbf{f} = \mathbf{B}^{\dagger}\boldsymbol{\tau}, \qquad \mathbf{B}^{\dagger} = \mathbf{B}^{\top}(\mathbf{B}\mathbf{B}^{\top})^{-1}.$$

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ L_{y,VSP} & -L_{x,VSP} & -L_{y,VSP} & -L_{x,VSP} & L_{x,BT} \end{bmatrix} \qquad \mathbf{f} = \begin{bmatrix} f_{VSP1,x} & f_{VSP1,y} & f_{VSP2,x} & f_{VSP2,y} & f_{BT} \end{bmatrix}^{\top}$$

For the VSPs:

$$\begin{aligned} \tau_x &= u f_{max} \cos(\alpha), \\ \tau_y &= u f_{max} \sin(\alpha), \\ \tau_n &= \tau_x L_{y,VSP} - \tau_y L_{x,VSP}, \qquad (VSP_1) \\ \tau_n &= -\tau_x L_{y,VSP} - \tau_y L_{x,VSP}, \qquad (VSP_2) \end{aligned}$$

For the bow thruster:

$$\begin{aligned} \tau_x &= 0, \\ \tau_y &= u f_{max}, \\ \tau_n &= \tau_y L_{x,BT}. \end{aligned}$$

충남대학교 CHUNGNAM NATIONAL UNIVERSITY

130

Gauslaa, Elias. *Navigation, guidance, and control for autonomous docking of ships*. MS thesis. NTNU, 2020.

On the slide	Meaning
the plan view, $\mathbf{VSP_1}$ and $\mathbf{VSP_2}$ aft and a tunnel unit forward	five actuator degrees of freedom against three rows — over-actuated, unlike this week's square B
$L_{y,VSP}$, $L_{x,VSP}$, $L_{x,BT}$	the moment arms of derivation step 2, now three of them
the photograph	CSEI, the NTNU model vessel the configuration is taken from
$B^{\dagger} = B^{\mathsf{T}}(BB^{\mathsf{T}})^{-1}$	the pseudo-inverse that replaces the inverse once B stops being square — Week 7
the five-column B with $f_{VSP1,x}$, $f_{VSP1,y}$ paired per unit	an azimuth unit is carried as two orthogonal forces, not as a magnitude and an angle
the citation box, *Gauslaa 2020*	the source of equations (7.44)–(7.47) below

$$\boldsymbol{\tau} = B\mathbf{f}, \qquad \mathbf{f} = B^{\dagger}\boldsymbol{\tau}, \qquad B^{\dagger} = B^{\mathsf{T}}\left(BB^{\mathsf{T}}\right)^{-1}$$

$$B = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ L_{y,VSP} & -L_{x,VSP} & -L_{y,VSP} & -L_{x,VSP} & L_{x,BT} \end{bmatrix}, \qquad \mathbf{f} = \begin{bmatrix} f_{VSP1,x} \\ f_{VSP1,y} \\ f_{VSP2,x} \\ f_{VSP2,y} \\ f_{BT} \end{bmatrix}$$

Unit	Contribution
azimuth unit, magnitude u at angle α	$\tau_x = u f_{max} \cos\alpha$, $\tau_y = u f_{max} \sin\alpha$
VSP 1	$\tau_n = \tau_x L_{y,VSP} - \tau_y L_{x,VSP}$
VSP 2	$\tau_n = -\tau_x L_{y,VSP} - \tau_y L_{x,VSP}$
bow tunnel thruster	$\tau_x = 0$, $\tau_y = u f_{max}$, $\tau_n = \tau_y L_{x,BT}$

Every column is still read off the geometry by the rule of derivation step 2; there are simply five of them against three rows. The inverse is replaced by the Moore–Penrose pseudo-inverse, which among the infinitely many $\mathbf{f}$ satisfying $B\mathbf{f} = \boldsymbol{\tau}$ returns the one of least Euclidean norm.

That solution is not the end of the problem, and the source the slides draw on says so directly: the pseudo-inverse *does not take into account constraints like thruster saturation and forbidden sectors*. What it produces is a force per axis per unit, which still has to be turned into a command. For an azimuth unit that means a magnitude and an angle, and for a tunnel thruster a magnitude alone:

$$f_{cmd,i} = \sqrt{f_{i,x}^2 + f_{i,y}^2} \tag{7.44}$$

$$\alpha_i = \operatorname{atan2}(f_{i,y},\ f_{i,x}) \tag{7.45}$$

$$f_{cmd,i} = f_i \qquad \text{(tunnel thruster)} \tag{7.46}$$

$$u_i = \frac{f_{cmd,i}}{k_i} \tag{7.47}$$

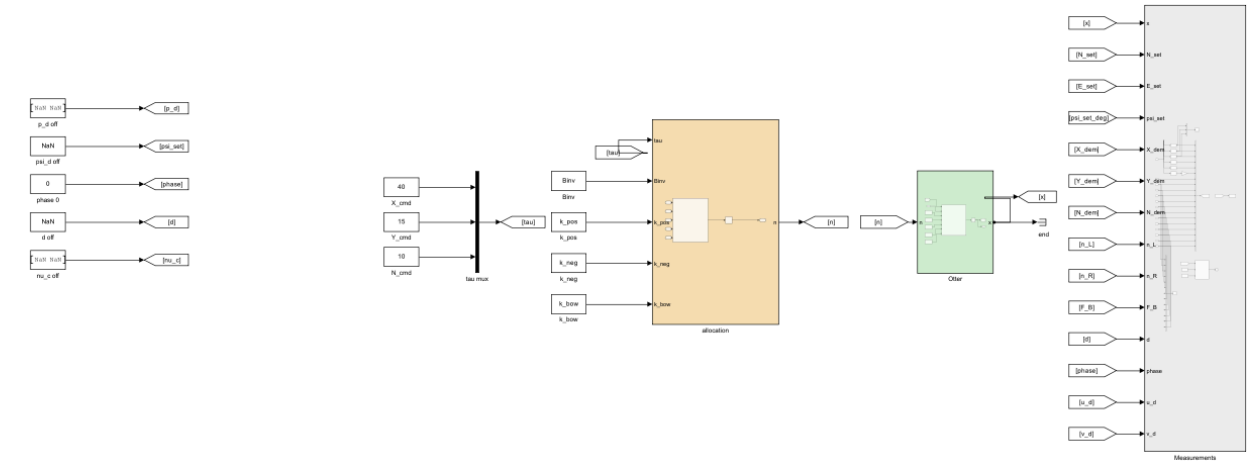
Equation	What it is for	Where it reappears
(7.44)	the magnitude an azimuth unit must produce	Week 7, as the quantity that saturates
(7.45)	the angle it must be turned to	Week 7, as the quantity with a mechanical limit that bounds the attainable set
(7.46)	a tunnel thruster has no angle to choose	this week: the bow unit's column is fixed, which is why B is square
(7.47)	force back to a normalised command	derivation step 8, $u_i = f_i/k_i$, identical in form

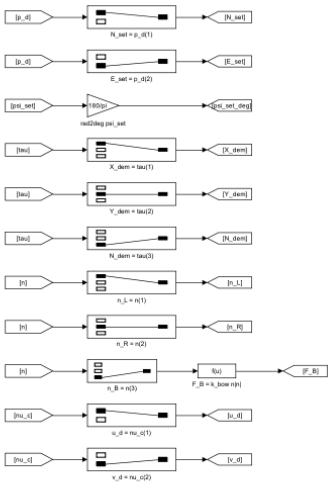
The numbering is that of the thesis the slides cite, and the equations are transcribed here rather than reproduced as an image because they are a textbook page rather than a figure. Nothing in this sub-section is used by any model of this week; it is here so that the step from a square B to a wide one is a continuation rather than a new subject.

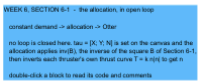
Experiment 6-1 · Three axes, commanded one at a time (20 min)

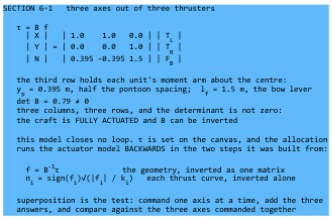
What it measures. Whether the algebra of B survives contact with the hull. A square B of rank 3 says the three axes are independent as a matter of linear algebra; it says nothing about whether the hull's inertia agrees.

The model. `W06_3_thrust.slx` — no feedback anywhere. Three Constants on the canvas set X, Y and N directly; the allocation block solves for the three thrusts; the hull integrates. The point of running it open loop is that nothing can hide a failure by correcting it.









In the figure	Meaning
`X_cmd`, `Y_cmd`, `N_cmd` on the left	the generalised force demand $[X\ Y\ N] = [40\ \ 15\ \ 10]$, read off the canvas, in newtons and newton-metres
`allocation`	solves $\boldsymbol{B}\mathbf{f} = \boldsymbol{\tau}$ and converts thrust to shaft speed; green-free because it is not the plant
`Otter` (green)	the 6-DOF hull from `otter.m`, with the bow force entering as an external force
`Measurements` (grey)	the 17-column log, and the live animation
the single wide signal returning from `Otter`	the 12-state vector $\mathbf{x}$; every subsystem that needs part of it selects internally

Opening and running.

```
W06_3_thrust_run
```

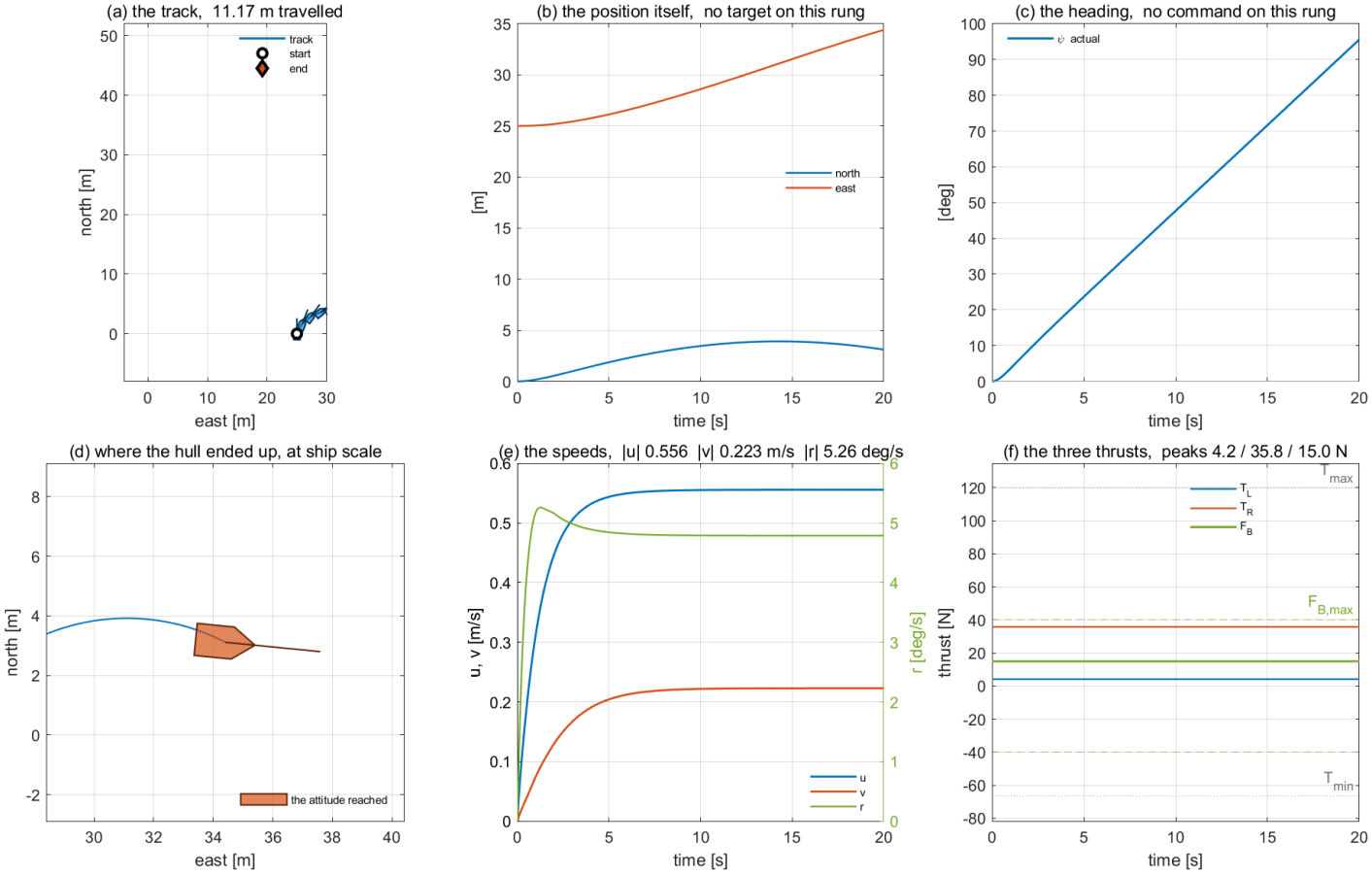
Expected output:

```
W06 Rung 3  the allocation, in open loop  (W06_3_thrust)
  canvas demand    X  40.00 N   Y  15.00 N   N  10.00 N m
  shafts           n_L   19.42   n_R   56.86 rad/s   F_B  15.00 N

  demand -> thrust, one axis at a time
    case            X [N]    Y [N]  N [N m]        n_L       n_R   F_B [N]   psi [deg]    |r|max
    surge only      40.00     0.00     0.00      42.49     42.49      0.00        0.00      0.000
    sway only        0.00    15.00     0.00     -66.48     50.70     15.00       -2.63      0.814
    yaw only         0.00     0.00    10.00      33.80    -44.32      0.00      119.30      6.074
    all three       40.00    15.00    10.00      19.42     56.86     15.00       95.62      5.263

  superposition   T(X) + T(Y) + T(N) - T(X,Y,N)
    residual       T_L  3.553e-15   T_R  0.000e+00   F_B  0.000e+00  N
    tau from B     X   40.000   Y   15.000   N   10.000  N, N m
```

The script runs the model once at the canvas values with the live view on, then re-runs it three times, writing one axis at a time into the same three Constants and reading the thrusts back. It restores the canvas afterwards, so the model on disk is unchanged.



Reading the figure against the derivation.

Where to look	What is there	Which line predicts it
`residual` line	3.6×10^{-15} N and two exact zeros	step 6 — B is invertible and constant, so the allocation is linear and the three single-axis solutions add to the three-axis one
`tau from B` line	$X = 40.000$, $Y = 15.000$, $N = 10.000$	the recovered force equals the demand on the canvas, to the printed precision
`sway only` row, $F_B = 15.00$	the bow force equals the sway demand exactly	step 4, second row of B: $Y = F_B$ and nothing else contributes
`sway only` row, $n_L = -66.48$ against $n_R = +50.70$	the stern pair oppose each other, and the magnitudes differ	$k_{neg} < k_{pos}$: the same thrust needs a larger shaft speed astern
`surge only` row, $n_L = n_R = 42.49$	the pair run together, and the moment arms cancel	step 4, third row: $a(T_L - T_R) = 0$ when the two are equal
`yaw only` row, $n_L = 33.80$ against $n_R = -44.32$	the pair oppose, and F_B stays at zero	the bow unit contributes to Y and N together; the stern pair alone can place a pure moment
panel (f) of the figure	the three thrusts flat for the whole run, far below T_{max}	nothing is closing a loop, so nothing changes the demand after $t = 0$
panels (b) and (c) of the figure	the position carries no target curve, the heading panel is titled "no command on this rung", and no quay wall is drawn	nothing is closing a loop, so there is no N_{set}, E_{set} or ψ_{set} to draw and no berth to approach; a dashed curve or a quay here would stand for a command that was never issued

What the figure says. The three axes are independent in force, and the residual line proves it to fifteen decimal places. They are not independent in motion. The `sway only` run commands $N = 0$ exactly, the delivered moment is 0.00 N·m, and yet the heading ends at -2.63 deg after twenty seconds, having passed through a turn rate of 0.814 deg/s. Nothing in B is wrong; what separates the axes is B, and B is not the only matrix in the problem.

Write the sway and yaw rows of the rigid-body equation with the surge row dropped, since $X = 0$ throughout:

$$\begin{bmatrix} M_{22} & M_{26} \\ M_{26} & M_{66} \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} Y \\ 0 \end{bmatrix}$$

Solving the second row for $\dot{r}$,

$$\dot{r} = \frac{-M_{26}\,Y}{M_{22}M_{66} - M_{26}^2} = \frac{-12.25 \times 15}{162.5 \times 42.6515 - 12.25^2} = -0.027099 \text{ rad/s}^2 = -1.553 \text{ deg/s}^2.$$

Symbol	Quantity	Value, source
M_{22}	sway inertia including added mass	162.5 kg, `otter_M.m`
M_{26}	sway–yaw added-mass coupling	12.25 kg·m, `otter_M.m`
M_{66}	yaw inertia including added mass	42.6515 kg·m^2, `otter_M.m`

The off-diagonal term M_{26} is the whole of the effect. A sideways force applied at the origin accelerates the hull in yaw even when the applied moment is zero, because the sway and yaw equations are coupled through added mass. The sign is right: a positive Y gives a negative $\dot{r}$, and the run ends at a negative heading. The magnitude is right too — the initial acceleration is 1.553 deg/s^2 and the rate peaks at 0.814 deg/s, roughly half a second's worth, because hydrodynamic damping starts opposing it immediately. Damping then removes the rate and leaves the **angle** behind: a rate that returns to zero does not return the angle it integrated.

> **★ What the allocation can and cannot promise**
>
> B guarantees that the commanded generalised force is the generalised force delivered. It promises nothing about the motion that force produces, because the map from force to acceleration is M^{-1} and M is not diagonal. Decoupling the axes in force is the allocation's job; decoupling them in motion is the controller's, and that is why Section 6-3 closes a loop on heading even when nothing commands a turn.

What to try	What to watch
double-click `Y_cmd` on the canvas, set it to 25, and run again	the heading drift grows with the force, in the ratio the equation above predicts
set `N_cmd` to -22.5 with `Y_cmd` at 15	the heading drift does not vanish — the leak is through M, not through an uncancelled moment
set `rp = [0 0 -0.35]'`, moving the origin onto the centreline station	M_{26} changes, and so does the drift; the coupling is a property of where the frame is
set `cfg = otter_config('base')` in `W06_0_setup` and rerun	the allocation fails on the sway row: a rank-2 B has no solution for $Y \neq 0$

> **✎ In class**
>
> - **Purpose**: make *fully actuated* a statement about a matrix, not a slogan.
> - **Point to**: the `residual` line, then the -2.63 deg of heading in a run that commanded $N = 0$.
> - **Ask**: if the force is exactly right and the moment is exactly zero, what turned the hull?
> - **Take away**: B separates the axes in force; M couples them in motion. Two different matrices, two different jobs.

6-2. Allocation, and the ceiling that binds first

This section answers: how is $B\mathbf{f} = \boldsymbol{\tau}$ actually solved, and what happens when the answer asks for more thrust than exists.

What is observed. The bow thruster is rated at 40 N. A pure sway demand of 40 N is therefore inside its rating, and the allocation produces it. Yet at $Y = 40$ N the hull also acquires 9.24 N of surge force and 3.65 N·m of yaw moment that nobody asked for, and the heading runs to $+31.25$ deg. At $Y = 35$ N none of that happens. Something binds between the two, and it is not the bow thruster.

The result. B is square and non-singular, so the solve is one matrix inverse, taken explicitly:

$$B^{-1} = \begin{bmatrix} \frac{1}{2} & -a & \frac{1}{2y_{pont}} \\ \frac{1}{2} & +a & -\frac{1}{2y_{pont}} \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0.5 & -1.898734 & 1.265823 \\ 0.5 & 1.898734 & -1.265823 \\ 0 & 1 & 0 \end{bmatrix}, \qquad a = \frac{l_f}{2y_{pont}}.$$

Read row by row, that inverse is three statements about the vessel, and those three statements are what makes the ceiling predictable:

$$F_B = Y, \qquad N_s = N - l_f F_B, \qquad T_L = \tfrac{1}{2}\left(X + \frac{N_s}{y_{pont}}\right), \qquad T_R = \tfrac{1}{2}\left(X - \frac{N_s}{y_{pont}}\right).$$

Nothing is solved by hand in the code — the block multiplies by B^{-1} — but the lines above are the same arithmetic written out, and each ceiling in step 5 is read off one of them.

Symbol	Quantity	Value, source
N_s	the yaw moment left for the stern pair after the bow unit has acted	N·m
a	the arm ratio $l_f/(2y_{pont})$, the $(1,2)$ entry of B^{-1}	1.898734 `verify_w06_actuation` check 4
T_{max}	maximum ahead thrust, one propeller	119.682 N $= k_{pos}n_{max}^2$
T_{min}	maximum astern thrust, one propeller	-66.708 N $= -k_{neg}n_{min}^2$
$F_{B,max}$	bow thruster rating	40 N, `W06_0_setup`
Y_{max}	attainable sway force at $X = N = 0$	35.1329 N, derived below

Derivation, one line at a time.

1. The second row of B is $[0\ \ 0\ \ 1]$. It has a single non-zero entry, so the sway demand determines the bow thrust on its own: $F_B = Y$. Equivalently, the third row of B^{-1} is exactly $[0\ \ 1\ \ 0]$ — `verify_w06_actuation` check 4. This is the only line of the allocation that does not share a degree of freedom with another.
2. The bow unit sits l_f forward, so producing F_B has already produced the moment $l_f F_B$, wanted or not. What remains for the stern pair is $N_s = N - l_f F_B$.
3. The first and third rows give the stern pair's sum and difference: $T_L + T_R = X$ and $y_{pont}(T_L - T_R) = N_s$. Solving the pair gives the two expressions above.
4. Specialise to a **pure sway demand**, $X = 0$ and $N = 0$. Then $N_s = -l_f Y$ and

$$T_L = -\frac{l_f}{2y_{pont}}\,Y = -aY, \qquad T_R = +aY, \qquad F_B = Y, \qquad a = \frac{1.5}{0.79} = 1.898734.$$

A sideways force of one newton costs the stern pair 1.899 N each, in opposite directions. The hull is inefficient in sway by almost a factor of four, and the reason is geometric: the bow unit's arm is nearly twice the stern pair's full separation.

5. Each of the three thrusts has a ceiling, and each ceiling becomes a ceiling on Y:

From	Constraint	Ceiling on Y
port propeller driven astern	$-aY \geq T_{min}$	$Y \leq 66.708/1.898734 = 35.1329$ N
starboard propeller driven ahead	$+aY \leq T_{max}$	$Y \leq 119.682/1.898734 = 63.0325$ N
bow thruster rating	$F_B = Y \leq F_{B,max}$	$Y \leq 40.0000$ N

6. The smallest of the three binds, so $Y_{max} = 35.1329$ N. **The limit on sideways force is not set by the bow thruster.** It is set by how hard one stern propeller can be driven in reverse, because a propeller astern is weaker than the same propeller ahead: $k_{neg} < k_{pos}$ in `otter.m`, and $|T_{min}|$ is only 56 % of T_{max}.
7. The three ceilings above are stated in newtons, but the quantity that actually saturates is a **shaft speed**, because that is what the allocation hands to the motion model and what each unit physically limits. The two statements are the same constraint in two coordinates: $T = k\,n|n|$ is strictly increasing in n, and the ratings are defined as $T_{max} = k_{pos}n_{max}^2$ and $T_{min} = -k_{neg}n_{min}^2$ — one definition, in `W06_vars`. Clipping n at n_{max} and clipping T at T_{max} therefore select the same set of attainable thrusts, and the ceiling on Y is unchanged by which coordinate the Saturation block sits in.

Demand	Shaft speeds the inverse asks for	After the clip	Which unit bound
$Y = 35$ N	$[-101.544,\ +77.446,\ +56.204]$ rad/s	unchanged	none — $35 < Y_{max}$
$Y = 40$ N	$[-108.555,\ +82.793,\ +60.084]$	$[-101.737,\ +82.793,\ +60.084]$	the port shaft, asked for 6.7 % more than it has
$Y = 50$ N	$[-121.368,\ +92.565,\ +67.176]$	$[-101.737,\ +92.565,\ +60.084]$	the port shaft and the bow unit

8. Past the ceiling the hull receives B times the thrusts the **clipped** shaft speeds produce, which is no longer the demand. With $Y = 40$: the bow unit is exactly at its rating, $T_R = a \times 40 = 75.949$ N is within T_{max}, and the port propeller delivers $k_{neg}n_{min}^2 = -66.708$ N instead of the -75.949 N asked for. The delivered force follows:

$$X = T_L + T_R = -66.708 + 75.949 = +9.241 \text{ N},$$

$$N = y_{pont}(T_L - T_R) + l_f F_B = 0.395(-142.657) + 1.5 \times 40 = -56.350 + 60 = +3.650 \text{ N} \cdot \text{m}.$$

The sway itself still arrives in full: $Y = T_3 = 40.000$ N, because the bow unit was not the unit that bound.

9. The same arithmetic at $Y = 50$, where the bow unit clips as well and the demanded axis can no longer be met either:

$$X = -66.708 + 94.937 = +28.229 \text{ N}, \qquad Y = 40.000 \text{ N}, \qquad N = 0.395(-161.645) + 60 = -3.850 \text{ N} \cdot \text{m}.$$

10. This is the measurable prediction, and it has two stages. **While the units that bind are the ones serving the demand only indirectly, the commanded axis still comes out in full and the excess appears in the axes that were supposed to stay at zero.** Only when the unit that carries the demanded axis itself clips does the demanded axis fall short. Saturation does not degrade gracefully in either stage, which is why an over-demand in sway shows up first not as a shortfall in sway but as an unexplained heading change.
11. Sweeping X instead of fixing it at zero gives the attainable set at $N = 0$, bounded above and below by

$$Y_{hi}(X) = \min\left(\frac{X/2 - T_{min}}{a},\ \frac{T_{max} - X/2}{a},\ F_{B,max}\right), \qquad Y_{lo}(X) = \max\left(\frac{X/2 - T_{max}}{a},\ \frac{T_{min} - X/2}{a},\ -F_{B,max}\right)$$

over $X \in [2T_{min},\ 2T_{max}]$. The set is a hexagon, and $X = 0$ is its **narrowest** place in Y. Asking for sway while already making way is cheaper than asking for sway from rest.

The whole allocation is one block, and it runs the actuator chain of §6-1 backwards in the three stages that chain had — geometry, thrust curve, signed square — in the reverse order. The code is reproduced here because it is what a student sees on double-clicking the block.

```
function n = allocate(tau, Binv, k_pos, k_neg, k_bow)
% Run the actuator model backwards: tau -> f -> u -> n.
%
%   forward    u_i = n_i|n_i|      f = K u        tau = B f
%   inverse    f = B^-1 tau        u = K^-1 f     n_i = sign(u_i) sqrt(|u_i|)
%
%   B    = [    1         1       0  ]    f   = [T_L; T_R; F_B]  [N]
%          [    0         0       1  ]    tau = [X; Y; N]        [N, N, N m]
%          [ y_pont   -y_pont   l_f ]
%
% B is square and non-singular (det B = 0.79), so the solution is unique and
% there is no null space to choose within. Binv comes from the setup script,
% which inverts the B that otter_B() derives from the column rule.
f = Binv * tau;

% K is diagonal, so K^-1 is three divisions. Which coefficient applies depends
% on the sign for a stern propeller, because a propeller pushing astern is
% less efficient than the same propeller pushing ahead. It does not depend on
% the sign for the bow unit: a tunnel thruster is symmetric.
k = [k_pos; k_pos; k_bow];
if f(1) < 0, k(1) = k_neg; end
if f(2) < 0, k(2) = k_neg; end

u = f ./ k;

% n_i = sign(u_i) sqrt(|u_i|). The absolute value is not decoration: slide 133
% writes sqrt(u_i), and without the |.| every astern command returns a complex
% shaft speed.
n = sign(u) .* sqrt(abs(u));
end
```

In the code	In the derivation	Note
`f = Binv * tau`	the three lines read off B^{-1}	one multiply replaces them; row 3 of B^{-1} being $[0\ \ 1\ \ 0]$ is `F_B = Y`
`k = [k_pos; k_pos; k_bow]`, then two sign tests	step 5's two different stern ceilings	the asymmetry $k_{neg} < k_{pos}$ is what makes Y_{max} bind on the astern side
`u = f ./ k`	$\mathbf{u} = K^{-1}\mathbf{f}$	§6-1 step 13 — K^{-1} acts on **forces**, which is why B^{-1} is applied first
`n = sign(u) .* sqrt(abs(u))`	§6-1 step 14	the inverse of $u_i = n_i\lvert n_i\rvert$, valid for both signs
the output is $\mathbf{n}$, not $\mathbf{f}$	—	the plant takes shaft speeds, so the allocation ends where the motion model begins and no force crosses the boundary
no `\`, no `pinv`	—	`verify_w06_actuation` check 5 confirms `Binv*tau` equals `B \ tau` to 3.6×10^{-14} N over 200 random demands

In the code	In the figure	Note
row 3 of B^{-1} acting on $Y = 25$ N	panel (b), $F_B = +25.0$ N	derivation step 1 — the second row of B has one entry
the $\pm a$ entries acting on the same Y	the ∓ 47.5 N arrows at the transom	derivation step 4, $aY = 1.8987 \times 25$
the implied $N_s = N - l_f F_B$	the $+37.5$ N·m label on the bow arm against the -37.5 N·m at the transom	the moment the bow unit has already made, and the moment that cancels it

> **⚠ An inverse is not an allocation**
>
> `allocate` returns the unique exact solution, and the Saturation block downstream clips the shaft speeds it produced. That is adequate here only because B is square: there is nothing to optimise, and B^{-1} is the answer rather than a choice of answer. With more actuators than degrees of freedom the inverse does not exist, $B^{\dagger}$ takes its place, and the clipped vector is no longer the best attainable approximation of the demand — the sub-section *Beyond three thrusters* quotes the source on exactly this point. Week 7 poses the allocation as a constrained minimisation instead, and its first experiment measures how far the clipped pseudo-inverse falls short.

Experiment 6-2 · Which limit binds first (15 min)

What it measures. The three ceilings of derivation step 5 and which of them binds (step 6), measured on a hull that is actually asked to go sideways; then the leak of derivation steps 7 to 9, measured above the ceiling.

The model. `W06_2_sway.slx` — the three loops of Section 6-3 with a demand that has no surge component at all: $N_d = 0$ and $E_d = 13$ on the canvas, due east from rest. The ceiling is reached by the loop itself rather than being commanded.

The sweep that follows borrows `W06_3_thrust.slx`, the open-loop rig of Experiment 6-1. It has to: the saturation sits **inside** the position loop, so a closed loop can never ask for more sway force than it is allowed to have. Only an open-loop rig can be given a demand past the ceiling and asked what comes out. The script restores the rig's canvas values when it is done.

Opening and running.

```
W06_2_sway_run
```

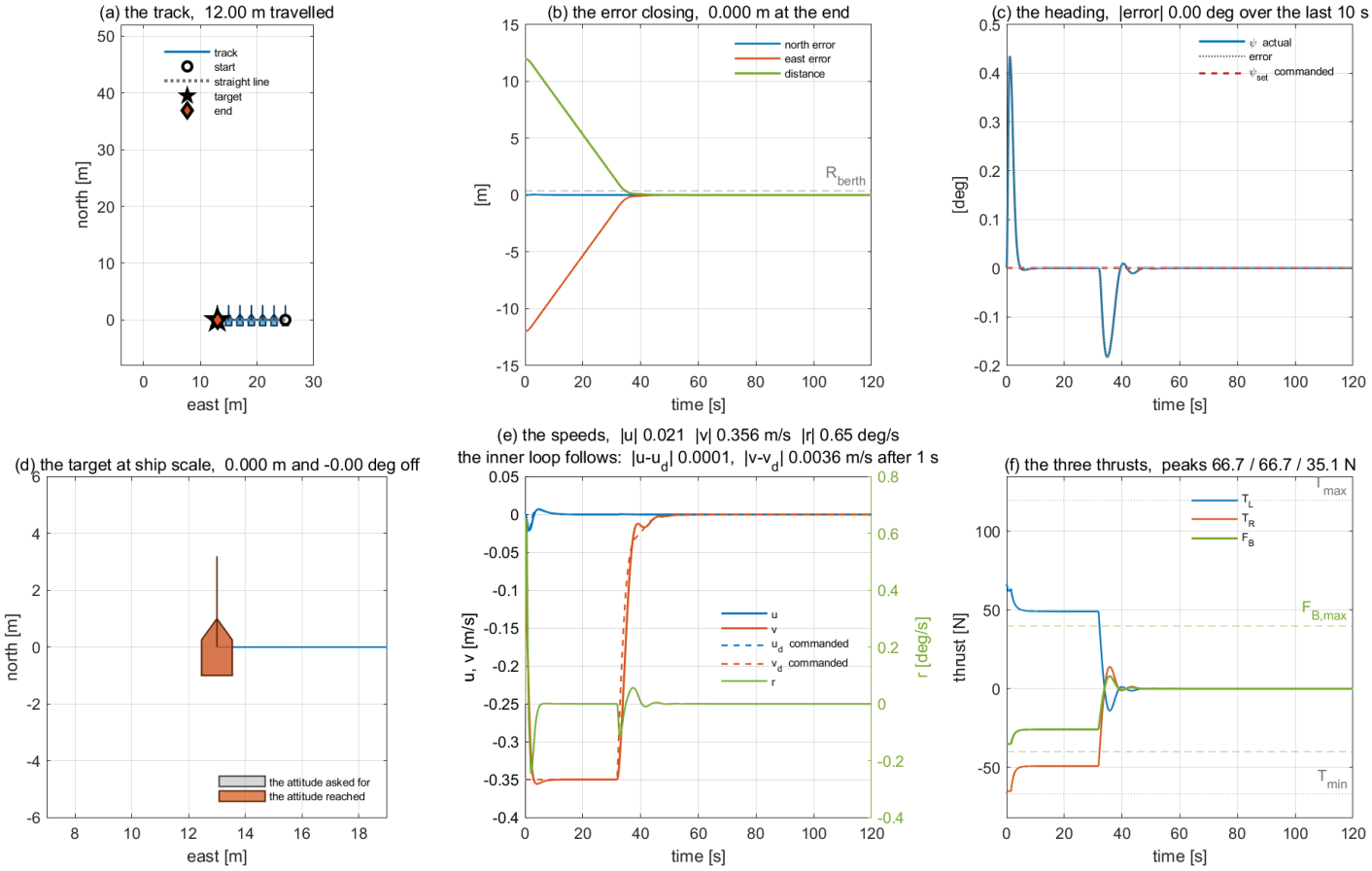
The script runs the model once with the live track on, prints the three ceilings beside the force the loop actually reached, then sweeps Y from well inside the ceiling to well past it.

Expected output:

```
W06_2_sway  —  120 s
  final position error   -0.000 m north,  0.000 m east   (0.000 m away)
  final heading error     -0.00 deg
  final speeds           u =  0.000, v = -0.000 m/s,  r =  0.000 deg/s
  peak sway speed         0.356 m/s
  saturated  X   0.0 % of the run,  Y   1.3 %
  peak demand            X =    3.05 N, Y =   35.13 N, N =    2.64 N m
  peak bow force         F_B =  35.13 N

W06 Rung 2  pure sway, and the limit that binds first  (W06_2_sway)
  demand          (  0.00,  13.00) m
  reached         ( -0.00,  13.00) m   final error  0.000 m
  peak surge       u  0.021 m/s        peak sway  v  0.356 m/s (limit 0.35)
  peak demand      X   3.05 N          Y  35.13 N (limit  35.13)
  peak bow force   F_B  35.13 N        rating  40.00 N
  three ceilings on Y, and which one binds
    stern ASTERN   |Y| <= |T_min|/a =  66.71/1.899 = 35.1329 N
    stern AHEAD    |Y| <=  T_max /a = 119.68/1.899 = 63.0325 N
    bow rating     |Y| <= F_B_max            = 40.0000 N
    binding, and so W06_vars.Y_max          = 35.1329 N
  Y saturated       1.3 % of the run

  sweeping Y across the ceiling, open loop  (X = 0, N = 0)
       Y [N]       T_L       T_R      F_B        X        Y        N   X pred   N pred  psi(end)
       20.00    -37.97     37.97    20.00     0.00    20.00     0.00     0.00     0.00     -2.84
       30.00    -56.96     56.96    30.00    -0.00    30.00    -0.00     0.00     0.00     -3.12
       35.00    -66.46     66.46    35.00     0.00    35.00     0.00     0.00     0.00     -3.23
       35.13    -66.71     66.71    35.13     0.00    35.13     0.00     0.00     0.00     -3.23
       40.00    -66.71     75.95    40.00     9.24    40.00     3.65     9.24     3.65     31.25
       50.00    -66.71     94.94    40.00    28.23    40.00    -3.85    28.23    -3.85    -37.58
```



Reading the figure against the derivation.

Where to look	What is there	Which line predicts it
panel (a)	a straight line due east, 12.00 m travelled against 12.00 m of straight-line distance	the demand has no north component, and the hull holds heading while translating sideways
panel (f)	T_L and T_R mirrored about zero, peaks 66.7 / 66.7 N, with F_B peaking at 35.1 N	step 4 — $T_L = -aY$ and $T_R = +aY$, so the stern pair is 1.899 times the sway force each while the bow unit carries Y itself
panel (f) against the banner	the stern pair reaches 66.7 N, which is $\lvert T_{min} \rvert$; the bow unit stops at 35.1 N, well short of its 40 N rating	step 6 — the actuator that saturates is at the *other* end of the hull from the one the demand names
the banner line `peak demand Y 35.13 N (limit 35.13)`	the loop asks for exactly Y_{max} and is held there 1.3 % of the run	step 6, measured rather than derived
panel (e)	$\lvert v \rvert$ peaks at 0.356 m/s against the 0.35 m/s inner-loop limit, $\lvert u \rvert$ at 0.021 m/s	the sway axis is the one working; the 6 mm/s overshoot is the velocity loop's own transient
sweep rows 20 through 35.13	$X = 0.00$ and $N = 0.00$ throughout	step 4 — inside the attainable set the solve is exact
the T_L column, rows 20 to 35.13	-37.97, -56.96, -66.46, -66.71: exactly $-1.8987\,Y$	step 4, $T_L = -aY$
the row at $Y = 35.13$	$T_L = -66.71$, the first row at which $T_L = T_{min}$	step 6 — this is Y_{max}, and the port propeller is the actuator that reaches its limit
the F_B column, row $Y = 40$	40.00 — the bow unit is at its rating but not over it	step 5, third row of the table: the bow rating is the *second* ceiling, not the first
`X pred` and `N pred` against `X` and `N`, rows 40 and 50	9.24 / 3.65 and 28.23 / -3.85, agreeing to two decimals	steps 7 and 8, computed from the clipped thrust vector
the `psi(end)` column	-3.2 deg below the ceiling, then $+31.25$ and -37.58 deg above it	step 9 — above the ceiling the leak dominates the M_{26} drift of Experiment 6-1 by an order of magnitude
the sign change in N between rows 40 and 50	$+3.65$ then -3.85 N·m	at $Y = 40$ only T_L is clipped; at $Y = 50$ the bow unit clips too, and the sign of the residual moment flips

What the figure says. Three numbers are available as a sway ceiling and only one of them is the answer. The bow thruster's 40 N rating is the number printed on the unit and it is not the limit; the limit is 35.13 N, set by a propeller at the other end of the hull being driven backwards. A specification written from component ratings would be wrong by 14 %, and wrong in the unsafe direction.

The leak is the more important half. Below the ceiling, two of three axes hold at exactly zero. Above it, the commanded axis continues to be satisfied — Y reads 40.00 in both over-demand rows — while the two axes that were supposed to be quiet are not. Saturation in a coupled allocation is not a soft degradation of the thing demanded; it is a transfer of error into the things not demanded.

What to try	What to watch
double-click `E_d` in `W06_2_sway.slx`, set it to `30`, and press Run	the hull still arrives, but Y now sits at 35.13 N for far longer: the ceiling does not change, only how much of the run is spent against it
raise `n_bow_max` to `95` in `W06_0_setup` and rerun — the rating it implies is $k_{bow}\, n^2 = 100.0$ N	Y_{max} does not move: it was never the bow unit that bound
set `k_neg = k_pos` in `otter_config` and rerun	the astern ceiling rises to 63.03 N and the bow rating becomes the binding one at 40 N
on `W06_3_thrust.slx`, set `X_cmd` to `60` and sweep `Y_cmd` again	the ceiling rises — step 10, $X = 0$ is the narrowest place in the hexagon
on `W06_3_thrust.slx`, set `Y_cmd` to `40` and `N_cmd` to `-3.65`	the heading stays near zero: the leak is a known quantity and can be pre-compensated, though only for one operating point

> ✎ **In class**
>
> - **Purpose**: separate a component rating from a vessel capability.
> - **Point to**: panel (f), where the bow trace stops at 35.1 N while the stern pair reaches 66.7 N; then the three ceilings in the banner; then the sweep row at $Y = 35.13$ where T_L reaches T_{min}.
> - **Ask**: the bow thruster is rated 40 N and is not saturated at $Y = 40$. What is?
> - **Take away**: the sway limit of this hull is a reverse-thrust limit at the stern. Past it, the error appears in surge and yaw, not in sway.

6-3. Three loops, and the one rotation between them

This section answers: given that every generalised force is now attainable, what structure turns a position error into that force.

What is observed. A position error is measured in NED, in metres north and metres east. A force is produced in the body frame, along x_b and y_b. Hand the northing error directly to the surge controller and the result is not a slow response or a noisy one: with the hull heading east, the vessel drives 90 degrees away from the target at full commanded speed, and the error grows. The controller is not mistuned. Two of its signals are expressed in different frames, and nothing in the loop says so.

The result. The chain from a target point to three shaft speeds is four stages, of which three are feedback loops and one is a change of frame:

$$\underbrace{\mathbf{p}_e = \mathbf{p}_d - \mathbf{p}}_{\text{NED}} \longrightarrow \underbrace{\mathbf{p}_b = R(\psi)^{\mathsf{T}}\mathbf{p}_e}_{\text{the rotation}} \longrightarrow \underbrace{\boldsymbol{\nu}_d = \mathrm{sat}(K_{p,pos}\,\mathbf{p}_b)}_{\text{position P}} \longrightarrow \underbrace{X, Y \text{ from a PI on } \boldsymbol{\nu}_d - \boldsymbol{\nu}}_{\text{velocity PI}}$$

with the yaw axis closed separately by

$$N = \mathrm{sat}\big(K_p\,\mathrm{ssa}(\psi_d - \psi) - K_d\,r\big).$$

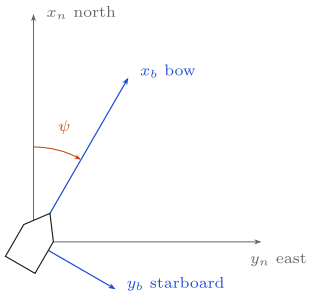
Symbol	Quantity	Value, source
$\mathbf{p}_d, \mathbf{p}$	target and measured position, NED	m
$\mathbf{p}_e$	position error in NED	m
$\mathbf{p}_b$	the same error, read in the body frame	m
$R(\psi)$	rotation from body to NED	—
$K_{p,pos}$	position proportional gain	0.30 s^{-1}, `W06_0_setup`
u_{lim}, v_{lim}	velocity command ceilings	0.80, 0.35 m/s
$K_{p,u}, K_{i,u}$	surge PI gains	200, 200
$K_{p,v}, K_{i,v}$	sway PI gains	150, 120
K_b	anti-windup back-calculation gain	1
K_p, K_d	heading P and D gains	300, 100
$\mathrm{ssa}(\cdot)$	smallest signed angle, $\mathrm{atan2}(\sin\cdot, \cos\cdot)$	rad

The rotation, and nothing more than the rotation

The body axes are the NED axes turned through the heading angle ψ. That single sentence is the whole content of $R(\psi)$, and the figure below is three ways of saying it.

(1) Two frames: one on the ground, one on the hull

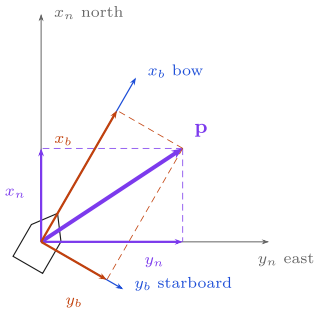
x_n (north) and y_n (east) are **fixed to the ground** and never move. x_b (bow) and y_b (starboard) are **fixed to the hull** and turn with it. The angle between them is the heading ψ



The hull frame is the NED frame **turned** by ψ. That is the whole of what a rotation matrix does

(2) One vector, two pairs of numbers

The purple arrow $\mathbf{p}$ is a **single** vector: neither its length nor its direction changes. What changes is **which ruler reads it**

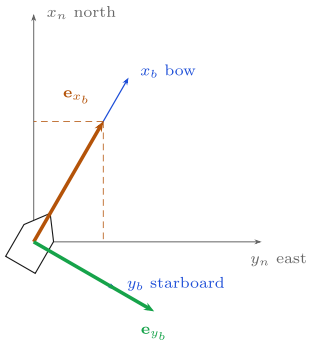


read with the hull ruler $\mathbf{p}^b = (2.000,\ 1.000)$

read with the ground ruler $\mathbf{p}^n = (1.232,\ 1.866)$

(3) The columns of R are nothing to memorise

A column of R is one **unit vector** of the hull frame, read with the ground ruler. The first column is $\mathbf{e}_{x_b}$, the second is $\mathbf{e}_{y_b}$



$\mathbf{e}_{x_b} = (\cos\psi,\ \sin\psi) = (0.866,\ 0.500)$

$\mathbf{e}_{y_b} = (-\sin\psi,\ \cos\psi) = (-0.500,\ 0.866)$

$$\underbrace{\begin{bmatrix} x_n \\ y_n \end{bmatrix}}_{\text{in NED}} = \underbrace{\begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix}}_{R(\psi)} \underbrace{\begin{bmatrix} x_b \\ y_b \end{bmatrix}}_{\text{in the hull}}$$

$$\begin{bmatrix} x_b \\ y_b \end{bmatrix} = R(\psi)^{\mathsf{T}} \begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} x_n \\ y_n \end{bmatrix}$$

Reading the other way is a transpose. $R^{-1} = R^{\mathsf{T}},\ RR^{\mathsf{T}} = I,\ \det R = 1.$ No inverse is ever computed: the rows and the columns are swapped

Numerical check at $\psi = 30°$: $R = \begin{bmatrix} 0.866 & -0.500 \\ 0.500 & 0.866 \end{bmatrix}$, each column has length 1, the two columns have inner product 0, $\det R = 1$. In MATLAB this is one line, `R = [cos(psi) -sin(psi); sin(psi) cos(psi)]`

In the figure	Meaning
left panel, two pairs of axes sharing an origin	$\boldsymbol{x_b, y_b}$ are $\boldsymbol{x_n, y_n}$ turned by $\boldsymbol{\psi}$ — nothing is moved, only re-labelled
middle panel, one arrow with two pairs of numbers	one physical vector; two coordinate pairs, because two sets of axes read it
right panel, the columns of $\boldsymbol{R}$	each column of $\boldsymbol{R}$ is one body unit vector written in NED coordinates
the equation band	$\boldsymbol{R^{-1} = R^{\mathsf{T}}}$, so reading in the other direction is a transpose and costs nothing

The columns are the part worth remembering, because they remove the need to remember anything else. The body $\boldsymbol{x}$-axis points along $\boldsymbol{(\cos\psi, \sin\psi)}$ in NED; the body $\boldsymbol{y}$-axis, ninety degrees to starboard of it, points along $\boldsymbol{(-\sin\psi, \cos\psi)}$. Stacking those two as columns,

$$\boldsymbol{R(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix},}$$

which maps body coordinates to NED. The chain needs the opposite direction, and because $\boldsymbol{R}$ is orthogonal the opposite direction is the transpose:

$$\mathbf{p}_b = R(\psi)^{\mathsf{T}}\,\mathbf{p}_e = \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} p_{e,N} \\ p_{e,E} \end{bmatrix}.$$

```
function p_b = to_body(p_e, psi)
% Rotate the position error from NED into the body frame.
%
%            [ cos(psi)  -sin(psi) ]
%   R(psi) = [ sin(psi)   cos(psi) ]     maps the BODY frame to NED,
%
% so the transpose maps NED to the body frame. R is orthogonal, so the
% transpose is the inverse and nothing is lost in the change of frame: the
% distance to the target is the same number in either one.
c = cos(psi);
s = sin(psi);
p_b = [  c*p_e(1) + s*p_e(2) ;      % 전진 방향의 오차 / along the bow
        -s*p_e(1) + c*p_e(2) ];     % 우현 방향의 오차 / to starboard
end
```

In the code	In the figure	Note
`c*p_e(1) + s*p_e(2)`	the projection of the arrow onto $\boldsymbol{x_b}$ in the middle panel	the first row of $\boldsymbol{R}^{\mathsf{T}}$, which is the first **column** of $\boldsymbol{R}$
`-s*p_e(1) + c*p_e(2)`	the projection onto $\boldsymbol{y_b}$	the second row of $\boldsymbol{R}^{\mathsf{T}}$
`c` and `s` computed once	—	`psi` is a signal, so the two trigonometric calls are the cost of the block; computing them once is the only optimisation present
no normalisation, no inverse	the equation band, $\boldsymbol{R^{-1} = R^{\mathsf{T}}}$	an orthogonal matrix needs neither

Two numbers make the block concrete. With a target $\mathbf{1}$ m north and $\mathbf{2}$ m east of the vessel and a heading of $\mathbf{30}$ degrees,

$$x_e = 0.866 \times 1 + 0.500 \times 2 = 1.866\ \mathrm{m}, \qquad y_e = -0.500 \times 1 + 0.866 \times 2 = 1.232\ \mathrm{m},$$

so the target is $\mathbf{1.866}$ m ahead and $\mathbf{1.232}$ m to starboard. The distance is unchanged: $\sqrt{1^2 + 2^2} = \sqrt{1.866^2 + 1.232^2} = 2.236$ m. `verify_w06_actuation` check 13 confirms $R^{\mathsf{T}}R = I$, $\det R = 1$ and these four numbers.

The failure named at the top of this section is the same arithmetic with $\psi = 90$ degrees and a target 10 m due north:

$$x_e = \cos\psi\, p_{e,N} + \sin\psi\, p_{e,E} = 0 \times 10 + 1 \times 0 = 0, \qquad y_e = -\sin\psi\, p_{e,N} + \cos\psi\, p_{e,E} = -1 \times 10 + 0 = -10.$$

There is nothing ahead and 10 m to **port**. Hand $p_{e,N} = 10$ to the surge PI instead of $x_e = 0$ and the surge controller drives the hull forward at the velocity ceiling — which, at this heading, is due east, 90 degrees from the target. The error does not merely fail to shrink; it grows. One matrix transpose stands between a working position controller and a vessel that runs away from its target.

Why three loops and not one gain

A single gain from position error to thrust is shorter to write and does not work. There are five separate reasons, and each of them names a thing the cascade has and the single gain does not.

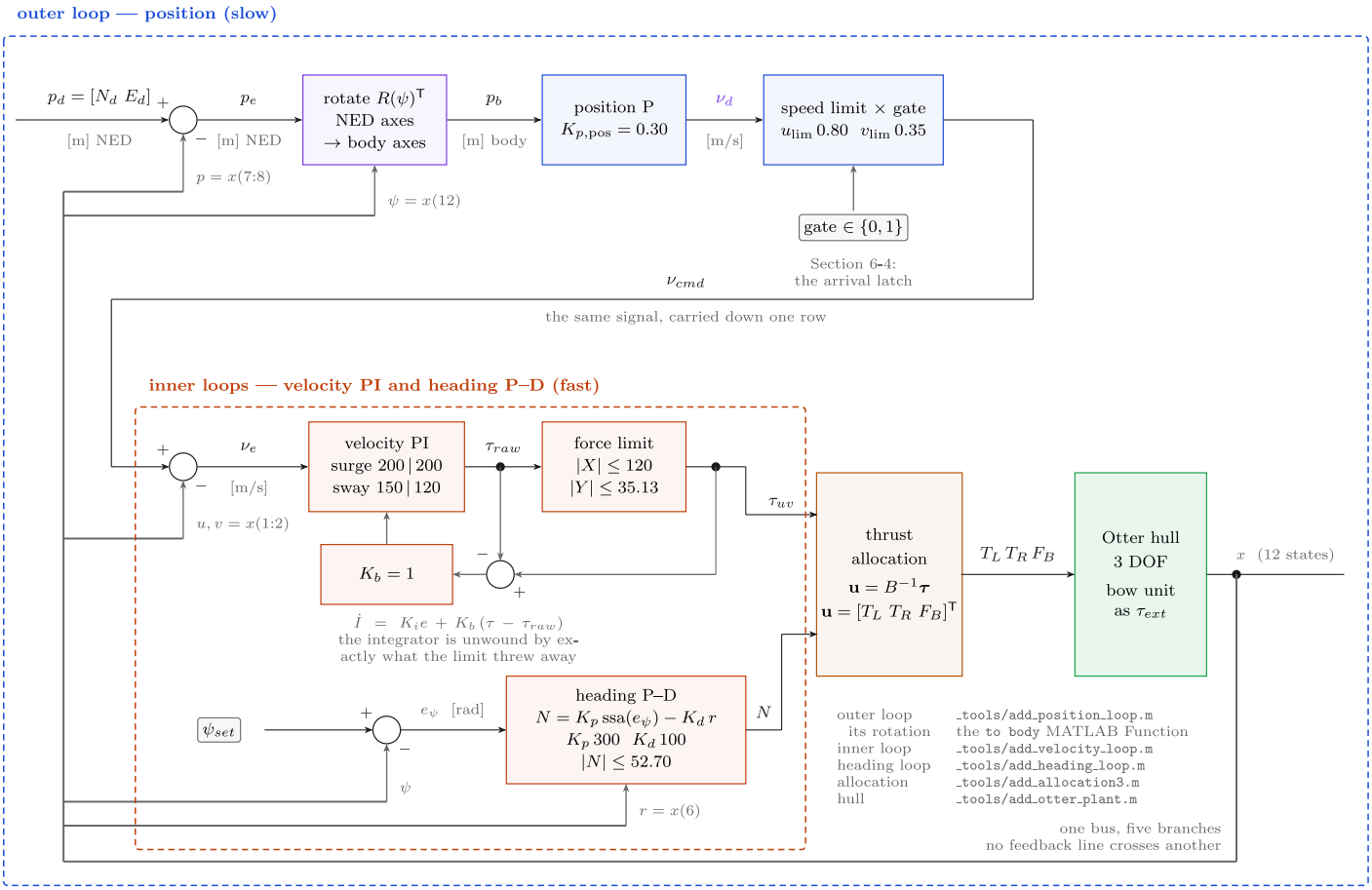
	What the cascade provides	What is lost without it
1	a place to clip the **velocity** command: $u_{lim} = 0.80$, $v_{lim} = 0.35$ m/s	a thrust clip limits force, not speed; a vessel 40 m from its target would arrive at whatever speed the drag happened to allow
2	an inner loop whose integral absorbs drag and current before the outer loop sees them	the outer loop would have to carry integral action on position, and a position integrator winds up over an entire transit
3	units that match their jobs: $K_{p,pos}$ is s^{-1}, a velocity per metre; $K_{p,u}$ is N/(m/s), a force per velocity error	one gain from metres to newtons mixes the two, and retuning for a heavier payload changes the braking distance
4	a controller that knows its own speed	thrust set from position error alone never measures velocity, so it cannot brake — it can only stop pushing
5	a heading loop with **no** integral term, so a steady yaw offset is accepted rather than fought	integral action on heading, against a yaw moment the allocation cannot supply, drives the stern pair to opposing saturation

The heading loop deserves the explicit mention in row 5. It is P-D and the D term is the **measured yaw rate**, not the derivative of the error: $N = K_p\,\mathrm{ssa}(\psi_d - \psi) - K_d\,r$. Differentiating the error would differentiate ψ_d as well, and in Section 6-5 ψ_d changes discontinuously at a phase boundary. Feeding back r costs nothing and cannot differentiate a step.

What the outer loop asks — *the berth is 12 m north and 3 m east from here: at how many m/s should the hull close that?* The answer is proportional to the distance that is left, and what leaves the block is a **velocity**, not a force

Why a rotation sits between them — the outer loop measures **metres north and east**; the inner loop holds **m/s ahead and to starboard**. The two frames differ by ψ, so one rotation has to stand between them

What the inner loops ask — *0.80 m/s ahead was ordered and 0.62 m/s is being made: how much harder?* P answers at once, I removes what is left, and the limit fixes what the three thrusters are actually asked for. The heading loop runs beside it on the same measurements



In the figure	Meaning
the top row, left to right	the outer position loop: error, rotation into the body frame, the P gain, the velocity cap
the labelled carriage return at the right of the top row	the **forward** continuation of the chain onto the second row, not a feedback path — the diagram is one chain folded, and the fold sits on the boundary between the outer and the inner loop
the bottom row	the inner velocity PI, the force cap, the allocation, and the shaft-speed conversion
the band below, labelled with K_p and K_d	the heading loop, running in parallel and producing N
the single vertical trunk on the left with five rightward branches	the 12-state measurement vector $\mathbf{x}$; each subsystem selects what it needs ($\psi = x(12)$, $r = x(6)$, and so on)
the short loop marked $K_b = 1$ around the velocity PI	the anti-windup back-calculation: $\mathrm{sat}(X) - X$ fed back into the integrator input
every gain triangle unsigned, every minus sign at a summing junction	the sign convention of this course: gains are positive and subtraction is visible

The anti-windup path is drawn rather than configured. A Simulink `Integrator` has a *Limit output* option that does the same job inside the block, and a diagram built that way is indistinguishable from one with no anti-windup at all. The back-calculation is three blocks and it is visible, which is the reason it is used here.

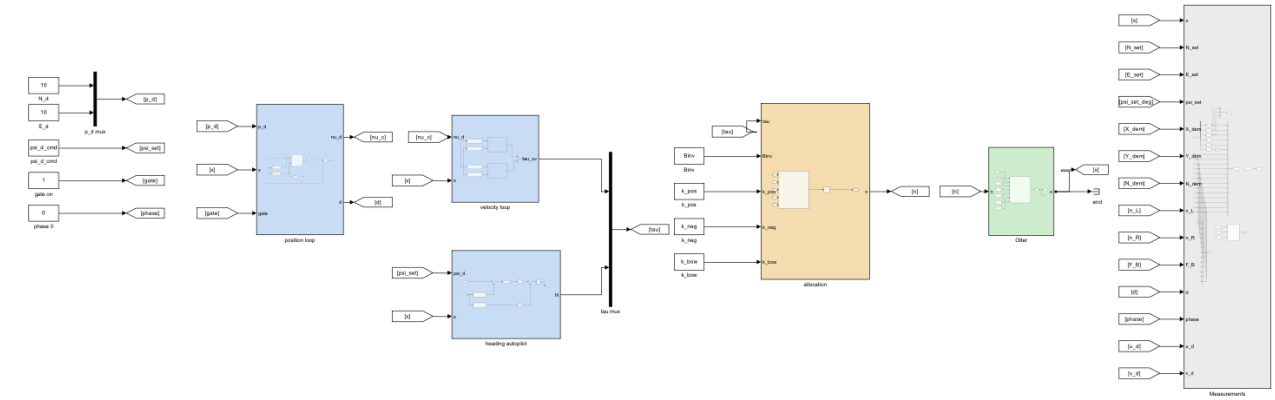
> **★ Where the time-scale separation comes from, and when it fails**
>
> A cascade is only valid when the inner loop settles much faster than the outer one commands. Here the outer loop's time constant is $1/K_{p,pos} = 3.33$ s and the velocity PI settles in a few tenths of a second, so the separation is roughly tenfold. Raise $K_{p,pos}$ towards 2 and the margin disappears: Experiment 6-4 measures 0.089 m of overshoot at $K_{p,pos} = 2.00$ against exactly zero at 0.15 and 0.30. The cascade does not fail loudly. It develops overshoot, and then ringing.

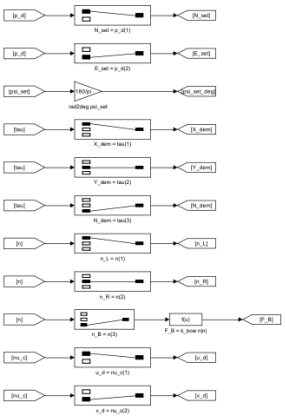
Experiment 6-3 · Holding a point (20 min)

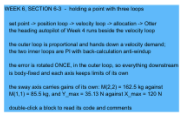
What it measures. Whether the three loops hold a fixed point, and what the third thruster is worth — measured by removing it and changing nothing else.

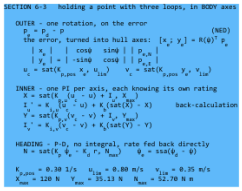
The model. `W06_1_point.slx`. The demand sits on the canvas as two Constant blocks, `N_d = 10` and `E_d = 10`; the start is $(0, 25)$ m and the commanded heading is 0 degrees throughout.

The second case changes nothing in the model. It sets `n_bow_max = 0` in the workspace before the run. The allocation's `shaft limit` block holds its bounds as expressions rather than numbers — `[ n_max; n_max; n_bow_max]` and `[ n_min; n_min; -n_bow_max]` — and those expressions are resolved from the workspace when the model compiles, so a zero collapses the bow unit's allowance to $[0, 0]$ rad/s. Every gain, every cap and every line stays where it was, and the same hull runs under-actuated.

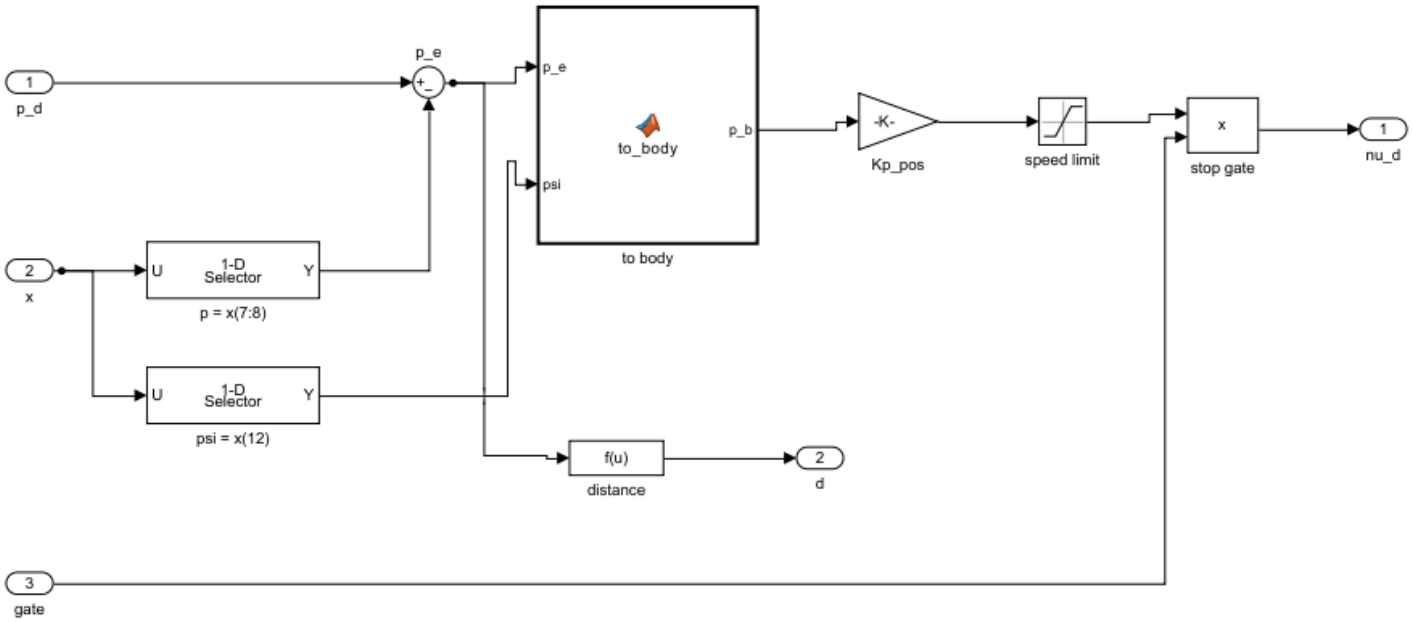








In the figure	Meaning
`position loop`	the rotation and the P gain, with the velocity cap on its output
`velocity loop`	two subsystems side by side, `u speed control` and `v speed control`, each a PI controller with the drawn anti-windup
`heading autopilot`	the P-D loop producing N
`allocation`	the one block of Section 6-2, with the `shaft limit` Saturation after it
the wide signal from `Otter` to the left edge	the measurement bus; the three controllers each select from it



In the figure	Meaning
the `to body` block	the MATLAB Function listed above — the only place a trigonometric function appears in the loop
the gain labelled `Kp_pos`	**0.30** s^{-1}; its value is a workspace variable name, so it is changed in `W06_0_setup` and nowhere else
the two Saturation blocks	the per-axis velocity caps u_{lim} and v_{lim}, which is where the asymmetry between ahead and sideways motion enters the controller

Opening and running.

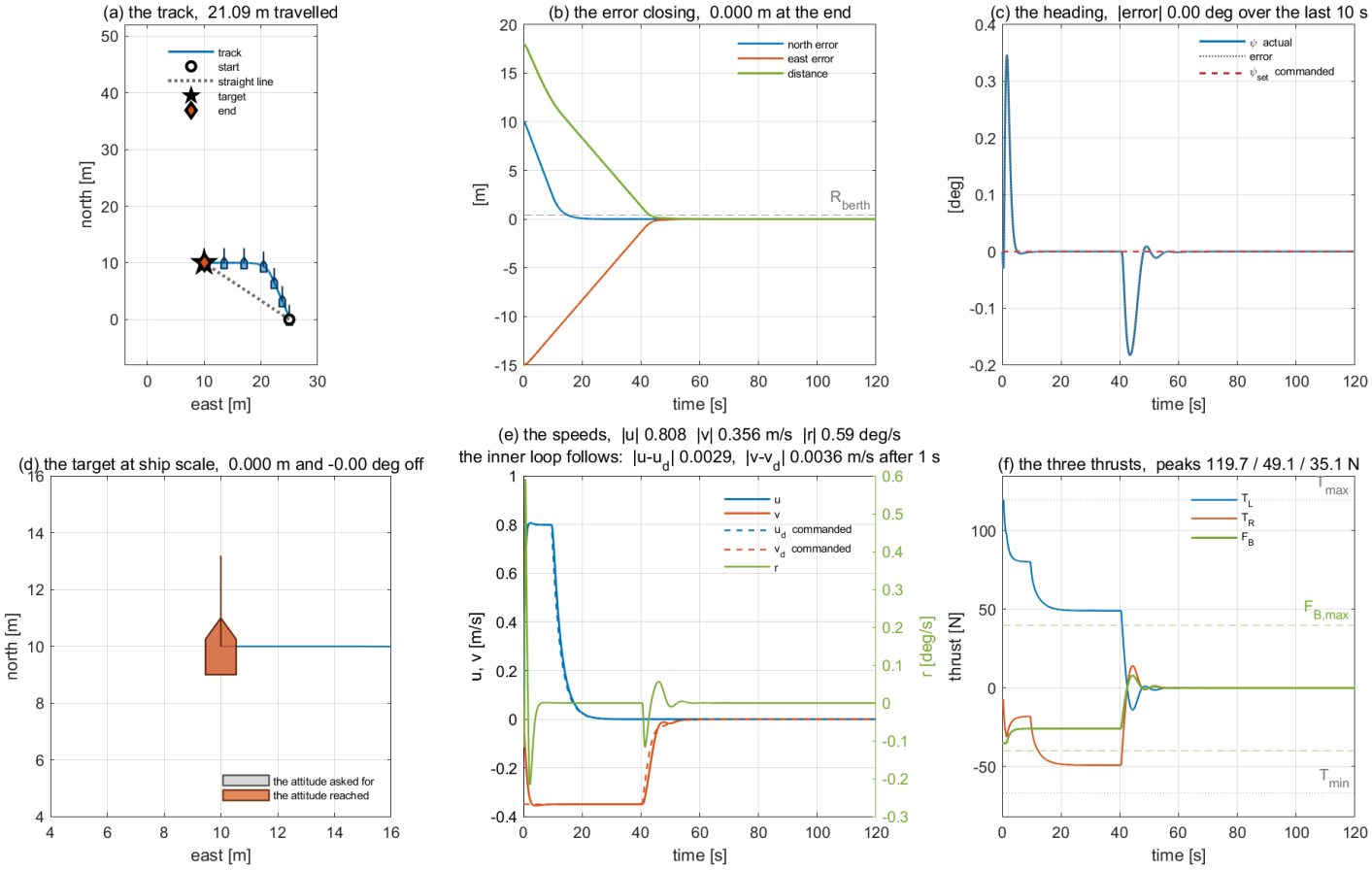
```
W06_1_point_run
```

Pressing **Run** on the canvas does the same thing: the model fills its own workspace, raises the live track window, and draws the panel when it stops. The script adds the comparison table — and only the script can, because the second case needs `n_bow_max = 0` in place before the model compiles.

Expected output:

```
W06 Rung 1  holding a point with three loops  (W06_1_point)
  demand          ( 10.00,  10.00) m   heading   0.00 deg
  reached         ( 10.00,  10.00) m   heading  -0.00 deg
  final error       0.000 m            heading  -0.00 deg
  within 0.5 m at   42.68 s
  peak speed       u 0.808 m/s (limit 0.80)   v 0.356 m/s (limit 0.35)
  peak demand      X 120.00 N (limit 120.00)   Y  35.13 N (limit  35.13)
  saturated        X  0.3 % of the run   Y  1.3 % - against a limit 3.42 times smaller

W06 Experiment 6-3  holding the point (10, 10) m, heading 0 deg
  case               t to 0.5 m [s]   d(end) [m]   psi error [deg]   max |v| [m/s]   max |Y| [N]
  three thrusters            42.68        0.000             -0.00           0.356         35.13
  bow unit off                 NaN       14.827             10.37           0.047         35.13
  F_B peak            with the bow unit  35.13 N,  without it   0.00 N
```



In the figure	Meaning
panel (a)	the track, start at $(0, 25)$ m and target at $(10, 10)$ m, with the hull drawn at intervals so the heading is visible along the path
panel (b)	the distance to the target against time; the curve that reaches the axis is the first case
panel (c)	the heading and the size of its error over the last ten seconds
panel (d)	the target at the scale of the hull, which is where a sub-metre error becomes legible
panel (e)	the three speeds $\lvert u\rvert$, $\lvert v\rvert$, $\lvert r\rvert$ against their caps. u and v are each drawn twice — solid for the measured speed and dashed for the command the position loop issued, in the same colour — so the panel shows the inner loop following the outer one, and its second title line gives the mean gap after the first second
panel (f)	the three thrusts T_L, T_R, F_B with their peaks marked

Reading the figure against the derivation.

Where to look	What is there	Which line predicts it
panel (a), the track	a curve, not a straight line, although the target is a straight 18.03 m away	the two caps differ by 3.42: the hull removes northing faster than easting, so the path bows
panel (a), the hulls along the path	every hull drawn parallel to the first	the heading loop holds 0 degrees while the position loop translates the vessel — the definition of full actuation in three degrees of freedom
panel (e), $\lvert u \rvert$ and $\lvert v \rvert$	both flat-topped, 0.808 and 0.356 m/s	both velocity caps bind; the small excess over 0.80 and 0.35 is the inner loop's finite bandwidth, since the cap is on the **command**
panel (f), F_B	flat at 35.13 N, below the 40 N rating	the ceiling of Section 6-2 is the stern pair's astern limit, not the bow unit's rating
`within 0.5 m at 42.68 s`	the approach takes most of a minute for 18 m	0.35 m/s is the binding cap over most of the path
`three thrusters`, `d(end) = 0.000` m	the error goes to zero, not to a small offset	the velocity PI's integral term removes the steady error; the position P loop needs none
`three thrusters`, `psi error = -0.00` deg	the heading is held throughout, although the hull moves sideways the whole time	the heading loop is independent of the position loop — they share only the allocation
`bow unit off`, `d(end) = 14.827` m	the vessel stops 14.8 m from its target	the start is $(0, 25)$ and the target $(10, 10)$: 10 m of northing, 15 m of easting. Heading is held at 0, so the hull can only move along north — the 10 m is removed and the 15 m is not
`bow unit off`, `max |v| = 0.047` m/s	essentially no sideways motion	rank 2 — the two-thruster B has image $Y = 0$, so there is no sideways force to produce one
`max |Y| = 35.13` N in **both** rows	the sway *demand* is at the Y_{max} cap in both cases	the demand is produced by the controller either way; with `n_bow_max = 0` the allocation cannot deliver it, and the column shows the demand, not the delivery
`F_B peak … without it 0.00 N`	the delivery, as opposed to the demand, is exactly zero	this is the column that confirms the second case actually ran under-actuated; the demand column alone could not say so
`bow unit off`, `t to 0.5 m = NaN`	the 0.5 m radius is never entered	the target is not in the reachable set of a rank-2 actuator configuration at fixed heading

What the figure says. The two rows differ in one number, `n_bow_max`, and the outcome differs by 14.8 m. That is the clearest available statement of what *fully actuated* buys: not a better response, but the existence of a response at all.

The residual is worth reading closely because it is not a tuning symptom. 14.827 m against the 15 m of easting the hull cannot remove is agreement to 1.2 %, and the remaining 0.17 m is accounted for by the 10.37 degrees of heading error at the end — a heading error that itself exists because the sway demand, which the allocation cannot meet, still perturbs the yaw axis through the clipped stern pair. Nothing here is mysterious, and nothing here is fixable by a gain.

What to try	What to watch
double-click `N_d` and `E_d` on the canvas of `W06_1_point.slx`, set them to $(10, 0)$	pure northing: $\lvert v \rvert$ in panel (e) stays near zero and F_B in panel (f) never leaves the axis, because a target dead ahead asks nothing of the third thruster
`n_bow_max = 0` in `W06_0_setup`, together with `psi_d_cmd = deg2rad(-56.3)`, pointing the bow at the target	the two-thruster hull now arrives: at a fixed, favourable heading an under-actuated vessel can reach a point — it simply cannot choose the heading and the point independently
`Kp_pos = 0.05` in `W06_0_setup`	the approach is slower and still exact; the integral lives in the inner loop, so outer-loop gain does not change the final error
`v_lim = 0.10` in `W06_0_setup`	the easting takes four times as long and the northing is unchanged — the caps are per axis, which is exactly the question Section 6-7 reopens
open the `position loop` subsystem, bypass the `to body` block and wire $\mathbf{p}_e$ straight through	at `psi_d_cmd = 0` nothing changes, because $R(0) = I$; set `psi_d_cmd = deg2rad(90)` and the failure of the opening paragraph appears in full

✎ In class

- **Purpose**: establish that the frame conversion is a structural requirement, not a refinement.
- **Point to**: panel (a) first — every hull along the path drawn parallel, which is full actuation in one picture — then the 14.827 m against the 15 m of easting, then the last row of the "What to try" table.
- **Ask**: with the rotation removed and the heading at zero, the model behaves identically. Why is that not evidence that the rotation is unnecessary?
- **Take away**: three loops, one rotation, and a velocity cap that only a cascade has a place for.

6-4. Stopping, and who declares arrival

This section answers: how far does the vessel travel after the target is set, and what test says it has arrived.

What is observed. With $K_{p,pos} = 0.30$ s^{-1} and a target 12 m ahead, the vessel reaches the 0.5 m radius at 17.00 s and comes to rest with no overshoot at all. With $K_{p,pos} = 2.00$ it reaches the same radius at 14.80 s — a gain of 2.2 s for a sevenfold gain increase — and overshoots by 0.089 m. The braking behaviour of the loop is not set by the inner controller or by the hull. It is set by one outer gain, and the relationship is simple enough to write down before the run.

The result. In the regime where the velocity command is saturated, the approach is at constant speed and the distance at which braking begins is

$$d_{brake} = \frac{u_{lim}}{K_{p,pos}}.$$

Inside that distance the command is linear in the error, the loop is first order, and

$$d(t) = d_{brake}\, e^{-K_{p,pos}(t - t_{brake})}, \qquad \tau = \frac{1}{K_{p,pos}}.$$

Symbol	Quantity	Value, source
d_{brake}	distance at which the velocity cap stops binding	m
τ	time constant of the linear approach	$1/K_{p,pos}$ s
R_{berth}	arrival radius	0.40 m
ψ_{tol}	heading tolerance	5 deg
v_{tol}	sway-speed tolerance	0.05 m/s
r_{tol}	yaw-rate tolerance	1.0 deg/s
ω	effective bandwidth of the inner velocity loop	back-solved from the measurement, step 4
λ	slow root of the cascade; $-1/\lambda$ is the decay actually observed	measured, Experiment 6-4

Derivation, one line at a time.

1. The position loop commands $u_d = \mathrm{sat}(K_{p,pos}\, x_e)$ with the ceiling u_{lim}. Far from the target the saturation binds, so $u_d = u_{lim}$ and the approach is at constant commanded speed regardless of distance.
2. The saturation stops binding when $K_{p,pos}\, x_e = u_{lim}$, which is at $x_e = u_{lim}/K_{p,pos}$. That is the braking distance: $0.80/0.30 = 2.67$ m at the nominal gain.
3. Inside that distance, and **assuming the inner loop tracks its command instantly**, $\dot{x}_e = -u_d = -K_{p,pos}\, x_e$. The solution is exponential with time constant $1/K_{p,pos} = 3.33$ s. That assumption is no longer taken on trust: the log carries the command beside the response, u_d and v_d in columns 18 and 19 against u and v in columns 1 and 2, so the gap between them is a measured quantity. On this rung it peaks at $|u - u_d| = 0.800$ m/s at $t = 0$ — which is the command step itself, taken while the hull is still at rest, and not a tracking lag — falls below 0.02 m/s at $t = 1.38$ s, and averages 0.0028 m/s over the rest of the run. The assumption is therefore wrong for the first second and a half and very nearly right afterwards, which is why step 3 predicts the approach closely but not exactly, and why the next step still has to price the lag.
4. The inner loop is not instant. Model it as first order with bandwidth ω, so that $\dot{u} = \omega\,(u_d - u)$ alongside $\dot{x}_e = -u$. Eliminating u gives a second-order error equation:

$$\ddot{x}_e + \omega\,\dot{x}_e + \omega K_{p,pos}\, x_e = 0, \qquad s^2 + \omega s + \omega K_{p,pos} = 0.$$

The sum of the two roots is $-\omega$ and their **product** is pinned at $\omega K_{p,pos}$. As the fast root moves out towards $-\omega$, the product forces the slow root to move *away* from $-K_{p,pos}$, not towards it. The near-field decay is therefore **faster** than step 3 predicts, and measuring the slow root λ measures the inner loop:

$$\omega = \frac{\lambda^2}{\lambda - K_{p,pos}}.$$

This is the step that turns a disagreement into a number. Steps 5 and 6 are written with both constants, the ideal $1/K_{p,pos}$ and the measured $-1/\lambda$, so the experiment can say which one the hull obeys.

5. The exponential is the prediction that makes the arrival radius a *time*. Entering a radius R takes

$$t(R) = t_{0.5} + \tau\, \ln\frac{0.5}{R},$$

measuring from the time the 0.5 m radius is crossed. Halving the radius costs $\tau \ln 2$, independent of which radius is halved: 2.31 s with the ideal $\tau = 1/K_{p,pos}$, and less than that with the measured one.

6. Step 5 is also the argument against a tight arrival radius. Going from $R = 0.50$ m to $R = 0.05$ m costs $\tau \ln 10$ — 7.68 s on the ideal constant — for a vessel whose hull is 2.0 m long. At some radius the gain in accuracy is smaller than the drift the extra time allows.
7. Arrival therefore cannot be a distance test alone. A vessel inside R_{berth} while still moving sideways at 0.3 m/s has not arrived. The test used here is a conjunction of four bounded conditions:

$$\|\mathbf{p}_d - \mathbf{p}\| \le R_{berth} \ \wedge \ |\mathrm{ssa}(\psi - \psi_{berth})| \le \psi_{tol} \ \wedge \ |v| \le v_{tol} \ \wedge \ |r| \le r_{tol}.$$

8. Each of those four conditions flickers at its own boundary, and the conjunction flickers whenever any one of them does. The measurable prediction is a count: the sway-speed test changes state **twice** on a clean run, so a controller that switched on the raw conjunction would command the moored state, leave it, and command it again. The remedy is a latch, and Section 6-5 builds it.
9. The conjunction also has to survive a disturbance the control law cannot remove. The position loop is proportional — there is no integrator on $\mathbf{p}_e$ — so a steady current leaves a steady offset, and the first of the four terms simply never becomes true. A single test on $|v|$, or on distance alone, would have declared arrival anyway. The prediction is therefore a *failure*: with a 0.3 m/s cross-current the first term is true for 0 % of the run and the conjunction never latches.

Experiment 6-4 · Stopping (20 min)

What it measures. The braking distance of step 2, the two time constants of steps 3 and 4, the radius–time law of step 5, the state-change count of step 8, and the latch failure of step 9.

The model. `W06_4_stop.slx`. The canvas demand is $(12, 25)$ m and the vessel starts at $(0, 25)$ m with $\psi_d = 0$, so the motion is 12 m of pure surge: no sway term, no heading transient, and the one-dimensional derivation above applies without qualification. The hull, the three loops and the allocation are the same as rung 1 — only the demand on the canvas differs.

The arrival test of steps 8 and 9 needs two-dimensional motion and a current, so the last block of the script borrows **rung 1's model**, `W06_1_point.slx`, as an instrument. The current reaches the plant the same way the bow rating reached it in Experiment 6-3: `add_otter_plant` places V_c and β_c on the canvas as Constant blocks whose *Value* is the variable **name**, resolved from the workspace when the model compiles. Setting `V_c = 0.3` before `sim` therefore changes the run with no edit to any block, and the script restores it afterwards.

Opening and running.

```
W06_4_stop_run
```

Pressing **Run** on the canvas gives the first block and the figure; the sweep and the two arrival tables are the script's, because each of their rows needs a workspace value in place before the model compiles.

Expected output:

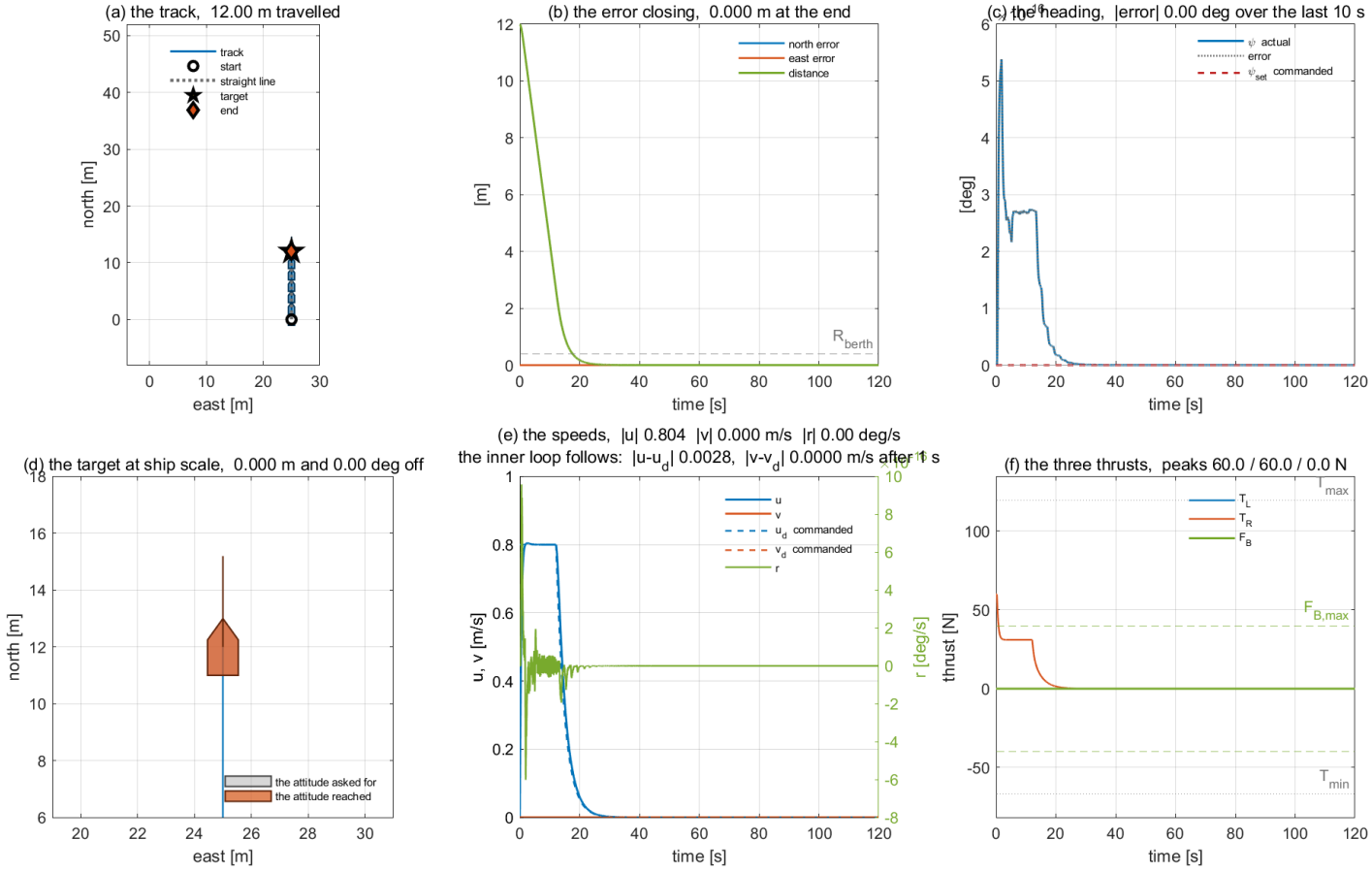
```
W06 Rung 4  stopping, and who declares arrival  (W06_4_stop)
  FAR   measured slope of d(t)   -0.7828 m/s   (-u_lim = -0.8000)
  the two regimes meet at        d_brake = u_lim/Kp_pos =  2.67 m   at  12.09 s
  NEAR  1/Kp_pos (ideal inner loop)   3.33 s   measured time constant  2.91 s
        the cascade is 2nd order: s^2 + w s + w Kp_pos,  w =  2.71 rad/s
        halving time             measured  1.00 -> 0.50 m   2.01 s
                                 measured  0.50 -> 0.05 m   2.02 s per halving
        the two agree to  0.4 %  -- the decay IS exponential
  reached 0.50 m at  16.98 s,  0.05 m at  23.69 s
  once inside R_berth = 0.40 m, d never rises above 0.3991 m again
  final      error 0.0000 m,  u 0.0000 m/s,  v 0.0000 m/s

  sweeping Kp_pos  (overshoot = how far north of the target the hull ever got;
                    tau is withheld on a row that overshoots: not one exponential)
      Kp_pos   u_lim/Kp [m]   measured [m]   1/Kp [s]    tau [s]  overshoot [m]   t to 0.5 m
        0.15           5.33           5.12       6.67       6.26          0.000        23.60
        0.30           2.67           2.51       3.33       2.91          0.000        17.00
        2.00           0.40           0.34       0.50        NaN          0.089        14.80

  when each radius is reached  (nominal Kp_pos = 0.30)
       R [m] first entry [s]    |U| there   Kp_pos R [m/s]    pred 1/Kp [s]     pred tau [s]
        2.00          12.97        0.679            0.600            12.36            12.95
        1.00          14.97        0.345            0.300            14.67            14.97
        0.50          16.98        0.171            0.150            16.98            16.98
        0.20          19.65        0.068            0.060            20.04            19.65
        0.05          23.69        0.017            0.015            24.66            23.69

  the four arrival terms, point hold at (10, 10) m,  V_c = 0.0 m/s due east
    term                     true [% of run]   first true [s]    changes
    d < R_berth                         63.7            43.54          1
    |psi err| < psi_tol               100.0             0.00          0
    |v| < v_tol                        62.0             0.00          2
    |r| < r_tol                       100.0             0.00          0
    all four at once                   61.8 % of the run
    at the end             d =  0.000 m,  v = -0.0000 m/s,  max |Y|  35.13 N of 35.13 N

  the four arrival terms, point hold at (10, 10) m,  V_c = 0.3 m/s due east
    term                     true [% of run]   first true [s]    changes
    d < R_berth                         0.0              NaN          0
    |psi err| < psi_tol               100.0             0.00          0
    |v| < v_tol                         0.6             0.00          1
    |r| < r_tol                       100.0             0.00          0
    all four at once                    0.0 % of the run
    at the end             d =  2.153 m,  v = -0.1080 m/s,  max |Y|  35.13 N of 35.13 N
```



In the figure	Meaning
panel (a)	the track, a straight 12.00 m from $(0, 25)$ to $(12, 25)$ m, with the hull drawn along it
panel (b)	the north error, the east error and the distance, against time, with R_{berth} drawn across. The east error is flat at zero on this rung, so the distance and the north error are the same curve — and the two regimes of the derivation are its straight segment and the curve that follows
panel (c)	the commanded heading, the heading, and the error between them, titled with the mean $\lvert$error$\rvert$ over the last ten seconds; all three are flat at zero on a pure-surge run
panel (d)	the target at the scale of the hull: two outlines, the attitude asked for in grey and the attitude reached in orange, with the track through them
panel (e)	u and v on the left axis, r on the right; only u leaves the axis on this rung. The dashed curve in each colour is that axis's command, u_d or v_d, so the distance between dashed and solid is the assumption of step 3 made visible. The second title line gives it as a number: $\lvert u - u_d \rvert$ 0.0028 m/s on average after the first second, against a peak of 0.800 m/s at $t = 0$ that is the command step itself
panel (f)	the three thrusts T_L, T_R, F_B against the ceilings of Section 6-2, on the one scale every rung shares

Reading the figure against the derivation.

Where to look	What is there	Which line predicts it
panel (b), the first segment	a straight line, slope -0.7828 m/s against $-u_{lim} = -0.8000$	step 1; the approach is at constant speed, and the 2 % shortfall is the inner loop tracking a saturated command
panel (b), the knee	the straight line bends at 12.09 s	step 2; the saturation stops binding at $d_{brake} = 0.80/0.30 = 2.67$ m
`u_lim/Kp` against `measured`, all three sweep rows	5.33 / 5.12, 2.67 / 2.51, 0.40 / 0.34	step 2; the measured value is 4–15 % short because the hull needs a moment to come off the velocity cap. The measurement is taken where $\lvert U \rvert$ first falls below $0.99\,u_{lim}$ — one stated rule, applied to every row
`1/Kp` against `tau`, nominal row	3.33 s predicted, 2.91 s measured	step 3 is **wrong by 13 %, and in the direction step 4 requires**: the slow root is faster than $-K_{p,pos}$, not slower
`w = 2.71 rad/s`	the inner loop's bandwidth, never measured directly	step 4, back-solved from $\lambda = -1/2.91$ through $\omega = \lambda^2/(\lambda - K_{p,pos})$ — the outer loop's decay is an instrument for the inner one
the two halving times	2.01 s and 2.02 s, agreeing to 0.4 %	step 5; one halving costs the same wherever it is taken, which is what *exponential* means. $2.91 \ln 2 = 2.02$ s, against 2.31 s on the ideal constant
`tau` on the $K_{p,pos} = 2.00$ row	`NaN`, not a number	step 4 at its limit: that row overshoots, so the near field is no longer a single exponential. A log fit still returns a value there; it measures nothing, and so it is withheld rather than printed
`overshoot`, last row only	0.089 m at $K_{p,pos} = 2.00$, zero at the other two	$\omega = 2.71$ rad/s against a demanded λ of -2.00: the roots are no longer separated, the damping of step 4's quadratic falls below one, and the hull passes the target
`t to 0.5 m` across the three gains	23.60, 17.00, 14.80 s	sevenfold gain for 8.8 s, and only 2.2 s of it past the nominal — most of the transit is at the velocity cap, where gain does nothing at all
`|U| there` against `Kp_pos R`	$0.679/0.600$, $0.345/0.300$, $0.171/0.150$, $0.068/0.060$, $0.017/0.015$	step 3; the speed at radius R is $K_{p,pos}R$, over a factor of forty in R
`pred 1/Kp` against `first entry`	drifts from 0.6 s early at $R = 2$ m to 1.0 s late at $R = 0.05$ m	step 3 alone, accumulating its 13 % error over two decades of radius
`pred tau` against `first entry`	$12.95/12.97$, $14.97/14.97$, $16.98/16.98$, $19.65/19.65$, $23.69/23.69$	step 5 with the **measured** constant: exact to 0.02 s, which is one sample of $h = 0.02$ s. The same 0.02 s separates `t to 0.5 m` $= 17.00$ in the sweep from `reached 0.50 m at 16.98` above it — the radius table interpolates between samples and the sweep does not
`d never rises above 0.3991 m again`	the approach is monotone inside the arrival radius	there is no overshoot at the nominal gain, so the distance term, once true, stays true — which is why its `changes` count is 1
`|v| < v_tol`, `changes = 2` at $V_c = 0$	the sway test leaves and re-enters its bound	step 8 — this is the chatter the latch exists to prevent, and the only term in the table that does it
`all four at once`, 61.8 % against `d < R_berth` at 63.7 %	the conjunction is true for less of the run than its loosest term	the gap is the sway excursion: a distance-only test would have declared arrival while the hull was still moving sideways
the $V_c = 0.3$ block, `d < R_berth` true 0.0 % of the run, `first true` = `NaN`	arrival is never declared, at any instant of the run	step 9; and `max |Y| 35.13 N of 35.13 N` says why — the sway demand is pinned at the Y_{max} of Section 6-2 and the current still wins

What the figure says. Two constants describe the whole approach, and the experiment separates them. The far field belongs to u_{lim} and the near field to $K_{p,pos}$, they meet at $u_{lim}/K_{p,pos}$, and the near field is exponential to 0.4 % over a factor of

twenty in radius. That much the one-line derivation of step 3 gets right.

What it gets wrong is the value, by 13 %, and the sign of the error is the interesting part. A cascade whose inner loop is not infinitely fast decays *faster* than the outer gain alone predicts, because the product of the two roots is fixed at $\omega K_{p,pos}$ while their sum is $-\omega$. Reading $\omega = 2.71$ rad/s out of a position measurement is the payoff: the inner loop's bandwidth was never commanded, never logged and never tuned for, and the outer loop measured it.

The same quadratic sets the ceiling. At $K_{p,pos} = 2.00$ the demanded decay is comparable with ω itself, the damping falls below one, and 0.089 m of overshoot appears — at which point the time-constant column stops being meaningful and is left blank. A derivation that predicts its own range of validity is worth more than one that fits everywhere.

The cross-current block is a different kind of result. The vessel ends 2.153 m away with the sway demand saturated at 35.13 N, and the arrival test reports *not arrived* for 100 % of the run. A 0.3 m/s current is 86 % of the v_{lim} ceiling and asks for the full attainable sway force; that this configuration cannot hold station in it is a statement about the hull, not a fault in the logic. The logic's merit is that it says so.

What to try	What to watch
`u_lim = 0.4` in `W06_0_setup`, `Kp_pos` unchanged	the braking distance halves to 1.33 m and the measured time constant stays near 2.91 s — the two regimes are set by different quantities
`Kp_pos = 1.00`	the boundary between the clean rows and the overshooting one lies between 0.30 and 2.00; find it, and compare it with $\omega = 2.71$ rad/s
`R_berth = 0.05`	arrival is declared 6.7 s later — $2.91 \ln 10$, not the 7.68 s the ideal constant predicts
`V_c = 0.2`, `beta_c = pi/2` in `W06_0_setup`	the margin returns; the failure threshold lies between 0.2 and 0.3 m/s, and Section 6-2's Y_{max} is why
add an integrator to the position loop, with `Ki_pos = 0.02`	the $V_c = 0.3$ offset is removed and the first term latches — at the cost of the windup Section 6-3 drew the anti-windup path for

> ✎ **In class**
>
> - **Purpose**: make arrival a stated condition rather than a judgement, and show a derivation that measures its own assumption.
> - **Point to**: the 3.33 against 2.91 first, then $\omega = 2.71$, then the `NaN` in the $K_{p,pos} = 2.00$ row, then the two `V_c` blocks side by side.
> - **Ask**: the measured decay is faster than $1/K_{p,pos}$. A slower inner loop should make the response worse, not better — what is wrong with that reasoning?
> - **Take away**: braking distance is $u_{lim}/K_{p,pos}$, the near field is exponential but not at $1/K_{p,pos}$, and arrival is four conditions at once — which is why it needs a latch, and why a current can keep it from ever latching.

6-5. Berthing as four latched phases

This section answers: how a manoeuvre with a required final heading is expressed as logic above the three loops, and why that logic must latch.

What is observed. A berth is not a point. It is a point, a heading, and an order: arrive at the heading *before* arriving at the point, because a quay is on one side and a hull that rotates while alongside sweeps into it. A single position target cannot express that order. Commanding the berth directly from the start of the run produces a vessel that translates to the quay on whatever heading the transit happened to leave it with, and then rotates there.

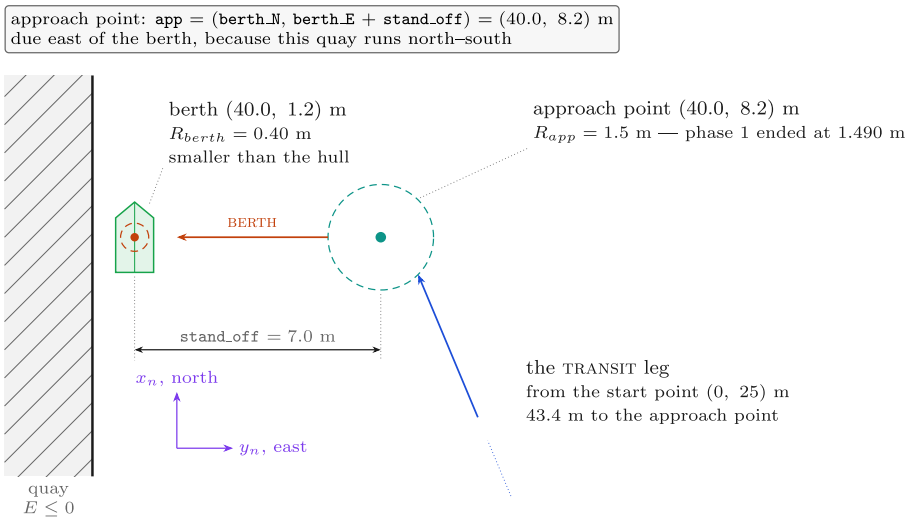
The result. The manoeuvre is four states, with the target point and the commanded heading as outputs of the state and the three loops of Section 6-3 unchanged beneath:

Phase	Target point	Commanded heading	Thrust gate
1 `TRANSIT`	the approach point `app`	bearing to `app`	on
2 `ALIGN`	the approach point `app`	ψ_{berth}	on
3 `BERTH`	the berth `brt`	ψ_{berth}	on
4 `MOORED`	the berth `brt`	ψ_{berth}	**off**

Transitions are one-way. Once taken, none is taken back.

Symbol	Quantity	Value, source
`berth_N`, `berth_E`	berth position, NED	40.0, 1.2 m
`stand_off`	offset of the approach point from the berth	7.0 m
`app`	approach point	$(40.0, 8.2)$ m
ψ_{berth}	required final heading	0 deg (alongside)
R_{app}	radius that ends phase 1	1.5 m
R_{berth}	radius that ends phase 3	0.40 m
ψ_{tol}, r_{tol}	tolerances that end phase 2	5 deg, 1.0 deg/s
v_{tol}	sway tolerance that ends phase 3	0.05 m/s

The geometry, before the logic



Section 6-5. The berthing geometry of `W06_H_berth`, drawn to scale: the hull is 2.0 m long, the arrival circle 0.40 m in radius. Phase 1 steers to the approach point and ends inside R_{app}; phase 3 replaces the target with the berth itself.

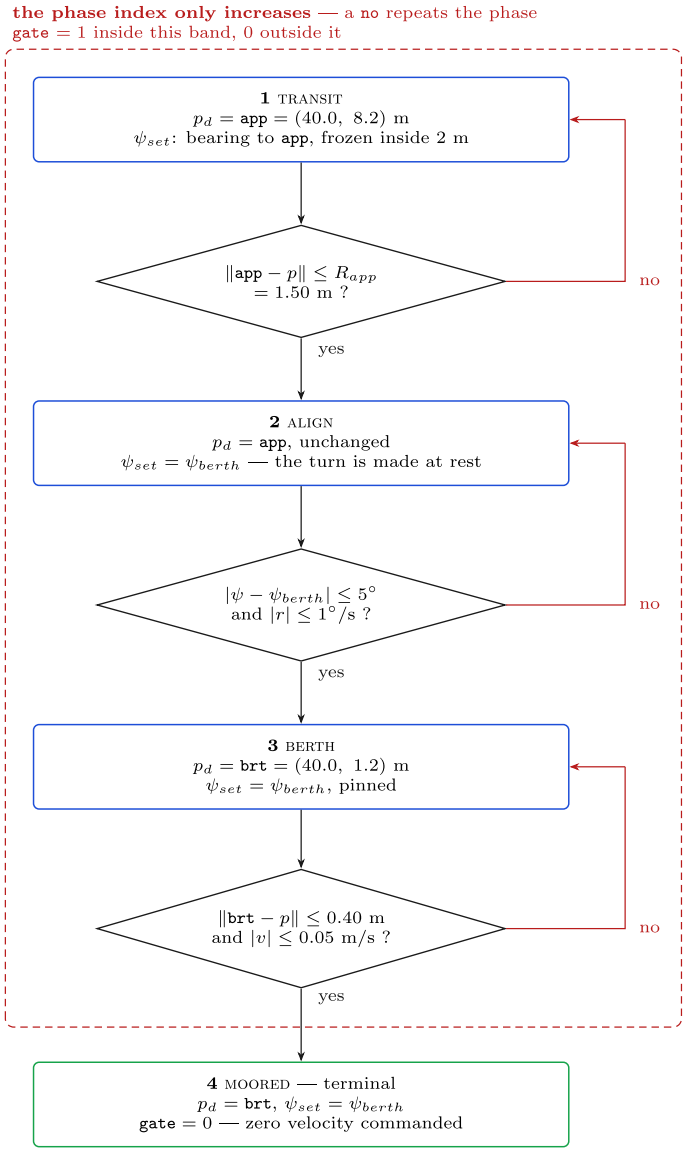
In the figure	Meaning
the screen axes, x_n up and y_n to the right	a plan view in NED: north is up, east is right, so the figure is read like a chart
the quay, hatched, at $E \le 0$	the obstacle; the berth sits 1.2 m off it, which is a little over half a hull beam
the berth circle, $R_{berth} = 0.40$ m	smaller than the hull, drawn to scale — the tolerance is tighter than the vessel it applies to
the approach point, $R_{app} = 1.5$ m	due **east** of the berth, because this quay runs north–south; `stand_off` is applied along E
the dashed `stand_off` dimension, 7.0 m	the length of the final leg, and therefore the distance over which the heading must already be correct
the `TRANSIT` leg from $(0, 25)$ m	43.4 m, which is 93 % of the run

The approach point exists so that the final leg is a straight line perpendicular to the quay. It is constructed, not chosen:

$$\mathbf{app} = \begin{bmatrix} \mathbf{berth_N} \\ \mathbf{berth_E} + \mathbf{stand_off} \end{bmatrix} = \begin{bmatrix} 40.0 \\ 1.2 + 7.0 \end{bmatrix} = \begin{bmatrix} 40.0 \\ 8.2 \end{bmatrix} \text{m},$$

which `verify_w06_actuation` check 14 confirms. The offset is applied along $\boldsymbol{E}$ alone because this quay runs north–south. A quay at another orientation would need the offset applied along its own normal, which is one rotation and is left to Assignment 6.

The four phases, and the three tests between them



Section 6-5. The berthing state machine of `W06_H_berth`, exactly as `phases()` builds it. Every test is evaluated afresh at each step, so a `no` does not go backwards — it repeats the same phase. The strategy is chosen by ψ_{berth} alone; the flow itself is the same for every strategy.

In the figure	Meaning
four boxes, one arrow each, no return arrows	the latch: the graph is a path, not a cycle
the $\mathbf{1} \to \mathbf{2}$ test	$\|\mathbf{app} - \mathbf{p}\| \le R_{app} = 1.50$ m — a distance only; the heading is still free
the $\mathbf{2} \to \mathbf{3}$ test	$\|\mathrm{ssa}(\psi - \psi_{berth})\| \le \psi_{tol} = 5$ deg **and** $\|r\| \le r_{tol} = 1$ deg/s — pointing the right way *and* no longer turning
the $\mathbf{3} \to \mathbf{4}$ test	$\|\mathbf{brt} - \mathbf{p}\| \le R_{berth} = 0.40$ m **and** $\|v\| \le v_{tol} = 0.05$ m/s
the $\mathbf{3} \to \mathbf{4}$ test uses v and **not** u	the hull comes alongside sideways; the speed that must be zero at contact is the sway component, and surge is already bounded by the position loop
`gate = 0` in phase 4	the shaft-speed command is multiplied by zero — mooring lines hold the vessel, not the thrusters

The order of the tests is the content of the state machine. Phase 2 does not test position, because the vessel is already inside R_{app} and rotating in place moves it only a little. Phase 3 does not re-test heading, because the heading loop holds ψ_{berth} throughout and re-testing would stall the approach on a transient. Each test asks only what its own phase is responsible for.

```
function [p_d, psi_set, gate, phase] = phases(berth_N, berth_E, stand_off, ...
                                              psi_berth, R_app, R_berth, x, ...
                                              psi_tol, v_tol, r_tol)
%#codegen
%  네 단계. 한 번 넘어가면 돌아오지 않는다 — 빗장이 걸려 있다.
%  Four phases. Once a transition is taken it is never taken back: it latches.
%
%    1 TRANSIT  접근점으로 간다. 선수각은 그쪽 방위 (5주차의 atan2 법칙)
%    2 ALIGN    제자리에서 psi_berth 로 돌린다
%    3 BERTH    그 선수각을 지킨 채 남은 stand_off 미터를 간다
%    4 MOORED   gate = 0 — 계류줄이 걸렸고 추진기는 0 으로 간다
%
%  왜 빗장을 거는가 / why it latches
%      단계마다 조건이 **경계에서** 참과 거짓을 왕복한다. 빗장이 없으면 배가
%      반경 R_app 안팎을 드나들 때 목표점이 접근점과 접안점 사이를 떨고,
%      안쪽 루프는 그 떨림을 추력으로 바꾼다.
%      Each condition flickers at its own boundary. Without the latch the
%      target would jitter between the two points and the inner loops would
%      turn that jitter into thrust.
%
%  왜 선수각을 **들고 있는가** / why the heading is HELD
%      1 단계의 선수각은 접근점 방위다. 접근점에 가까워지면 그 방위의 분모가
%      0 으로 가므로 2 m 안에서는 **마지막 값을 들고 있는다**. 이것이 이 절이
%      재는 예측 하나를 만든다 — 명령 선수각이 그 시각부터 평평해진다.
%      The bearing is ill-conditioned near the point, so inside 2 m the last
%      value is held. This section measures the prediction that makes: the
%      commanded heading goes flat from that instant.

persistent ph psi_hold
if isempty(ph), ph = 1; psi_hold = x(12); end

p   = [x(7); x(8)];     %  NED 위치 / position in NED
psi = x(12);
r   = x(6);
v   = x(2);

app = [berth_N; berth_E + stand_off];   %  접근점 / the approach point
brt = [berth_N; berth_E];               %  접안점 / the berth

switch ph
    case 1      %  TRANSIT
        e = app - p;
        if norm(e) > 2
            psi_hold = atan2(e(2), e(1));
        end
        if norm(e) <= R_app, ph = 2; end

    case 2      %  ALIGN
        psi_hold = psi_berth;
        de = atan2(sin(psi - psi_berth), cos(psi - psi_berth));
        if abs(de) <= psi_tol && abs(r) <= r_tol, ph = 3; end

    case 3      %  BERTH
```

```
        psi_hold = psi_berth;
        if norm(brt - p) <= R_berth && abs(v) <= v_tol, ph = 4; end

    otherwise   %  MOORED
        psi_hold = psi_berth;
end

if ph >= 3
    p_d = brt;
else
    p_d = app;
end
psi_set = psi_hold;
gate    = double(ph <= 3);   %  4 단계에서 0 — 추력을 끊는다 / thrust off
phase   = ph;
end
```

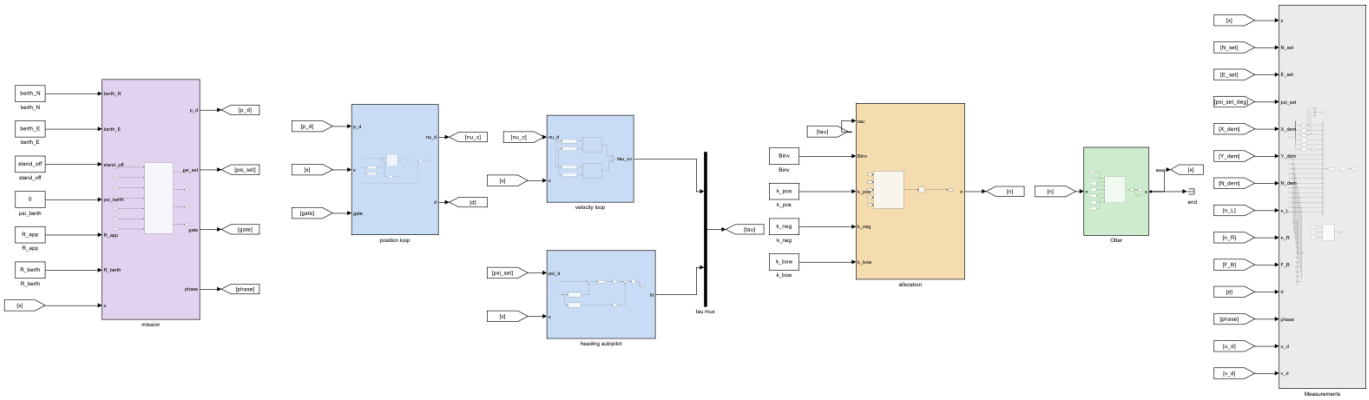
In the code	In the figure	Note
`persistent ph psi_hold`	the absence of return arrows	the latch is one persistent integer; nothing else is needed to make the graph one-way
`switch ph` with no `case` restoring a lower value	—	a phase can only be left by its own test, and only forwards
`if norm(e) > 2, psi_hold = atan2(...)`	the `TRANSIT` box	the bearing is recomputed only while the vessel is more than 2 m from the approach point; inside that, the last value is held
`de = atan2(sin(...), cos(...))`	the $2 \to 3$ test	**ssa** written out: the heading error is wrapped to $(-\pi, \pi]$, so a 350-degree error is read as -10 degrees and turned the short way
`abs(v) <= v_tol`, not `norm([u;v])`	the $3 \to 4$ test	as drawn — the sway component is the one that matters at contact
`if ph >= 3, p_d = brt; else p_d = app;`	the target column of the phase table	two phases share each target, which is why the target changes once in the whole run
`gate = double(ph <= 3)`	`gate = 0` in phase 4	a multiplication by one or zero, downstream of the allocation
`%#codegen`	—	the block is compiled; `persistent` state is therefore held in generated code too, not only in interpreted simulation

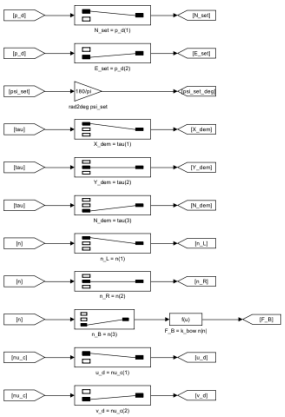
> **⚠ A latch is a decision that cannot be revisited**
>
> The latch removes chatter and, by construction, removes recovery. If phase 3 is entered and a current then pushes the vessel back outside R_{app}, the logic does not return to phase 1 — it continues trying to berth from wherever it is. Experiment 6-4 showed exactly that failure in a 0.3 m/s cross-current: the arrival test is never satisfied and the vessel holds station 2.153 m out, with the sway demand pinned at 35.13 N. For a vessel that must abort and retry, the states need guarded re-entry, which is a Stateflow chart rather than nine lines of MATLAB. This course stops at the latched form and measures what it costs.

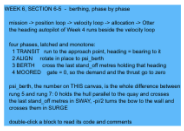
Experiment 6-5 · The four phases, measured (20 min)

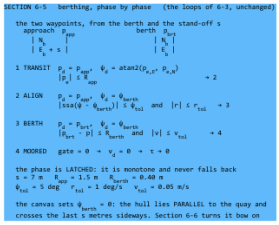
What it measures. When each phase is entered, how long it lasts, and where the vessel is at each boundary — against the three tests of the figure.

The model. `W06_5_dock.slx`. Start $(0, 25)$ m, berth $(40.0, 1.2)$ m, approach point $(40.0, 8.2)$ m, $\psi_{berth} = 0$.

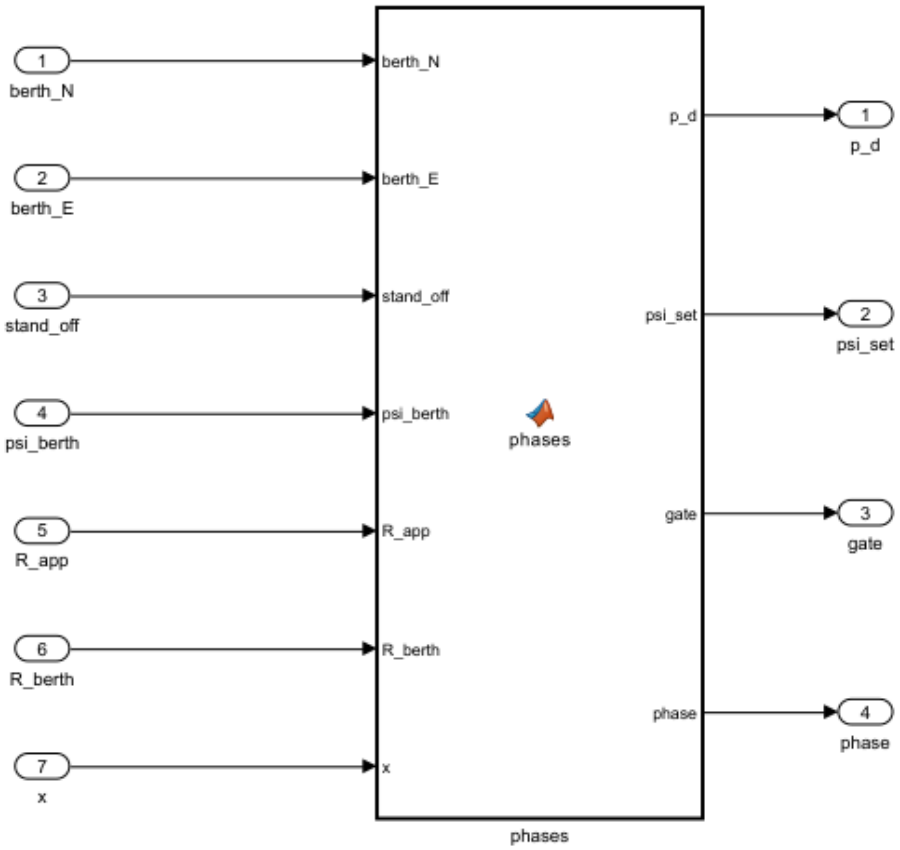








In the figure	Meaning
the `mission` subsystem, left of the position loop	the only block added to the Section 6-3 model; the three loops below it are unchanged
its outputs `p_d` and `psi_set`	the target point and the commanded heading — previously Constants, now signals
its output `gate`, running right to the shaft-speed block	the thrust gate, applied last so that phase 4 cuts the command and not the controller state
its output `phase`	logged as column 17, which is what the tables below read



In the figure	Meaning
the `phases` MATLAB Function block	the code listed above; the whole state machine is this one block
the measurement bus entering it as `x`	the block selects $\mathbf{p}$, ψ, r and v itself, so no Selector appears outside it
the Constants feeding its first six inputs	the berth, the stand-off, the required heading and the three radii — all commands, so all visible on the canvas

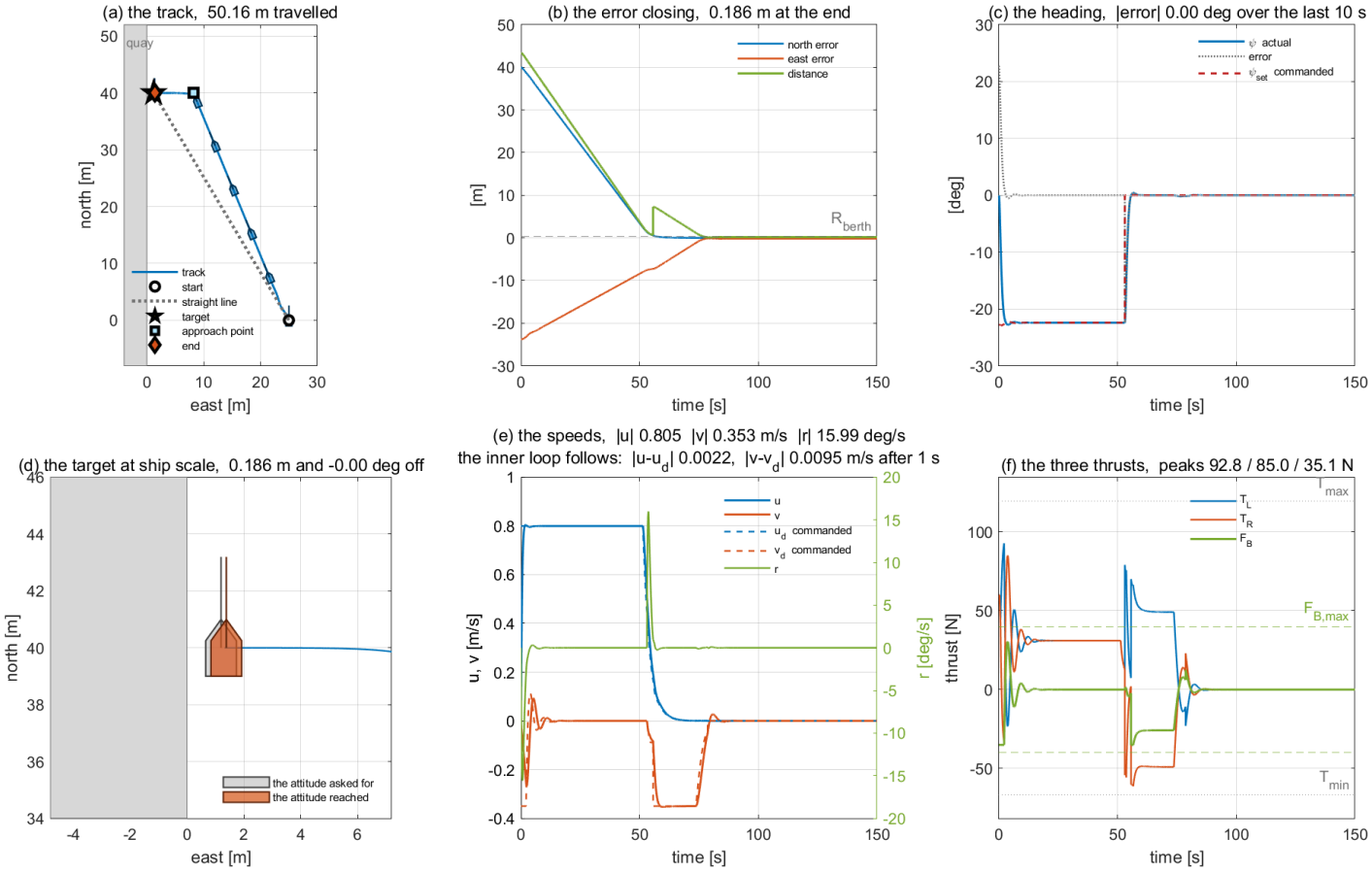
Opening and running. Pressing **Run** in Simulink raises the live track window and, when the run stops, the six-panel figure below. The script does the same from the command line and adds the phase table, which only it prints.

```
W06_5_dock_run
```

Expected output:

```
W06 Rung 5  berthing, phase by phase  (W06_5_dock)
  berth (40.00,  1.20) m,  approach (40.00,  8.20) m,  psi_berth   0.0 deg
    phase        entered    lasted   at the end of the leg
    1 TRANSIT      0.00s    53.02s   ( 38.61,   8.77) m  psi -22.42 deg  v  0.000 m/s
    2 ALIGN       53.04s     2.60s   ( 39.45,   8.47) m  psi  -0.10 deg  v -0.082 m/s
    3 BERTH       55.66s    22.98s   ( 40.00,   1.34) m  psi  -0.12 deg  v -0.051 m/s
    4 MOORED      78.66s    71.34s   ( 40.00,   1.39) m  psi  -0.00 deg  v  0.000 m/s
  phase is monotone (never falls back):  1
  final  position error  0.186 m   (tolerance 0.40 m)
         heading  error  -0.00 deg (tolerance  5.0 deg)
         speeds   u -0.000  v  0.000 m/s (tol 0.05)   r  0.000 deg/s (tol  1.0)
  |v| crossed v_tol 5 times before MOORED
```

Each row reports the state at the **last sample of that leg**, and the next leg is entered one step of $h = 0.02$ s later — which is why `1 TRANSIT` ends at 53.02 s and `2 ALIGN` is entered at 53.04 s.



In the figure	Meaning
panel (a)	the track from $(0, 25)$ to the berth, 50.16 m travelled against a 46.45 m straight line, with the hull drawn along it, the quay filled, the approach point marked as a square and the berth as a star. The dogleg through the approach point costs 3.71 m
panel (b)	the north error, the east error and the distance, against time, with R_{berth} drawn across; the distance is the one the position loop sees, so it steps when the target switches at the $2 \to 3$ boundary
panel (c)	the commanded heading, the heading, and the error between them. The command is a staircase: the bearing to the approach point in phase 1, then ψ_{berth} from phase 2 onwards
panel (d)	the berth at the scale of the hull: the attitude asked for in grey, the attitude reached in orange, with the quay edge and the last of the track between them
panel (e)	u and v on the left axis, r on the right. The peaks are $\lvert u \rvert$ 0.805, $\lvert v \rvert$ 0.353 m/s and $\lvert r \rvert$ 15.99 deg/s, the last of them entirely inside the alignment leg. Each of u and v carries its command as a dashed curve of the same colour; the second title line reports $\lvert u - u_d \rvert$ 0.0022 and $\lvert v - v_d \rvert$ 0.0095 m/s on average after the first second, which is the cascade working across all four phases
panel (f)	the three thrusts T_L, T_R, F_B against the ceilings of Section 6-2, peaking at $92.8/85.0/35.1$ N — on the one scale every rung shares

Reading the figure against the derivation.

Where to look	What is there	Which line predicts it
end of phase 1, $(38.61, 8.77)$ m	$\sqrt{(40-38.61)^2+(8.2-8.77)^2} = 1.502$ m from `app`	the $1 \to 2$ test, $\|\mathbf{p}-\mathbf{p}_{app}\| \le R_{app} = 1.50$ m, firing within 2 mm of its threshold
end of phase 1, `psi = -22.42` deg	the transit left the vessel pointing 22.4 degrees off the required heading	the bearing to `app` from the start point; nothing in phase 1 controls the final heading
phase 2 lasts 2.60 s and moves the hull 0.89 m	22.3 degrees are removed while the position almost does not change	the target column of the phase table: phase 2 still targets `app`, so the position loop is holding station while the heading loop turns
end of phase 2, `psi = -0.10` deg	inside $\psi_{tol} = 5$ deg with a wide margin	the $2 \to 3$ test is a conjunction with $\|r\| \le 1$ deg/s; waiting for the rate to settle overshoots the angle tolerance in the useful direction
panel (b), the step in the distance trace at 55.66 s	the distance the loop sees jumps from 0.61 m to 7.29 m in one sample	`if ph >= 3, p_d = brt` — the one target change in the whole run, and the only discontinuity any of the three loops sees
phase 3 covers 7.15 m in 22.98 s	an average of 0.311 m/s against the $v_{lim} = 0.35$ m/s cap	for $\psi_{berth} = 0$ the final leg is pure **sway**, so the sway cap sets its duration
end of phase 3, `v = -0.051` m/s	0.001 m/s **above** $v_{tol} = 0.05$	the $3 \to 4$ conjunction was still false at that sample and true at the next — the boundary is resolved to one step, not to one leg
end of phase 3, $(40.00, 1.34)$ m	0.14 m from the berth, inside $R_{berth} = 0.40$ m	the $3 \to 4$ test; the margin is what the hull covered while $\|v\|$ decayed through v_{tol}
phase 4, $1.34 \to 1.39$ m over 71.34 s	0.05 m of coast after the gate closes, giving the reported 0.186 m	`gate = double(ph <= 3)` removes the command, not the momentum; with no current there is nothing further to push the hull
`final position error 0.186 m (tolerance 0.40 m)`	the coast is inside the tolerance the $3 \to 4$ test was written against	R_{berth} has to be larger than the stopping distance, which Section 6-4 measured
`phase is monotone (never falls back): 1`	a boolean, printed	the latch, measured rather than asserted
`|v| crossed v_tol 5 times before MOORED`	five separate moments at which an unlatched test would have declared arrival	Section 6-4 saw two crossings on a straight run; a manoeuvre with a rotation in it has more

What the figure says. The required final heading is not achieved by the position loop and was never asked of it. It is achieved by ordering two things the position loop cannot order: rotate, then translate. The whole of that ordering is one persistent integer.

Panel (e) makes the ordering visible in a way the table cannot. The yaw rate reaches 15.99 deg/s, and all of it is inside the 2.60 -second alignment leg — twenty-seven times the 0.59 deg/s peak of the point-hold rung of Section 6-3, where no heading change was ever asked for. The position loop is holding station throughout that burst, which is why panel (a) shows the track almost stationary while panel (c) shows the heading sweeping 22 degrees.

The five crossings of v_{tol} are the argument for the latch stated as a measurement. An arrival test evaluated afresh every step would have fired at the first of them, more than twenty seconds before the hull was at the berth.

What to try	What to watch
`R_app = 5.0`	phase 2 begins 5 m out and the alignment happens further from the quay — safer, and the final leg is longer than `stand_off` intends
`stand_off = 2.0`	phase 3 covers about 1 m instead of 7.15, so it lasts a few seconds; the vessel also aligns much closer to the quay, which is the trade `stand_off` exists to set
`v_tol = 0.5`	phase 4 is entered at the first of the five crossings, with the hull still moving sideways into the quay
`psi_tol = deg2rad(45)`	phase 3 begins at a heading up to 45 degrees off, and the final leg is then neither pure sway nor pure surge — the caps of Section 6-6 both bind
remove the latch: recompute `ph` from the tests every step	the phase trace chatters at each boundary and the target point oscillates between `app` and `brt`

✎ In class

- **Purpose**: show that a required final heading is a *sequencing* problem, not a control problem.
- **Point to**: panel (b) at 55.66 s, where the distance the loop sees steps from 0.61 m to 7.29 m — the one target change in the run.
- **Ask**: phase 2 lasts 2.60 s and moves the vessel 0.89 m. What would happen if phases 2 and 3 were merged?
- **Take away**: four phases, three tests, one persistent integer — and the three loops underneath entirely unchanged.

6-6. Alongside against bow-first

This section answers: what changes in a berthing manoeuvre when only the required final heading changes, and which actuator pays for it.

What is observed. Two berthing runs are set up identically — same start, same berth, same approach point, same gains, same caps — and differ in a single constant: $\psi_{berth} = 0$ in one and -90 degrees in the other. The first comes alongside the quay; the second noses into it bow-first. Both arrive. The alignment takes twice as long in the second, the final leg takes half as long, and the peak demands land on different axes.

The result. ψ_{berth} determines the body-frame direction of the final leg, and therefore which velocity cap and which thrust cap bind on it:

$$\textbf{alongside } (\psi_{berth} = 0°)\textbf{: final leg is pure sway,} \quad \textbf{bow-first } (\psi_{berth} = -90°)\textbf{: final leg is pure surge.}$$

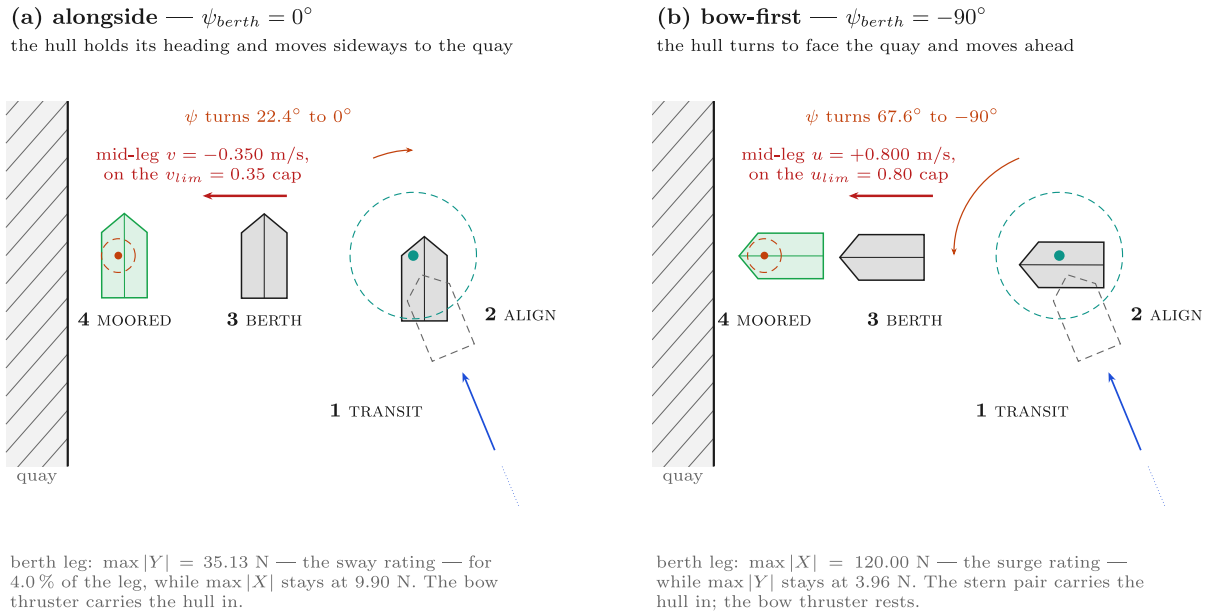
The duration of that leg follows from the cap that applies, plus one time constant of the position loop:

$$t_{berth} \approx \frac{\texttt{stand_off}}{v_{cap}} + \frac{1}{K_{p,pos}}, \qquad v_{cap} = \begin{cases} v_{lim} = 0.35 \text{ m/s} & \text{alongside} \\ u_{lim} = 0.80 \text{ m/s} & \text{bow-first} \end{cases}$$

Derivation, one line at a time.

1. The approach point lies due east of the berth, so the final leg runs due west, along $-\boldsymbol{E}$.
2. With $\psi_{berth} = 0$ the bow points north and that leg is to port: in the body frame it is $-\boldsymbol{y_b}$, pure sway. With $\psi_{berth} = -90$ degrees the bow points west and the leg is dead ahead: $+\boldsymbol{x_b}$, pure surge.
3. The velocity caps are per axis and unequal — $u_{lim} = 0.80$ m/s against $v_{lim} = 0.35$ m/s. The final leg therefore runs at 0.35 m/s alongside and 0.80 m/s bow-first, a factor of 2.3.
4. Over 7.28 m that gives 20.8 s against 9.1 s, to which the exponential tail of Section 6-4 adds $1/K_{p,pos} = 3.33$ s: a prediction of 24.15 s against 12.16 s.
5. The force caps are also unequal, and more sharply: $X_{max} = 120$ N against $Y_{max} = 35.13$ N, a factor of 3.4. The alongside leg is therefore expected to run against its force cap and the bow-first leg is not.
6. The alignment pays for this. Phase 2 turns the vessel from the transit bearing to ψ_{berth}, which is 22.4 degrees alongside and 67.6 degrees bow-first — a factor of 3.0 in angle, and the heading loop is not saturated, so the time scales similarly.

7. The measurable prediction is therefore a trade with a visible sign change: bow-first spends more time aligning and less time approaching, and loads the surge axis instead of the sway axis.



One line differs: `psi_berth`. The approach point, the three transition tests, the three control loops and the allocation B are identical in both runs — and so is every number in phase 1, because `psi_berth` is not read until phase 2. Turning further costs 2.88 s in ALIGN and saves 11.54 s in BERTH.

Section 6-6. Two berthing strategies on one grid. Every hull is drawn at a position and heading measured by `W06_G_berthing`; the dashed hull is the moment phase 2 begins, which is the same in both runs.

In the figure	Meaning
two panels on the same grid, the same quay and berth	only ψ_{berth} differs between them; everything the figure shows apart from the hull attitudes is identical
hull outlines numbered 1 to 4	drawn at the measured positions and headings of the phase table, to scale, 1 m $= 6$ mm
the arc between hulls 2 and 3 in each panel	the turn **asked** of the heading loop, labelled with it: ψ from -22.4 degrees to 0 on the left, and to -90 on the right. The first arc sweeps clockwise on the page and the second counter-clockwise, because ψ is positive clockwise from north and the two turns have opposite sign — 22.4 degrees to starboard against 67.6 to port
the mid-leg hull in the left panel, labelled $v = -0.350$ m/s	the alongside final leg is pure sway, at the sway cap
the mid-leg hull in the right panel, labelled $u = +0.800$ m/s	the bow-first final leg is pure surge, at the surge cap
hull 4 in each panel, against the berth circle	both arrive; the left one lies parallel to the quay, the right one perpendicular to it

The last row of the figure is the part that is not about control at all. A hull alongside presents its full length to the quay and can take lines fore and aft; a hull bow-first touches at one point and must be held by that point. Which of those is wanted is a seamanship question, and the controller's job is only to make both reachable.

> **Where this structure comes from**
>
> Carrying **two** berthing strategies and switching between them is the arrangement reported and sea-trialled by Bang et al. (2025), who place a transition logic above the manoeuvre so that one autonomous system can berth a USV either alongside or bow-first. The `psi_berth` Constant of this week is the degenerate case of that logic — a selection made once, on the canvas, before the run, instead of one made on conditions during it. The mission block of §6-5 already carries the machinery a running selection would need: latched phases, and an arrival test that is a conjunction of bounded terms rather than a single distance. What §6-5 does not carry is a rule for *re-entering* an earlier phase, and that omission is deliberate: re-entry earns its complexity only under a disturbance able to push the vessel back out, which is the case §6-4 measures rather than repairs.
>
> The berthing logic of §6-5 and §6-6 has no counterpart to copy in MSS. The copy this course runs against — the one `_tools/mss_path.m` resolves, its files dated 2026-08-31 — holds dynamic-positioning material (`demoDPThrusterModels` , `demoSemisubDPsystem` , `demoCS2passiveObserverDP`) but no berthing demonstration, and no file in it matches *berth* or *dock* by name. The phase machine here was therefore written from the derivation of §6-5 rather than adapted, which is why that section derives it rather than citing it.

Experiment 6-6 · The same berth, two attitudes (20 min)

What it measures. The predictions of steps 4, 5 and 6 — alignment time, berth-leg time, and which cap binds — from two models that differ in one Constant.

The models. `W06_5_dock.slx` and `W06_7_front.slx` . They come from the same builder entry, with the same gains, the same caps, the same allocation and the same mission block. One Constant on the canvas separates them:

	`W06_5_dock`	`W06_7_front`
`psi_berth`	`0`	`-pi/2`
the attitude that asks for	alongside, the quay off the port beam	bow-first, the bow at the quay
everything else	identical	identical

Opening and running. Each model runs on its own, and pressing Run in Simulink draws the same panel the script draws. Run both in turn and read the two outputs side by side.

```
W06_5_dock_run
W06_7_front_run
```

The output of the first is printed in full in Section 6-5. The leg table and the force block of each are repeated here, so that the two can be compared line against line:

```
W06 Rung 5  berthing, phase by phase  (W06_5_dock)
  berth (40.00,  1.20) m,  approach (40.00,  8.20) m,  psi_berth   0.0 deg
    phase        entered    lasted   at the end of the leg
    1 TRANSIT      0.00s    53.02s   ( 38.61,   8.77) m  psi -22.42 deg  v  0.000 m/s
    2 ALIGN       53.04s     2.60s   ( 39.45,   8.47) m  psi  -0.10 deg  v -0.082 m/s
    3 BERTH       55.66s    22.98s   ( 40.00,   1.34) m  psi  -0.12 deg  v -0.051 m/s
    4 MOORED      78.66s    71.34s   ( 40.00,   1.39) m  psi  -0.00 deg  v  0.000 m/s
  across the BERTH leg:  peak |u|  0.193 m/s,  peak |v|  0.353 m/s
                         peak |X|   9.90 N (limit 120.00),  peak |Y|  35.13 N (limit  35.13)
                         on the Y cap  4.0 % of the leg
  final  position error  0.186 m   (tolerance 0.40 m)
         heading  error  -0.00 deg (tolerance  5.0 deg)

W06 Rung 7  the same berth, bow on to the quay  (W06_7_front)
  berth (40.00,  1.20) m,  psi_berth  -90.0 deg  (rung 5 holds 0.0 deg)
    phase        entered    lasted   at the end of the leg
    1 TRANSIT      0.00s    53.02s   ( 38.61,   8.77) m  psi  -22.42 deg  u  0.520  v  0.000 m/s
    2 ALIGN       53.04s     5.48s   ( 39.78,   8.26) m  psi  -89.83 deg  u  0.016  v  0.087 m/s
    3 BERTH       58.54s    11.44s   ( 39.99,   1.60) m  psi  -90.00 deg  u  0.138  v  0.002 m/s
    4 MOORED      70.00s    80.00s   ( 39.99,   1.55) m  psi  -90.00 deg  u -0.000  v -0.000 m/s
  across the BERTH leg:  peak |u|  0.804 m/s,  peak |v|  0.086 m/s
                         peak |X| 120.00 N (limit 120.00),  peak |Y|   3.96 N (limit  35.13)
                         on the Y cap  0.0 % of the leg
  the surge limit is 3.42 times the sway limit
  final  position error  0.347 m   heading error   0.00 deg
```

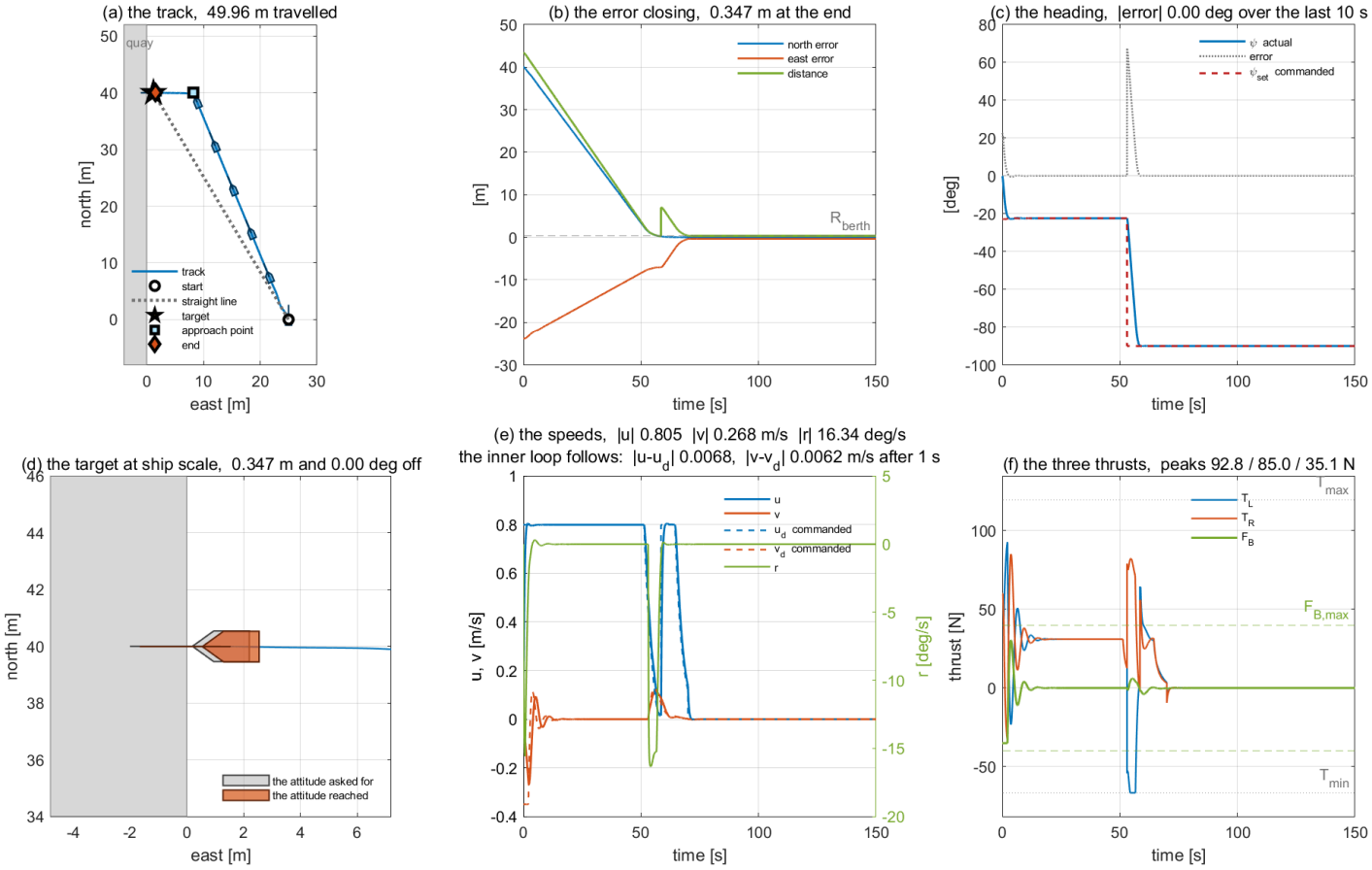
Two quantities the derivation needs are not printed, because each is a combination of printed numbers rather than a measurement of its own:

Quantity	How it is formed from the output above	Alongside	Bow-first
the turn asked of phase 2	the printed `psi_berth` minus the heading printed at the end of phase 1, -22.42 deg in both runs	22.42 deg	67.58 deg
the distance phase 3 must cross	from the position printed at the end of phase 2 to the printed berth $(40.00, 1.20)$ m	7.291 m	7.063 m

The second of these feeds the duration predicted in step 4, $d/v_{cap} + 1/K_{p,pos}$, with v_{cap} the cap on the axis the leg actually uses — $v_{lim} = 0.35$ m/s alongside, $u_{lim} = 0.80$ m/s bow-first — and $1/K_{p,pos} = 3.33$ s in both.

Reading the output against the derivation.

Where to look	What is there	Which line predicts it
`1 TRANSIT`, both runs: lasted 53.02 s, ending at $(38.61, 8.77)$ m and -22.42 deg	the first phase is identical to the last digit printed	phase 1 does not read ψ_{berth}; until the 1→2 test fires the two runs are the same simulation
the turn asked, 22.42 against 67.58 deg	a factor of 3.01	step 6: the transit heading is the same in both, so the turn asked is the difference between it and ψ_{berth}
`2 ALIGN`, 2.60 against 5.48 s	a factor of 2.11 for a factor of 3.01 in angle	the heading loop is not rate-limited, so a step three times larger does not take three times as long; the saturated middle of the larger turn runs at a rate the smaller one never reaches
`3 BERTH` against step 4	22.98 s measured against 24.16 s predicted, and 11.44 s against 12.16 s	step 4, within 4.9 % and 5.9 % with nothing fitted; both measurements fall **below** the prediction, because the prediction charges the full cap-speed distance and the hull is already moving when the leg begins
`peak |Y|`, 35.13 N alongside	exactly Y_{max}, to the printed digits	step 5: the alongside leg runs against the sway force cap
`on the Y cap`, 4.0 % alongside and 0.0 % bow-first	the cap is reached, and only briefly	the cap binds on the acceleration at the start of the leg, not on the steady approach
`peak |X|`, 9.90 N alongside against 120.00 N bow-first	a factor of 12.1	steps 2 and 5: the leg is sway in one case and surge in the other, so the demand lands on a different axis entirely — and the surge axis has 3.42 times the authority to meet it
`peak |v|` 0.353 alongside, `peak |u|` 0.804 bow-first	each just above its own cap	the caps are on the command, not on the measurement; the measured value overshoots slightly, as in Section 6-3
`4 MOORED`, entered at 78.66 against 70.00 s	bow-first is 8.66 s faster overall, 11.0 % of the manoeuvre	the 11.54 s saved on the berth leg exceeds the 2.88 s lost on the alignment
the east coordinate across phase 4, $1.34 \rightarrow 1.39$ against $1.60 \rightarrow 1.55$ m	0.05 m of coast in each, in opposite directions	phase 4 cuts the thrust, so what remains is the momentum each leg carried in, and the two legs approach on opposite axes
`final position error`, 0.186 against 0.347 m	both inside $R_{berth} = 0.40$ m	the arrival test; the bow-first value is larger because the leg arrives faster and the decay below v_{tol} covers more ground



In the figure	What is in it
panel (a)	the track from the start to the berth, with the quay filled, the hull drawn at intervals along it, the approach point squared and the berth starred; the dogleg of Section 6-5 is still there, because phases 1 and 2 are unchanged
panel (b)	the north error, the east error and the distance, with R_{berth} drawn across; the step at 58.54 s is the 2→3 handover of the target
panel (c)	the commanded heading, the heading, and the error between them; the 67.58 degree step at 53.04 s is the alignment, and it is the one visible difference from the rung-5 panel
panel (d)	the berth at the scale of the hull: the attitude asked for in grey, the attitude reached in orange, the two lying across each other at a right angle to the quay
panel (e)	u and v on the left axis, r on the right; the berth leg is a flat-topped u at the surge cap, with v never leaving the axis. The dashed curve of each colour is the command the position loop issued, and the second title line gives the mean gap between command and response after the first second
panel (f)	the three thrusts against the ceilings of Section 6-2, on the one scale every rung shares; T_L and T_R run together and high across the berth leg, where the rung-5 panel has them opposed and small

What the output says. One constant, changed from 0 to $-\pi/2$, moves a peak demand of 120 N from one axis to another and shortens the manoeuvre by 11.0 %. Nothing else in either model is different: same gains, same caps, same allocation, same four phases, same first 53 seconds to the printed digit.

The bow-first run is faster, and that is the less interesting half of the result. The more interesting half is that it is faster *because* the Otter's surge authority is 3.42 times its sway authority — the ratio the rung-7 script prints on its own line — which is a consequence of the hull form — two propellers aft and one small tunnel forward — and not of any choice made in this course. An orientation that aligns the dominant demand with the dominant actuator is faster. That is a statement about the vessel, and reading it off a controller's output is the point of running both cases.

The attitude is a literal on the canvas, so the first two of these are changed by double-clicking the `psi_berth` Constant in either model and typing a new value; the other two are workspace quantities, changed in `W06_0_setup.m` and picked up on the next run.

What to try	What to watch
`psi_berth` = `-pi/4`	neither cap dominates: both u and v are commanded on the berth leg, and its duration should fall between the 22.98 s and 11.44 s measured above
`psi_berth` = `pi/2`, nosing the stern in	the turn asked becomes 112.42 degrees, and the berth leg is pure **astern** surge — so it is held by $T_{min} = -66.71$ N rather than by $T_{max} = 119.68$ N. Check whether the leg is slower than the alongside one in the same proportion as the two limits
`v_lim = 0.80` in `W06_0_setup.m`, with `psi_berth` = `0`	the alongside leg should shorten towards the bow-first time while `on the Y cap` rises; if it does, the velocity cap and not the force cap was setting the duration
`stand_off = 15` in `W06_0_setup.m`	the gap between the two attitudes should grow roughly in proportion, because it accrues on the berth leg alone and phases 1 and 2 are unchanged by the stand-off

✎ **In class**

- **Purpose**: separate what the controller decides from what the hull decides.
- **Point to**: the `peak |X|` line of the two outputs — 9.90 N against 120.00 N, from one changed constant.
- **Ask**: bow-first is 8.66 s faster. Name a berth where it would nevertheless be the wrong choice.
- **Take away**: ψ_{berth} chooses which actuator pays, and on this hull surge has 3.42 times the authority of sway.

6-7. Where the one rotation sits

This section answers: the thrusters are bolted to the hull and the berth is bolted to the shore, so exactly one rotation must stand between them — does it matter where it is placed?

What is observed. Two models hold the same point $(10, 10)$ m with the same three loops, the same position gain, the same heading autopilot and the same allocation. They differ only in where $R(\psi)$ appears. One rotates the position *error* in the outer loop and works entirely in hull axes below that; the other leaves the outer loop in north and east, and rotates the measured velocity *into* the inner loop and the computed force *out* of it. Each is run at $\psi_d = 0$ and at $\psi_d = 45$ degrees. All four stop on the point, to three decimal places. Nothing else about the four runs is the same.

The result. One rotation in the outer loop, or two in the inner — and the two are the same controller until a limit is reached:

$$\text{body-fixed:}\quad F^b = \mathrm{sat}\big(K_p\,(\nu_d - \nu) + I\big), \qquad \text{NED:}\quad F^b = R(\psi)^{\mathsf{T}}\,\mathrm{sat}\big(K_{p,ned}\,(\dot{p}_d - R(\psi)\,\nu) + I\big)$$

The outer loops that feed them differ in one operation, the cap:

$$\text{body-fixed:}\quad \nu_d = \begin{bmatrix} \mathrm{sat}(K_{p,pos}\, x_e,\ u_{lim}) \\ \mathrm{sat}(K_{p,pos}\, y_e,\ v_{lim}) \end{bmatrix}, \qquad \begin{bmatrix} x_e \\ y_e \end{bmatrix} = R(\psi)^{\mathsf{T}} (p_d - p)$$

$$\text{NED:}\quad \dot{p}_d = \nu_p \, \min\!\left(1,\ \frac{v_{ned,max}}{\|\nu_p\|}\right), \qquad \nu_p = K_{p,pos}\,(p_d - p)$$

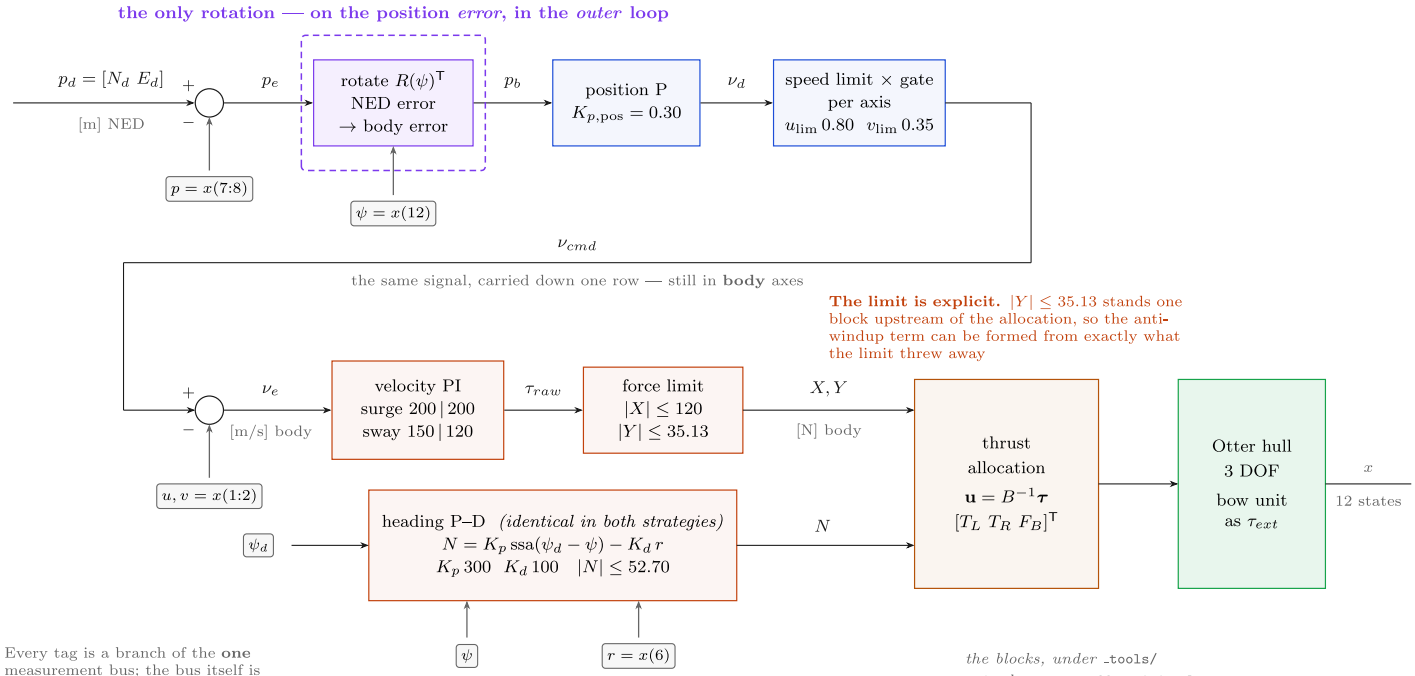
Symbol	Quantity	Value, source
$R(\psi)$	rotation from hull axes into NED	$\begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix}$, Section 6-3
$p_d - p$	position error, north and east	m, from the plant state $x_{7:8}$
$K_{p,pos}$	outer position P gain	0.30 s^{-1}, the same constant in both models
$u_{lim},\ v_{lim}$	per-axis speed caps, hull axes	0.80, 0.35 m/s, Section 6-3
$v_{ned,max}$	speed cap by magnitude, NED	0.35 m/s $= v_{lim}$, `W06_vars.m:79`
$K_p,\ I$	inner PI, hull axes, per axis	surge $200 \mid 200$, sway $150 \mid 120$
$K_{p,ned},\ K_{i,ned}$	inner PI, NED, **both axes**	150, 120 — the sway values
Y_{max}	sway force the hull can make	35.13 N, `W06_vars.m:69`
$2X_{max}$	the limit the NED PI applies instead	240 N per axis, `add_velocity_loop_ned.m:144`

Derivation, one line at a time.

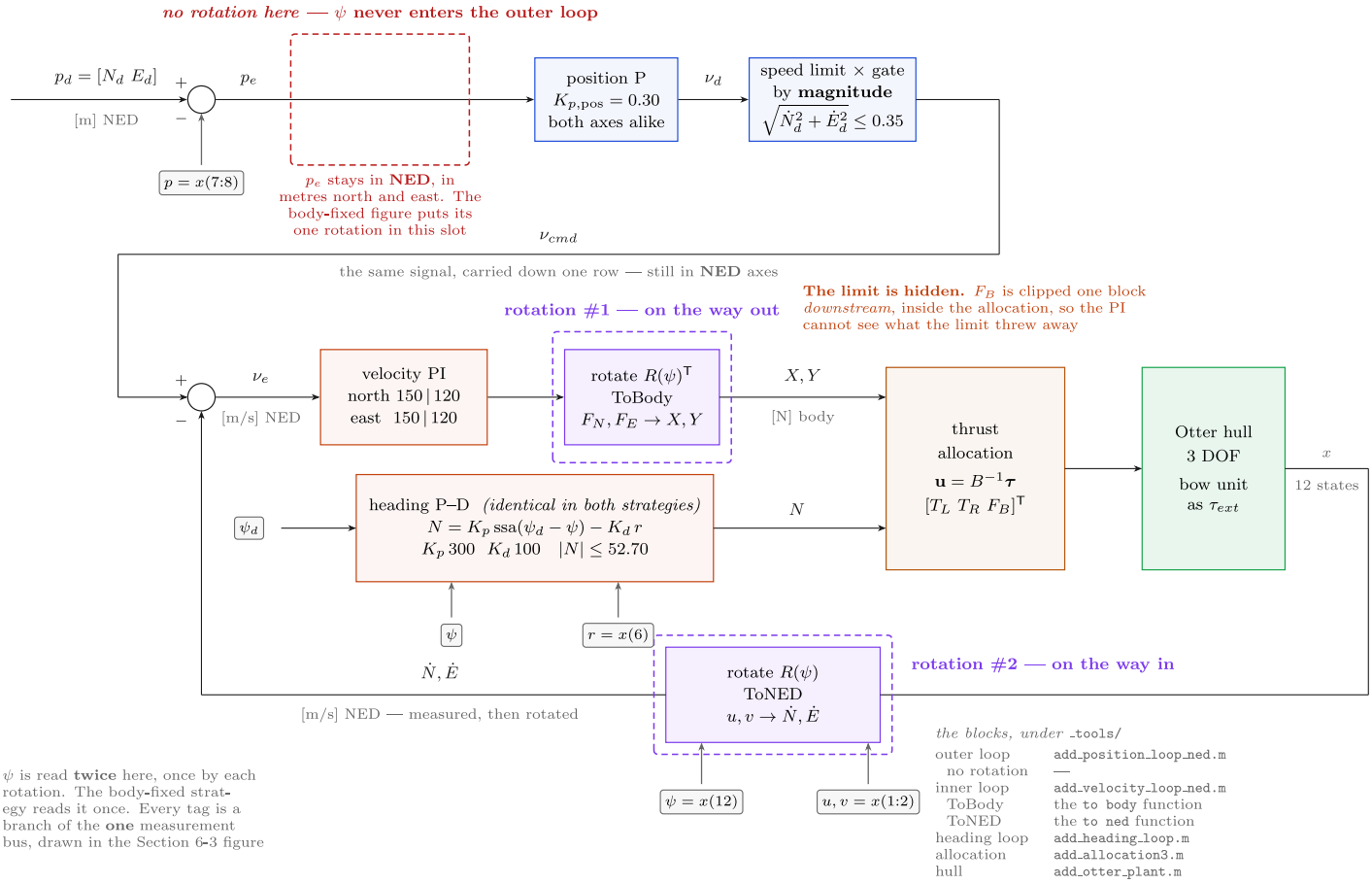
1. The thrusters are bolted to the hull, so the allocation of Section 6-2 takes X, Y, N in hull axes. The berth is a point on the shore, so the error of Section 6-3 is born in north and east. Exactly one rotation must appear on the path between them — no fewer, or the axes do not match; no more, unless they cancel.
2. There are two places to put it. On the error, at the top: $p_e^b = R(\psi)^{\mathsf{T}}(p_d - p)$, after which every signal below is in hull axes and no further rotation appears. Or on the two signals that cross the inner loop's boundary: the measured velocity on the way in, $R(\psi)\,\nu$, and the computed force on the way out, $R(\psi)^{\mathsf{T}} F$. That is one rotation against two.
3. Before any limit is reached the two are the same controller. For a given error and heading, the physical velocity commanded by the body-fixed outer loop and by the NED outer loop agree to machine precision: check 15 of `verify_w06_actuation` sweeps 24 headings and reports a largest difference of 2.001×10^{-16} m/s. Correctness therefore cannot choose between them, and neither can the gains, which are set from the same constants.
4. What differs is one fact about the frames: **a hull axis has a fixed physical identity and a NED axis does not.** x_b is always surge — two propellers, $M_{11} = 85.5$ kg, $|X| \le 120$ N. North is surge when $\psi = 0$, sway when $\psi = 90$ degrees, and a mixture in between.
5. First consequence, on the cap. The body-fixed outer loop caps each axis at its own limit, 0.80 and 0.35 m/s. The NED outer loop cannot: two per-axis caps at 0.35 would allow $0.35\sqrt{2} = 0.495$ m/s on a diagonal, which at $\psi = 45$ degrees is partly sway, and the hull cannot make it. The cap must therefore be on the **magnitude**, at the slower axis's value, $v_{ned,max} = 0.35$ m/s. The body-fixed command may reach $\sqrt{0.80^2 + 0.35^2} = 0.873$ m/s on a diagonal; the NED command may reach 0.350 m/s in any direction. A ratio of 2.49, with no gain changed.
6. Second consequence, on the shape of the command. A per-axis cap scales the two components by *different* factors and therefore turns the velocity vector away from the error. A magnitude cap scales both by the *same* factor and leaves the direction alone. The body-fixed track must therefore bend; the NED track must be straight. Which way the body-fixed track bends depends on ψ — the fast axis points where the bow points — but that it bends does not.
7. Third consequence, on the gains. The body-fixed inner loop may be tuned per axis, because each axis has a fixed inertia: $M_{11} = 85.5$ against $M_{22} = 162.5$ kg, hence surge $K_p = 200$ and sway $K_p = 150$. The NED inner loop may not: a gain set for surge becomes excessive once the vessel has turned and that axis is sway. Both NED axes therefore carry the **slow axis's** gains, $150 \mid 120$.
8. Fourth consequence, on the saturation, and this is the one that leaves a mark on the integrator. The body-fixed sway PI knows its own actuator rating, $Y_{max} = 35.13$ N, and saturates there — one block *upstream* of the allocation. The quantity $\mathrm{sat}(F) - F$ therefore exists, and the back-calculation term $K_b(\mathrm{sat}(F) - F)$ of Section 6-3 carries exactly what the limit threw away. The NED PI cannot know its rating, because on the north axis it depends on ψ; it saturates loosely at

$2X_{max} = 240$ N, and the true limit is applied *downstream*, inside the allocation, after the rotation. The NED controller integrates against a wall it cannot feel.

9. The measurable predictions follow. The NED run must take the same time at every heading, because no heading enters its outer loop; the body-fixed run must not. The body-fixed sway demand must stop exactly at Y_{max}; the NED sway demand must pass it. And the body-fixed advantage must be temporary, because it holds only while the fast axis still has error to close.

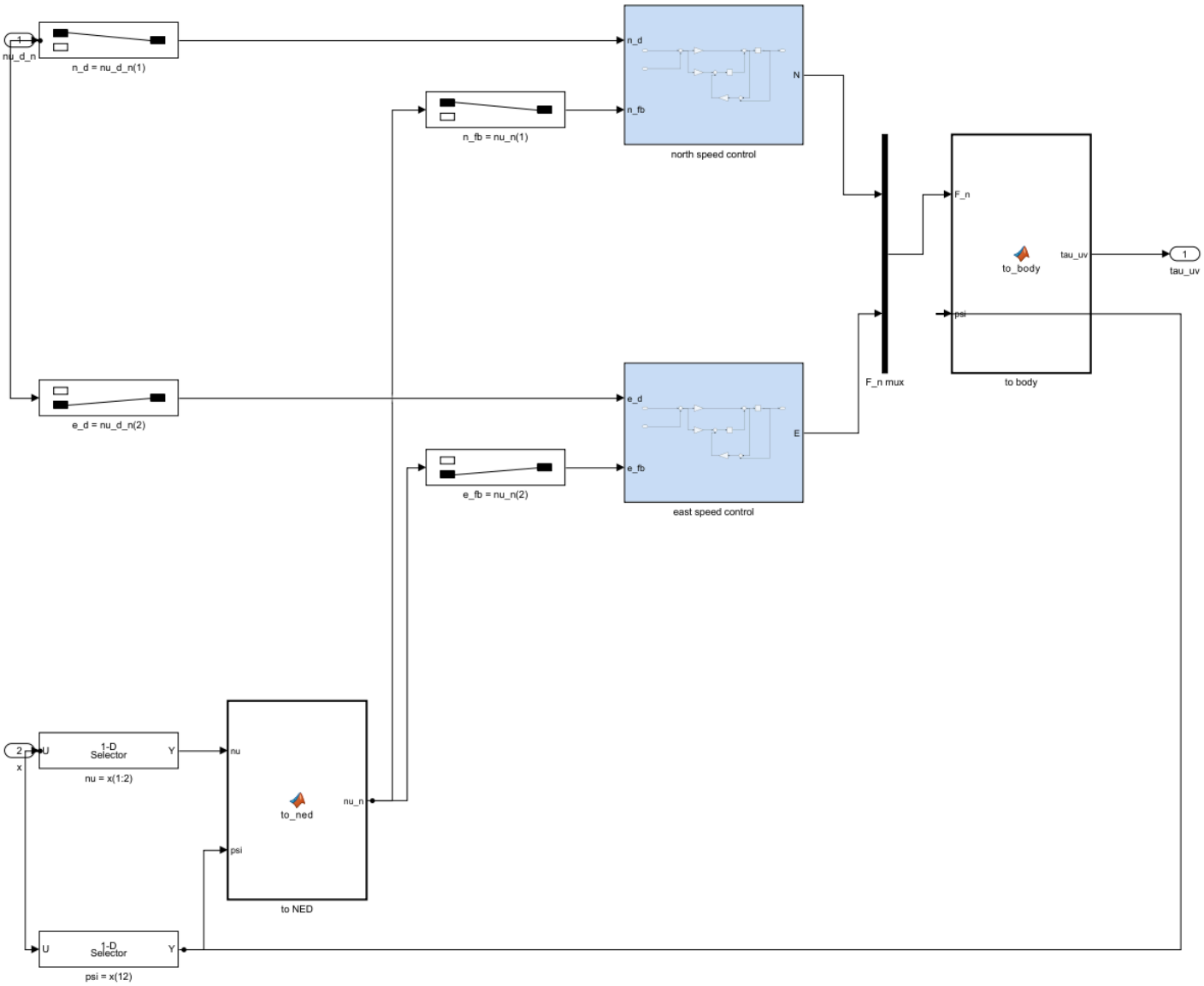


In the figure	Meaning
the purple cell at the top left, `rotate` $R(\psi)^\mathsf{T}$	the **only** rotation in the whole diagram, on the position error, in the outer loop
`speed limit × gate`, `per axis`, u_{lim} 0.80 v_{lim} 0.35	two independent caps, each at its own axis's limit — step 5
`velocity PI`, `surge` 200 | 200, `sway` 150 | 120	per-axis gains, because each axis has a fixed inertia — step 7
`force limit`, $\lvert X\rvert \le 120$, $\lvert Y\rvert \le 35.13$	the actuator rating, stated in the loop that can act on it
the orange note, *the limit is explicit*	$\mathrm{sat}(F) - F$ is available here, so anti-windup is exact — step 8
the `heading P--D` band at the bottom	identical, character for character, in both strategies



In the figure	Meaning
the red dashed cell at the top left, *no rotation here*	the slot the previous figure fills; ψ never enters this outer loop at all
`speed limit × gate` , `by magnitude` , $\sqrt{\dot{N}_d^2 + \dot{E}_d^2} \le 0.35$	one cap on the length of the vector, at the slow axis's value — step 5
`velocity PI` , `north` $150 \mid 120$, `east` $150 \mid 120$	one gain pair for both axes — step 7
the purple cell `to NED` , marked *rotation #1 — on the way in*	$u, v \to \dot{N}, \dot{E}$, the measurement entering the loop
the purple cell `to body` , marked *rotation #2 — on the way out*	$F_N, F_E \to X, Y$, the force leaving it
the note that ψ is read **twice**	two rotations need two copies of the heading; the previous figure needs one
the orange note, *the limit is hidden*	the bow unit's own ceiling acts on its shaft speed inside the allocation, downstream, where no back-calculation term can be formed — step 8

Both sheets are drawn on the same grid, so holding one over the other shows a single difference: one filled cell in the outer loop against two filled cells in the inner.



In the figure	Meaning
`to NED` at the left and `to body` at the right	the two rotations, each a MATLAB Function block, with `psi = x(12)` branching to both
the two subsystems `north speed control` and `east speed control`, one above the other	the same arrangement as the body-frame model's `u speed control` and `v speed control`, so the two can be opened side by side — and the same gain names `Kp_ned` and `Ki_ned` appear inside both
the `north limit` and `east limit` blocks reading `2*X_max`	the loose limit of step 8, written as an expression so the model shows where the number comes from
the `Kb` path from each saturation back to its integrator	the anti-windup wiring is present and correct — it simply has the wrong limit to work from

Experiment 6-7 · The same point, held in two frames (20 min)

What it measures. All three predictions of step 9, from two models run at two commanded headings each; and, for the NED model, what produces a peak speed above its own 0.350 m/s ceiling.

The models. The same demand $(10, 10)$ m stands on both canvases and both stop at $T_{final} = 120$ s. The table is the whole difference between them:

	`W06_1_point.slx`	`W06_8_ned.slx`
where the outer loop measures the error	hull axes, after one $R(\psi)^{\mathsf{T}}$	north and east, with no rotation
what the speed cap applies to	each axis, u_{lim} 0.80 and v_{lim} 0.35	the magnitude, $v_{ned,max}$ 0.35
the inner PI	per axis, $200 \mid 200$ and $150 \mid 120$	one pair, $150 \mid 120$, on both axes
where the force limit is applied	inside the loop, at X_{max} and Y_{max}	downstream, inside the allocation
everything else	identical	identical

The commanded heading is on neither canvas: the `psi_d_cmd` Constant holds its own *name*, so each script places a value in the workspace and runs its own model a second time at 45 degrees. No block is edited between the two runs.

Opening and running.

```
W06_1_point_run
W06_8_ned_run
```

Expected output — the two-heading block of each script, and the block rung 8 prints before it. The rung-1 script's other table is the one already read in Section 6-3.

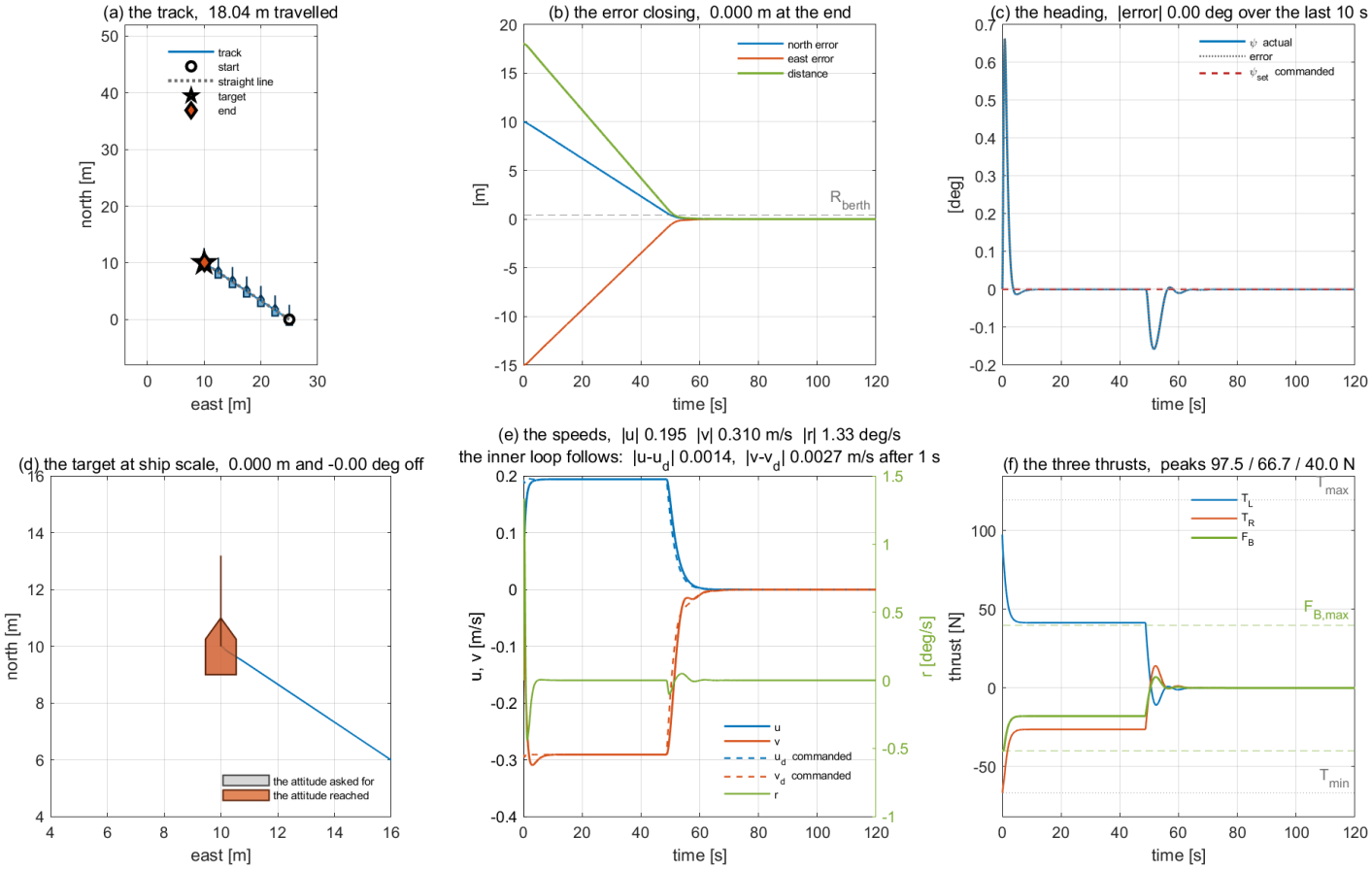
```
W06 Rung 1  the same point, two commanded headings  (body-fixed outer loop)
  psi_d [deg]  t to 0.5 m [s]   d(end) [m]  max |U| [m/s]   max |X| [N]   max |Y| [N]
  0                    42.68        0.000          0.878        120.00         35.13
  45                   49.36        0.000          0.714        120.00         35.13
  the per-axis caps allow hypot(0.80, 0.35) = 0.873 m/s on a diagonal
  at psi_d 0, |U| stays above the NED ceiling 0.35 m/s until t = 40.46 s

W06 Rung 8  the same point, held in NED  (W06_8_ned)
  demand          ( 10.00,  10.00) m
  reached         ( 10.00,  10.00) m   final error  0.000 m
  within 0.5 m at   51.00 s
  peak speed       u 0.195   v 0.310 m/s
  peak SPEED MAGNITUDE  0.363 m/s   (the NED loop limits this, not each axis: 0.35)
  peak demand      X  29.12 N (limit 120.00)   Y  43.68 N (limit  35.13)
  peak bow force   F_B  40.00 N  (rating 40.00) - the cap here is on SPEED, not force

W06 Rung 8  the same point, two commanded headings  (NED outer loop)
  psi_d [deg]  t to 0.5 m [s]   d(end) [m]  max |U| [m/s]   max |X| [N]   max |Y| [N]
  0                    51.00        0.000          0.363         29.12         43.68
  45                   51.00        0.000          0.468         65.87         43.68
  the cap is on the MAGNITUDE, one value in any direction: 0.350 m/s
  at the psi 45 peak, t =  1.38 s:  u +0.378  v -0.277 m/s
                                  N demanded  52.70 N m (N_max 52.70) - inside the opening turn
  sway the NED PI never saw:  43.68 demanded - 40.00 delivered =  3.68 N
```

Three quantities the argument needs are differences of two printed lines rather than lines of their own:

Quantity	How it is formed	Value
what the body-fixed strategy gains at $\psi_d = 0$	the NED `t to 0.5 m` less the body-fixed one	$51.00 - 42.68 = 8.32$ s
what is left of that gain at $\psi_d = 45$ degrees	the same subtraction on the second row	$51.00 - 49.36 = 1.64$ s
how far the NED sway demand passes the pure-sway limit	rung 8's `peak demand Y` less the `limit` printed beside it	$43.68 - 35.13 = 8.55$ N



In the figure	What it shows	What to read here
(a) track	the path over ground, hull drawn every few seconds	a straight line from start to target: a magnitude cap keeps the direction
(b) errors	north, east and distance against time	distance passes 0.5 m at 51.00 s and settles on 0.000 m
(c) heading	command, measured, error	ψ holds the commanded value throughout; the outer loop never consults it
(d) pose	attitude asked for in grey, attitude reached in colour	the two coincide, so the heading loop is not what the approach is waiting on
(e) speeds	u, v on the left axis, r on the right, each with its command dashed in the same colour	neither axis reaches its own body-fixed limit; $\lvert U \rvert$ is what is capped. The command is rotated into body axes before it is logged, so the dashed and solid curves of a colour are comparable here exactly as on the body-fixed rungs
(f) thrusters	T_L, T_R, F_B against their limits	F_B reaches the 40 N rating line, which no body-fixed run in this week does

The body-fixed panel for the same demand is the one already read in Section 6-3; it is not repeated here.

Reading the output against the derivation.

Where to look	What is there	Which line predicts it
`d(end)`, all four rows	0.000 m	step 3: both are correct controllers; where the rotation sits does not decide whether the loop closes
`t to 0.5 m`, NED rows	51.00 s at **both** headings	step 9: ψ never enters the NED outer loop, so the approach cannot depend on it
`t to 0.5 m`, body-fixed rows	42.68 against 49.36 s	step 9: the body-fixed approach depends on how the error lies relative to the hull
`max |U|`, body-fixed 0.878 against the printed diagonal 0.873	0.6 % above the commanded ceiling	the cap is on the command; the inner PI overshoots its own setpoint slightly, as in Section 6-3
`max |U|`, NED at $\psi_d = 0$, 0.363 against 0.350	3.7 % above its own ceiling	the same second-order overshoot, now of the single cap the NED loop has
`max |U|`, NED at $\psi_d = 45$ degrees, 0.468	34 % above the cap, far outside that overshoot	the two lines below it: the peak falls at $t = 1.38$ s with the yaw demand exactly on N_{max} — a transient of the opening turn through the allocation, Section 6-2, not an effect of the frame
`|U| stays above the NED ceiling until t = 40.46 s`	nearly the whole 42.68 s approach	step 5: the per-axis caps let the body-fixed command exceed the NED ceiling for as long as either axis still has error to close
`max |X|`, body-fixed 120.00 N at both headings	the surge force cap	the body-fixed command is large enough to press the surge axis to its limit
`max |X|`, NED 29.12 and 65.87 N	never near 120	its velocity command is 2.49 times smaller, so the surge PI is never pressed
`max |Y|`, body-fixed 35.13 N	exactly Y_{max}	step 8: the limit stands inside the loop that computes the demand
`max |Y|`, NED 43.68 N at both headings	8.55 N past Y_{max}	step 8: the NED PI saturates at 240 N and never learns about 35.13
`peak bow force F_B 40.00 N (rating 40.00)`	the bow unit on its physical rating	step 8 one layer lower: in the body-fixed run F_B peaks at 35.13 N because the loop stopped it first

What the output says. The first line of the table is the one to read twice: $d(end) = 0.000$ m, in all four runs. Both strategies are correct. A derivation that ended at correctness would have nothing left to say, and would have missed everything this experiment measures.

What the placement of the rotation decides is which frame the *limits* are expressed in, and limits are where a linear design stops being linear. In hull axes the limits are constants of the vessel — 0.80 and 0.35 m/s, 120 and 35.13 N — and a controller written in hull axes can use all four of them. In north and east the same physical limits become functions of ψ, and a controller written there can only use the weakest value of each: one speed, one gain pair, one loose force bound. The three costs of steps 5, 7 and 8 are three faces of that single fact.

The costs are not equal. Being slow is visible and bounded: 8.32 s at $\psi_d = 0$, and only 1.64 s at $\psi_d = 45$ degrees, once the body-fixed advantage has been spent on the bend in its own track. Asking for 43.68 N of sway is different in kind. Two separate limits lie below that demand, and the run passes both: $Y_{max} = 35.13$ N, the sway the hull can hold when no yaw is asked of it at the same time, and $F_{B,max} = 40$ N, the bow unit's own rating, which the output shows it reaching exactly. The demand is still enforced — the allocation clips it, and the vessel arrives — but it is enforced where the integrator cannot see it happen, so the integral state carries credit for work the hull never did. With no current and a reachable target, that costs only time. Under a current the integrator is what holds station, and the difference stops being cosmetic — §6-4 puts the same structure in a 0.3 m/s cross-current and measures the station it settles for.

The last three lines of the output are a caution about measurement, not about frames. The NED peak speed at $\psi_d = 45$ degrees, 0.468 m/s, sits 34 % above its own 0.350 m/s ceiling, and the natural suspicion is the surge leak of Section 6-2 — a sway demand above Y_{max} producing unasked-for surge. The script answers by printing the single instant at which $|U|$ peaks rather than a second maximum to subtract. That instant is $t = 1.38$ s, inside the opening turn, with $u = +0.378$ and

$v = -0.277$ m/s and the yaw demand exactly on $N_{max} = 52.70$ N·m. Asked for a yaw it cannot produce alongside the surge and sway, the allocation clips per thruster, and what reaches the hull is not what any one axis requested. The peak belongs to the heading turn passing through the allocation — Section 6-2's subject, not this section's. The $\psi_d = 0$ run, whose yaw demand never approaches N_{max}, peaks only 3.7 % above its ceiling: the inner PI overshooting its own command, the same effect as 0.878 against 0.873 on the body-fixed side.

> **⚠ Compare at one instant, not maximum against maximum**
>
> The two maxima of a signal pair fall at different times, so their difference describes no moment of the run. `W06_8_ned_run` therefore prints u, v and the yaw demand at the single index where $|U|$ peaks, and every claim about the $\psi_d = 45$ degree peak above is read at that index. The `43.68 demanded - 40.00 delivered` line is deliberately labelled as two peaks rather than as an instantaneous excess: it bounds the sway the bow unit could not supply across the run, and nothing more.

What to try. The demand $(10, 10)$ m is a literal on both canvases, so the first row below is changed by double-clicking the `N_d` and `E_d` Constants. Every other row is a workspace quantity, edited in `W06_0_setup.m` and picked up on the next run.

What to try	What to watch
`N_d = 10` and `E_d = 0` on both canvases, a target due north	at $\psi_d = 0$ the error lies along the fast axis and the two tracks coincide; the body-fixed advantage should be at its largest. Compare the two `t to 0.5 m` rows
`Kp_ned = 200` and `Ki_ned = 200`, the surge values	$\psi_d = 0$ should improve and $\psi_d = 45$ degrees should overshoot — step 7's reason for carrying the slow axis's gains, made visible
`v_ned_max = 0.873`, the body-fixed diagonal ceiling	the NED run is handed the body-fixed *speed* and nothing else; whether the 8.32 s gap closes says how much of it was the cap and how much the per-axis shape of step 6
the NED cap per axis instead: edit `cap_speed` in `position loop NED` to clip each component at `v_ned_max`	the command should reach 0.495 m/s on the diagonal and the sway demand should rise further above Y_{max}
`Y_max` raised to 120, with `W06_1_point` only	the body-fixed sway demand passes 35.13 N, which proves the 35.13 in the table was the saturation and not a coincidence of the trajectory

> **✎ In class**
>
> - **Purpose**: show that a frame choice which cannot affect correctness still decides what the controller is allowed to know.
> - **Point to**: the two loop figures side by side — one filled cell against two — then `max |Y|` , 35.13 against 43.68, and the bow unit standing on its 40 N rating in panel (f).
> - **Ask**: all four runs stop on the point. Name the quantity that the NED controller is wrong about anyway, and say where in the diagram it would have to be measured to fix it.
> - **Take away**: put the rotation where it leaves the limits in the frame they are constants in. For a hull, that is hull axes.

6-8. The ladder, rung by rung

What is observed. Each of the seven experiments before this one asks a single question and then stops, so what changed between one experiment and the next is never in view. Run back to back into one table, the same seven runs answer a different question: what each rung of the week actually added. Exactly one thing is new on each rung, so the difference between two rows is what that one thing did.

The result. Seven runs of seven models, measured with every metric defined as the section that introduced it defines it, give three tables and eight figures. Two rows carry the argument of the whole week. Rung 2 translates 13 m due east with $|u|_{max} = 0.021$ m/s; rung 4 translates 12 m due north with $|v|_{max} = 0.000$ m/s and $F_B = 0.0$ N. Each is a straight line of exactly its own length. No two-thruster hull can produce both.

The rungs are numbered as the undergraduate capstone course numbers them, so that the two ladders can be read side by side. Each rung owns a model and a run script of its own, and the demand that rung is given is a literal on its own canvas, so the table below only says which model to open.

Rung	Model	What is new on this rung	What to check
1 `point`	`W06_1_point`, demand $(10, 10)$ m	the three-loop cascade closing on a point	$d(\text{end}) = 0.000$ m, reached in 42.68 s
2 `sway`	`W06_2_sway`, demand $(0, 13)$ m	the task a stern pair alone cannot do	$\lvert u\rvert_{max} = 0.021$ m/s; path = straight line $= 12.00$ m
3 `thrust`	`W06_3_thrust`, demand X, Y, N	the loops taken away — the allocation and $T = kn\lvert n\rvert$ left exposed	$F_B = 15.0$, $T_L = 4.2$, $T_R = 35.8$ N, each reproducible by hand
4 `stop`	`W06_4_stop`, demand $(12, 25)$ m	pure surge, as the contrast rung 2 needs	$\lvert v\rvert_{max} = 0.000$ m/s, $F_B = 0.0$ N, $T_L = T_R = 60.0$ N
5 `dock`	`W06_5_dock`, $\psi_{berth} = 0$	the four latched phases	entered at 0.00, 53.04, 55.66 and 78.66 s
7 `front`	`W06_7_front`, $\psi_{berth} = -90$ deg	the same berth on the other attitude	latches at 70.00 s with $\lvert v\rvert_{max} = 0.268$ m/s
8 `ned`	`W06_8_ned`, demand $(10, 10)$ m	the one rotation moved into the inner loop	51.00 s to 0.5 m, and F_B reaching 40.0 N

Four things about the shape of that table are worth saying plainly.

- **Rung 6 is absent, and deliberately so.** In the undergraduate course, rung 6 is rung 5 run against a Gazebo/VRX simulation of the same vessel. This course works in Otter dynamics throughout, so there is no sixth rung to run. The numbering skips it rather than closing the gap, because closing it would make the two ladders disagree about which rung is which.
- **Rungs 7 and 8 do not stand on rung 6.** Rung 7 is rung 5 with one Constant changed; rung 8 is rung 1 with the rotation moved. Each stands on the rung its own model was copied from, not on the one numbered before it.
- **Rung 3 is the one place where the two ladders run in opposite directions.** The capstone *adds* the thrust curve at rung 3, because its earlier rungs command thrust directly. The models of this course carry the allocation and the curve from rung 1 onwards, so rung 3 instead *strips the loops away* and leaves the actuation exposed. Either way the rung measures the same thing: what the actuators do when nothing is closing around them.
- **A blank entry is an absence, not a failure.** Rung 3 has no target, so it has no distance to one and no arrival; only the two berthing rungs have phases. The tables print those as `NaN`.

Where each model is drawn. The diagrams are not repeated here — each is read in the section that built it, and the ladder only runs them in order. Three shapes account for all seven: the cascade, the cascade with a state machine in front of it, and the cascade taken away.

Rungs	Models	Shape	Its diagram is in
1, 2, 4	`W06_1_point`, `W06_2_sway`, `W06_4_stop`	the three-loop cascade, one rotation	§6-3, drawn for `W06_1_point`; the other two differ only in the two demand Constants
3	`W06_3_thrust`	no loops at all	§6-1, with the allocation and the thrust curve left exposed
5, 7	`W06_5_dock`, `W06_7_front`	the cascade with `phases` in front of it	§6-5, drawn for `W06_5_dock`; `W06_7_front` differs in the `psi_berth` Constant alone
8	`W06_8_ned`	the cascade with the rotation moved inward	§6-7, with the rotation drawn twice

Experiment 6-8 · The whole ladder, measured in one place (25 min)

What it measures. Every rung of the week, with each metric defined exactly as its own section defines it, so that one row may be subtracted from another.

The models. All seven, each run exactly as it stands: `W06_1_point.slx`, `W06_2_sway.slx`, `W06_3_thrust.slx`, `W06_4_stop.slx`, `W06_5_dock.slx`, `W06_7_front.slx`, `W06_8_ned.slx`. The script changes nothing in any of them. Its table `L` names only which model to run, which section introduced it, and the position the final pose is held against; the demand itself is already on the canvas, which is why a row of the table cannot silently disagree with the model it names.

Opening and running.

```
W06_J_ladder
```

Expected output:

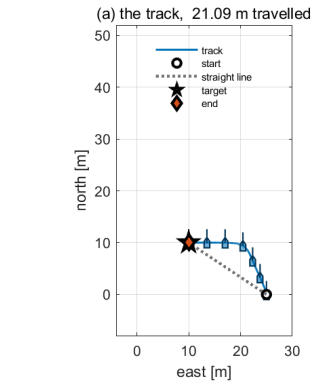
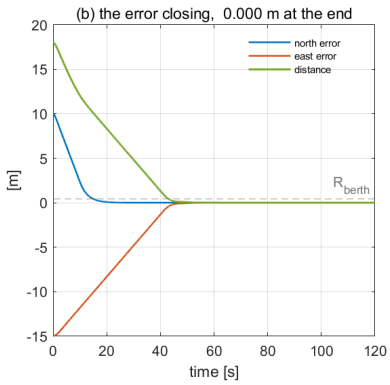
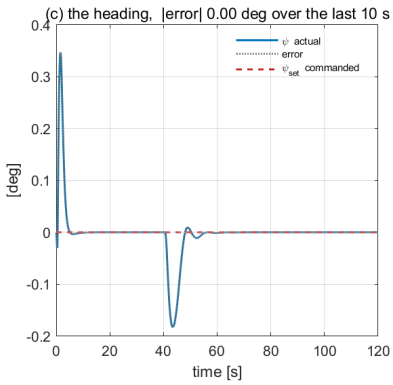
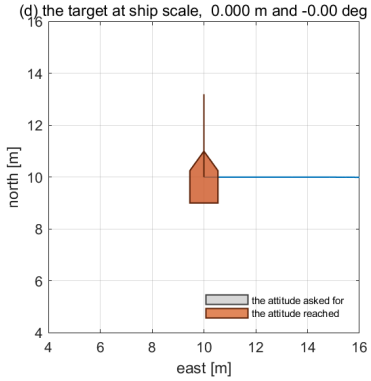
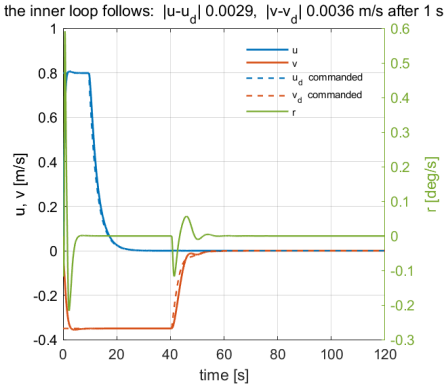
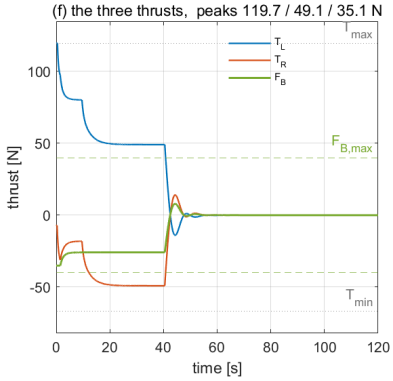
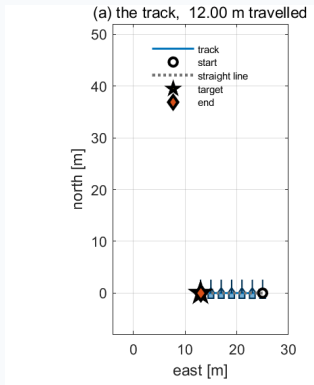
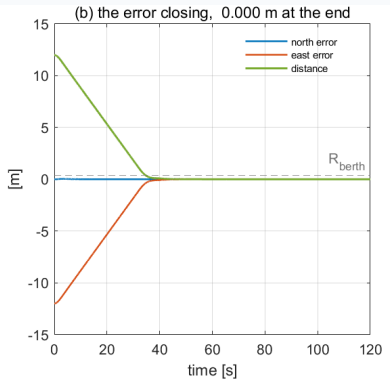
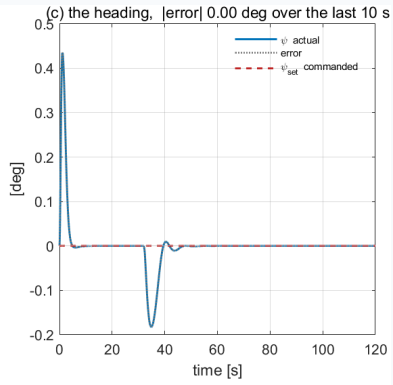
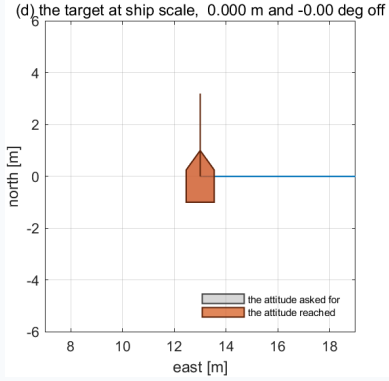
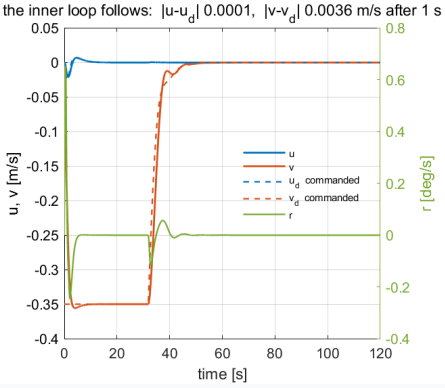
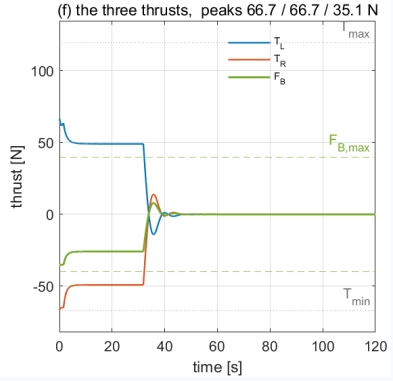
```
W06 Experiment 6-8  the rungs of the ladder, measured in one place

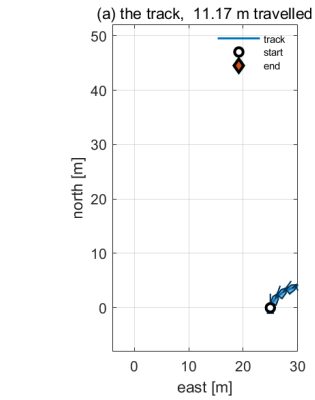
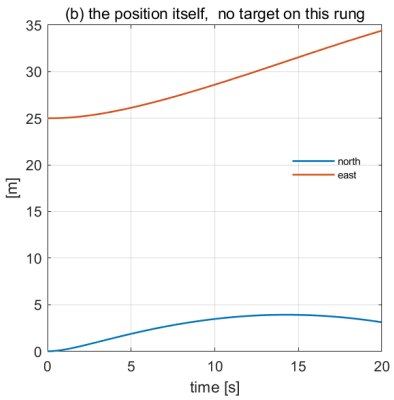
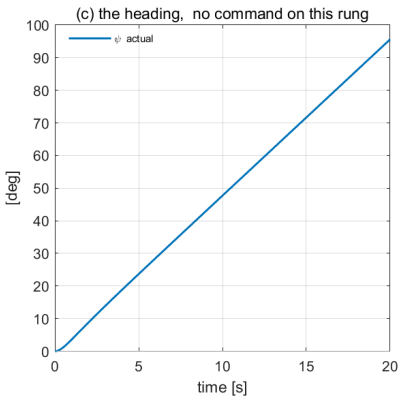
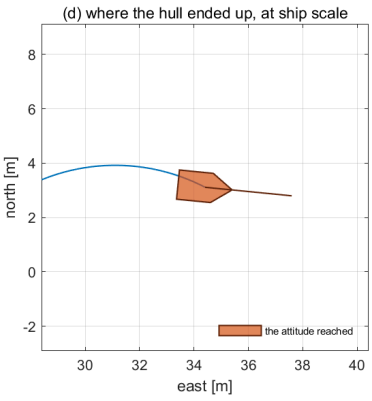
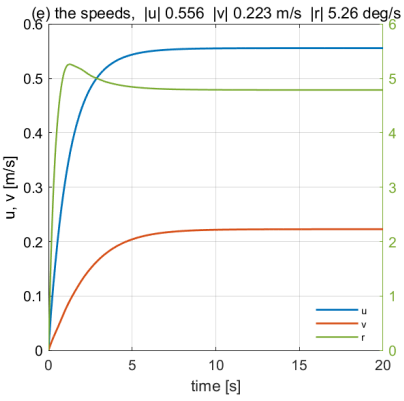
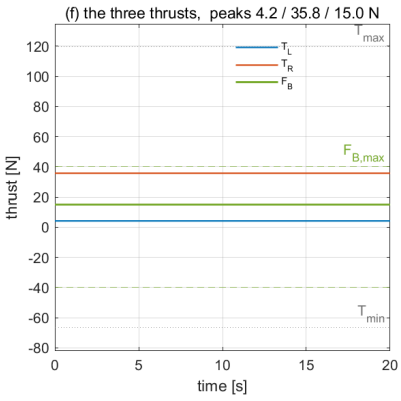
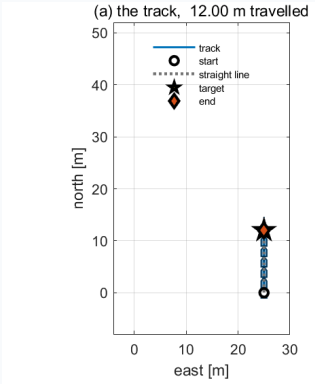
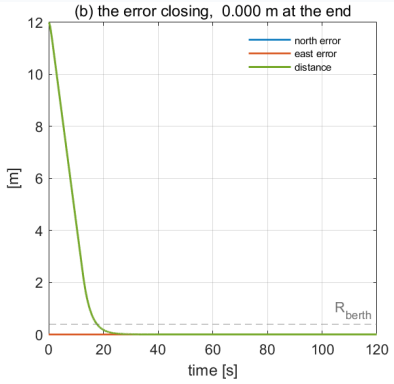
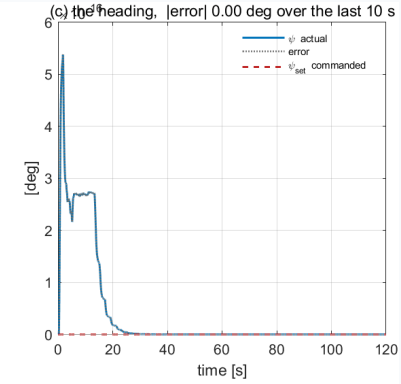
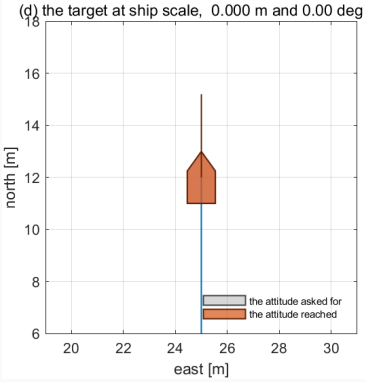
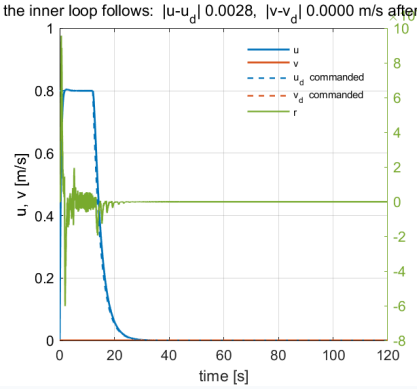
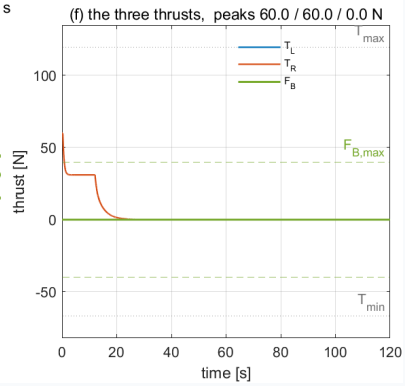
  table 1, the mission
  rung      t 0.5 [s]    phase1    phase2    phase3    phase4  d(end)[m]    psi err last 10 [m]
  1 point       42.68       NaN       NaN       NaN       NaN      0.000      -0.00       0.000
  2 sway        34.10       NaN       NaN       NaN       NaN      0.000      -0.00       0.000
  3 thrust        NaN       NaN       NaN       NaN       NaN        NaN        NaN         NaN
  4 stop        17.00       NaN       NaN       NaN       NaN      0.000       0.00       0.000
  5 dock        75.94      0.00     53.04     55.66     78.66      0.186      -0.00       0.186
  7 front       56.22      0.00     53.04     58.54     70.00      0.347       0.00       0.347
  8 ned         51.00       NaN       NaN       NaN       NaN      0.000      -0.00       0.000

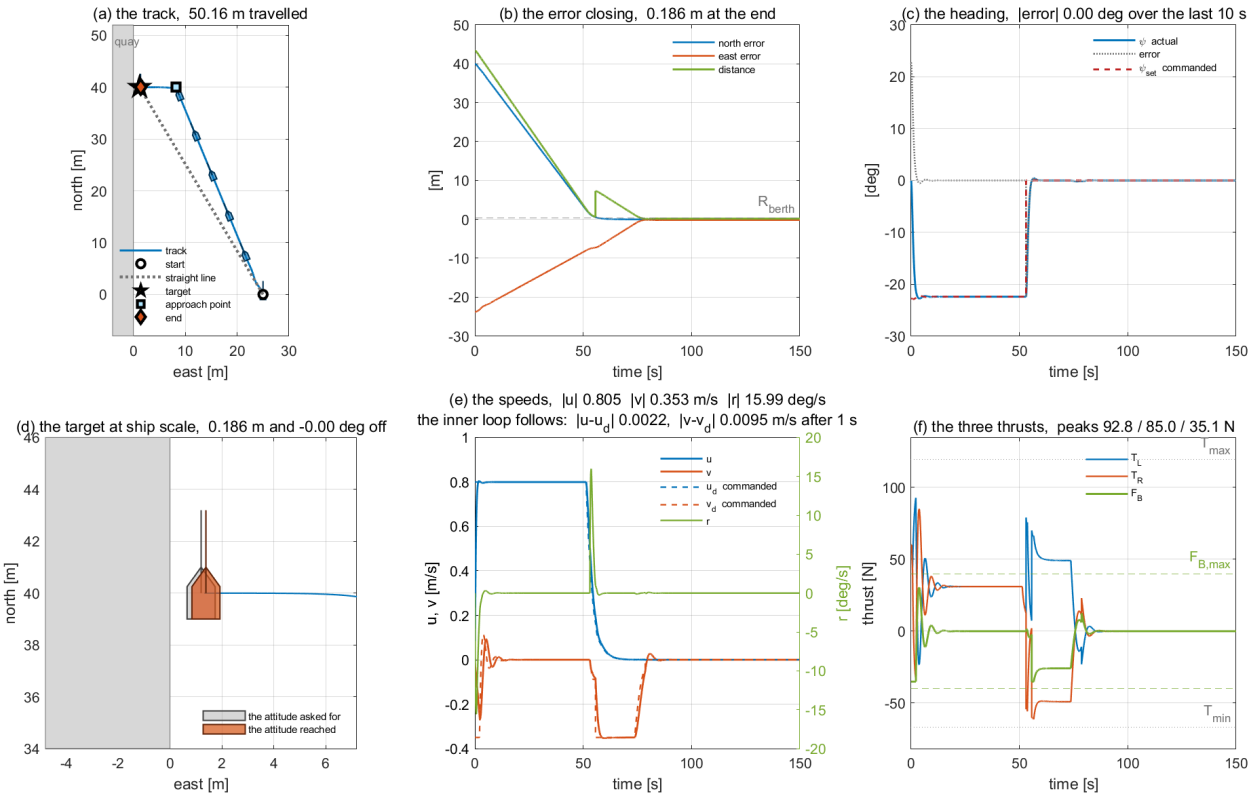
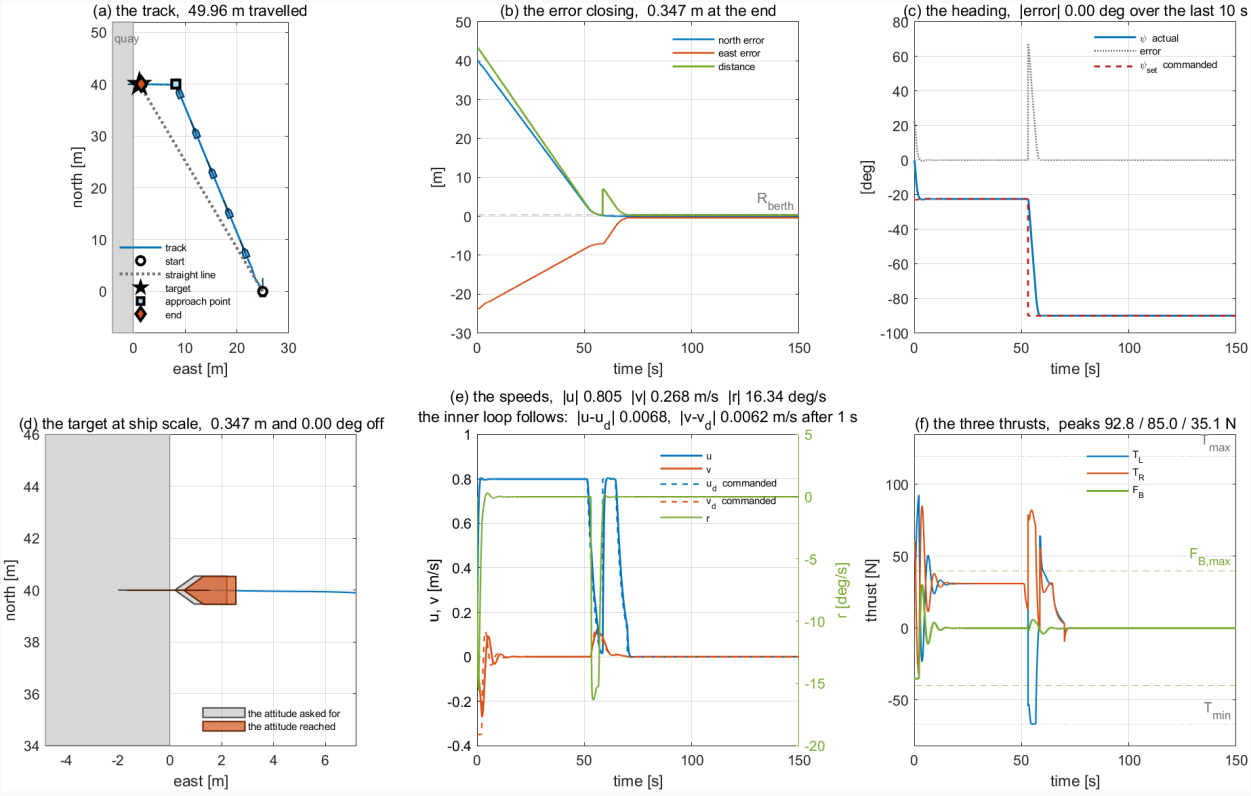
  table 2, the actuation
  rung       |u|max  |v|max  |r|max     T_L     T_R     F_B   |n|max  path [m]  straight
  1 point     0.808   0.356    0.59   119.7    49.1    35.1   103.93     21.09     18.03
  2 sway      0.021   0.356    0.65    66.7    66.7    35.1   101.74     12.00     12.00
  3 thrust    0.556   0.223    5.26     4.2    35.8    15.0    56.86     11.17      9.91
  4 stop      0.804   0.000    0.00    60.0    60.0     0.0    73.59     12.00     12.00
  5 dock      0.805   0.353   15.99    92.8    85.0    35.1    97.54     50.16     46.45
  7 front     0.805   0.268   16.34    92.8    85.0    35.1   101.74     49.96     46.36
  8 ned       0.195   0.310    1.33    97.5    66.7    40.0   101.74     18.04     18.03

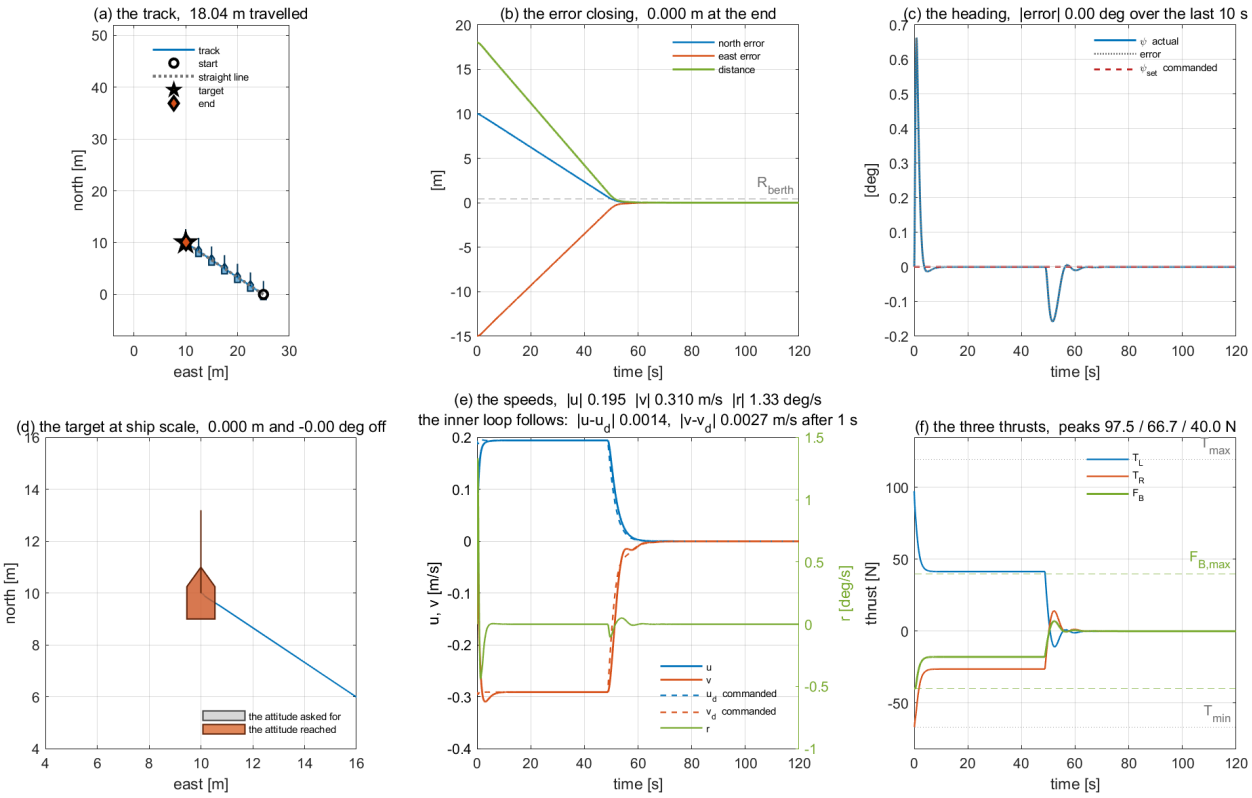
  table 3, the attitude
  rung      target N        E      psi    final N        E      psi  samples
  1 point     10.000   10.000     0.00     10.000   10.000    -0.00     6001
  2 sway       0.000   13.000     0.00     -0.000   13.000    -0.00     6001
  3 thrust       NaN      NaN      NaN      3.110   34.407    95.62     1001
  4 stop      12.000   25.000     0.00     12.000   25.000     0.00     6001
  5 dock      40.000    1.200     0.00     39.999    1.386    -0.00     7501
  7 front     40.000    1.200   -90.00     39.990    1.547   -90.00     7501
  8 ned       10.000   10.000     0.00     10.000   10.000    -0.00     6001
```

The script saves one figure per rung. The six panels are the same on every one — the track, the error closing, the heading, the target at ship scale, the speeds, the three thrusts — so that two rungs are compared by looking at the same place twice rather than by hunting.

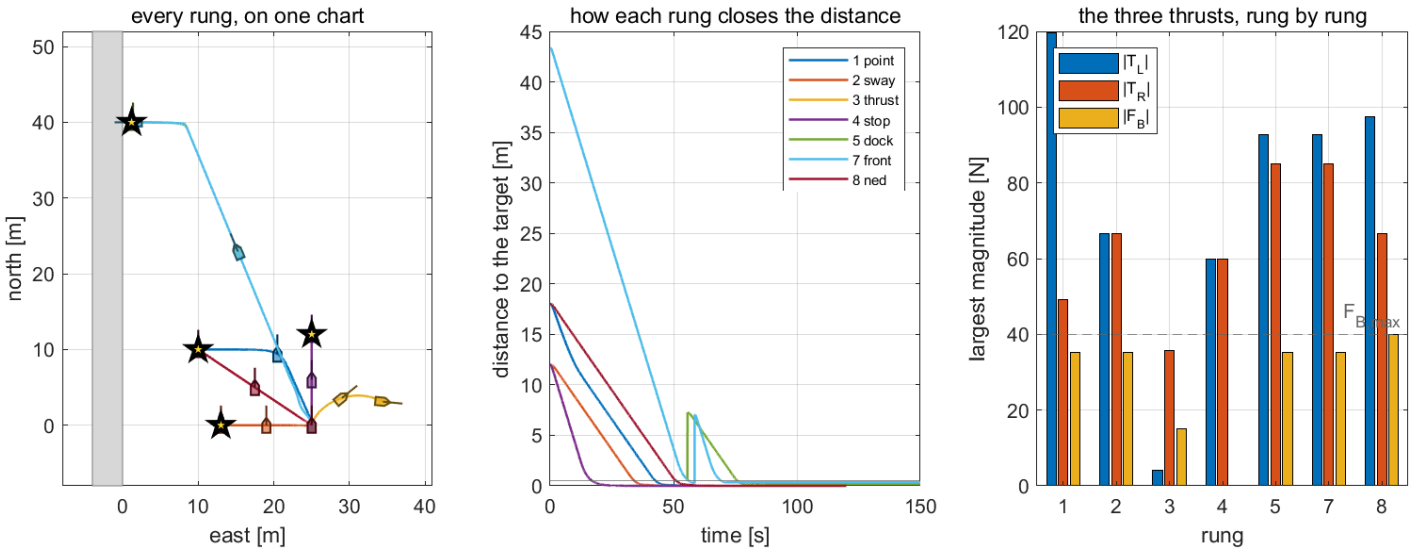
Rung	The run
1 `point`	     
2 `sway`	     

Rung	The run
3 `thrust`	     
4 `stop`	     

Rung	The run
5 dock	
7 front	

Rung	The run
8 ned	

The last figure does the one thing the seven cannot: it lays the rungs on top of each other.



Reading the figures against the derivations.

Where to look	What is there	Which section predicts it
rung 1, panel (f), T_L	119.7 N — T_{max} to the digit	§6-2: a surge demand on its 120 N cap puts the left unit exactly on its ceiling
rung 2, $\lvert n\rvert_{max}$ in table 2	101.74, which is $\lvert n_{min}\rvert$	§6-2: a pure sway demand drives both stern units astern, and astern is the weaker direction
rung 2, F_B	35.1 N — Y_{max}, not the bow unit's 40 N rating	§6-2 step 5: the sway ceiling comes from the stern reverse limit through the lever $a = 1.8987$
rung 2 against rung 4, `path` and `straight`	$12.00 = 12.00$ m on both	§6-3: both translations are straight; one is made with the bow unit and one without it
rung 3, F_B, T_L, T_R	15.0, 4.2, 35.8 N	§6-2: the three rows of B^{-1}, worked by hand below
rungs 5 and 7, `phase 2`	53.04 s on both	§6-6: phase 1 does not know ψ_{berth}, so the two runs are one simulation until the approach radius
rung 7 against rung 5, `t to 0.5 m` and `d(end)`	56.22 against 75.94 s, but 0.347 against 0.186 m	§6-6: bow-first spends longer aligning, less time berthing, and stops further out
rung 7 against rung 5, $\lvert v\rvert_{max}$	0.268 against 0.353 m/s	§6-6: the bow-first attitude asks less of the axis the hull is weakest in
rung 8, F_B	40.0 N — the only rung that reaches the bow unit's own rating	§6-7: the NED loop asks for 43.68 N, and what stops it is the allocation's `shaft limit` at $n_{bow,max} = 60.08$ rad/s, which is $F_{B,max} = 40$ N in the other coordinate, not the controller's clip at $Y_{max} = 35.13$ N
rung 8 against rung 1, `t to 0.5 m`	51.00 against 42.68 s for the same target	§6-7: the magnitude cap $v_{ned,max} = 0.350$ m/s binds, and $\lvert u\rvert_{max}$ falls to 0.195 m/s
the right panel of the ladder chart	only rung 8's $\lvert F_B\rvert$ bar touches the $F_{B,max}$ line	§6-7 again, read without any table at all
every rung, `samples`	$T_{final}/h + 1$ — 6001, 7501, 1001	§6-0: the fixed step $h = 0.02$ s

What the figures say. Rung 3 is the one rung a student can check with a pencil, because nothing is closing around it. The demand is $X = 40$ N, $Y = 15$ N, $N = 10$ N·m, and B^{-1} read row by row gives

$$F_B = Y = 15.000\ \text{N}, \qquad N_s = N - l_f F_B = 10 - 1.5 \times 15 = -12.5000\ \text{N·m},$$

$$T_L = \tfrac{1}{2}\left(40 + \frac{-12.5}{0.395}\right) = 4.1772\ \text{N}, \qquad T_R = \tfrac{1}{2}\left(40 - \frac{-12.5}{0.395}\right) = 35.8228\ \text{N},$$

against 15.0, 4.2 and 35.8 N logged. Nothing is clipped on this rung, so the solve and the measurement agree to the digits the table prints. That arithmetic is the whole of Section 6-2 in three lines, and it is checkable without a simulator.

The pair that defines the week is rung 2 beside rung 4. Both travel a straight line of exactly their own length with the same gains and the same model; rung 2 does it sideways and rung 4 does it ahead. Rung 4 never asks the bow unit for anything — $F_B = 0.0$ N — and its stern pair runs at $T_L = T_R = 60.0$ N, two equal forces summing to the surge demand with no moment between them. Rung 2 holds $\lvert u\rvert_{max} = 0.021$ m/s, pins the bow unit at Y_{max}, and drives both stern units to $\lvert T_{min}\rvert = 66.7$ N astern to cancel the moment that the bow unit is making at the same time. A hull with two stern units and nothing forward can run rung 4 and cannot run rung 2. That is the sentence the third thruster was added to make true, and the ladder measures it rather than asserting it.

The berthing pair, rungs 5 and 7, is where the ladder stops rewarding a single number. Bow-first reaches 0.5 m almost twenty seconds sooner and latches eight seconds sooner, and yet ends 0.16 m further from the berth. Both are inside $R_{berth} = 0.40$ m, so both are arrivals by the week's own declaration; which is better depends on whether the quay charges for time or for centimetres.

> **⚠ Subtracting two rows is only honest when one thing differs**
>
> Rows 1 and 8 differ in the frame; rows 5 and 7 in the attitude; rows 2 and 4 in the direction of travel. Each of those three subtractions means something. Rows 1 and 5 differ in the model, the target, the run length and the controller in front of the loops, so the 33 s between their arrival times measures nothing in particular. The ladder is built one change at a time so that subtraction stays legitimate; a rung added without that discipline makes the table longer and says less.

What to try. The first two rows below are literals on a canvas, changed by double-clicking a Constant in the model named. The next two are workspace quantities, edited in `W06_0_setup.m`. The last is a line of `W06_J_ladder.m` itself.

What to try	What to watch
`X_cmd = 0` in `W06_3_thrust`	the stern pair becomes ± one number and the hand check above loses a term; the moment is then made entirely by the pair
`N_d = 13` and `E_d = 0` in `W06_2_sway` — the same 13 m, turned due north	the same model and the same gains now run rung 4's physics: $\lvert u\rvert_{max}$ and $\lvert v\rvert_{max}$ exchange places and F_B falls from 35.1 to 0.0 N. The ladder's central pair, made on one canvas
`n_bow_max = 104`, then run `W06_8_ned_run` alone — a rating of 119.8 N	F_B passes 40.0 N, proving that the 40.0 in table 2 was the allocation's `shaft limit` and not the trajectory
`stand_off = 12`, which rungs 5 and 7 read from the workspace at compile time	the berth leg lengthens for both; watch whether the gap between their two `d(end)` values widens or closes
delete the rung 3 line from `L` and re-run	the tables lose their only `NaN` row, which is a good way to confirm that the blanks were the open-loop rung and not a failure

> **✎ In class**
>
> - **Purpose**: show the week as one ladder rather than as seven separate experiments.
> - **Point to**: table 2, rung 2 beside rung 4 — $\lvert u\rvert_{max}$ of 0.021 against 0.804, F_B of 35.1 against 0.0.
> - **Ask**: both rungs travel 12.00 m in a straight line with the same gains and the same loop structure, on two canvases that differ only in the demand. Name the property of B that lets rung 2 exist, and say which column of B carries it.
> - **Take away**: a rung is worth building only when exactly one thing on it is new; that is what makes the row below it readable.

6-9. The equations and the code, line against line

What is observed. Three blocks in this week are hand-written MATLAB rather than Simulink blocks: the allocation, the rotations, and the mission state machine. Each is listed in full in the section that needs it — §6-2, §6-3 and §6-5 — beside a table that reads the code against its *figure*. What none of those tables does is read the code against its *equation*, and that is the reading a student needs when the derivation is in one hand and the block is open in the other. The fourth hand-written file of the week, `_tools/otter_bow.m`, is read against its equations where it is introduced, in §6-1, and is not repeated here.

The result. Four tables. Each row names an equation of the week, the line of code that is that equation, and what the pairing turns on. No listing is repeated here; each stays in its own section, and these tables point at it.

Table 1 · The allocation. `allocate`, listed in §6-2, inside the `allocation` subsystem of all seven models. It is the one block every rung has, because every rung ends in three thrusters. Its three lines are the three stages of the actuator model run backwards, in the order the forward model composes them.

Equation	The line	What the pairing turns on
$\mathbf{f} = B^{-1}\boldsymbol{\tau}$	`f = Binv * tau;`	steps 1 to 3 — one multiply stands for all three, and `Binv` is formed in the setup script from the B the column rule produced, never typed in
$k_i = k_{pos}$ for $f_i \geq 0$, k_{neg} for $f_i < 0$	`if f(1) < 0, k(1) = k_neg; end`	§6-1 step 13 — the coefficient is chosen on the **sign of the force**, which is the whole reason B^{-1} has to act before K^{-1} and not after
$\mathbf{u} = K^{-1}\mathbf{f}$	`u = f ./ k;`	§6-1 step 8 inverted — K is diagonal, so its inverse is three divisions and no matrix
$n_i = \operatorname{sign}(u_i)\sqrt{\lvert u_i \rvert}$	`n = sign(u) .* sqrt(abs(u));`	§6-1 step 14 — the thrust curve inverted one unit at a time; without the $\lvert\cdot\rvert$ every astern command returns a complex shaft speed
$n_i \leftarrow \operatorname{sat}(n_i)$	**no line** — the `shaft limit` Saturation downstream of the function	step 7 — the clip is outside the solve, and that is where both leaks come from

> **★ The last row is the one that is not a line**
>
> The three lines above are an exact inverse, and the clip is a block after them. The clip sits on the shaft speeds rather than on the forces for two reasons: that is the quantity each unit is physically rated in, and it is the quantity `otter_bow` receives, so the bound stands at the boundary between controller and plant. It costs nothing in generality — $T = k\,n\lvert n \rvert$ is strictly increasing and the ratings are *defined* as $T_{max} = k_{pos} n_{max}^2$ and $T_{min} = -k_{neg} n_{min}^2$, so clipping n and clipping T select the same attainable set.
>
> Two consequences follow from the ordering alone. First, the inverse is taken once, against the demand as issued, and nothing re-solves after a clip: when the sway demand exceeds $F_{B,max}$ the stern pair has already been told to cancel a moment $l_f F_B$ that the bow unit will not make. Second, the stern pair is clipped against two bounds of different size — $n_{max} = 103.93$ rad/s ahead against $n_{min} = -101.74$ rad/s astern, which is $T_{max} = 119.68$ N against $T_{min} = -66.71$ N, because $k_{neg} < k_{pos}$ — so an antisymmetric pair clipped asymmetrically no longer sums to the surge that was asked for. Both leaks measured in Experiment 6-2 are consequences of where the Saturation sits, not of the algebra above it. Re-solving with the clipped $\mathbf{n}$ would remove the first; removing the second needs the demand itself to be reshaped, which is Week 7.

Table 2 · The outer loop's rotation. `to_body`, listed in §6-3, inside `position loop` of `W06_1_point` — and, unchanged, of `W06_2_sway`, `W06_4_stop`, `W06_5_dock` and `W06_7_front`.

Equation	The line	What the pairing turns on
$\mathbf{p}_b = R(\psi)^{\mathsf{T}}\mathbf{p}_e$	the two rows assigned to `p_b`	the whole block is one matrix–vector product
$x_e = \cos\psi\, p_{e,N} + \sin\psi\, p_{e,E}$	`c*p_e(1) + s*p_e(2)`	the first row of R^{T}, which is the first **column** of R — the commonest sign error in the week
$y_e = -\sin\psi\, p_{e,N} + \cos\psi\, p_{e,E}$	`-s*p_e(1) + c*p_e(2)`	the second row of R^{T}
$R^{-1} = R^{\mathsf{T}}$	no `inv`, no `\` anywhere in the file	orthogonality, confirmed by check 13 — an inverse would be arithmetic paid for nothing

Table 3 · The two rotations of the NED variant. `to NED` and `to body`, in `W06_8_ned` — the one model of the week whose outer loop works in north and east.

Equation	Where it sits	What the pairing turns on
$\boldsymbol{\nu}_e = R(\psi)\,\boldsymbol{\nu}_b$	the `to NED` block, on the **measured velocity**, entering the inner loop	§6-7: the inner loop is now fed a quantity in north and east
$\boldsymbol{\tau}_b = R(\psi)^{\mathsf{T}}\boldsymbol{\tau}_e$	the `to body` block, on the **computed force**, leaving it	the allocation only ever accepts hull axes, in every model of the week
the same $R(\psi)$ in both	one `psi = x(12)` signal branching to two blocks	there is still only one rotation in the design; it is drawn twice because it is applied to two different quantities
$\mathbf{v}_{cmd} \leftarrow \mathbf{v}_{cmd}\,\min\!\left(1,\ v_{ned,max}/\lVert\mathbf{v}_{cmd}\rVert\right)$	`cap_speed`, in `position loop NED`	§6-7 step 6: a cap on the **magnitude**, which shortens the command without turning it — a per-axis cap would turn it

Table 4 · Folding a heading error. The same $\mathbf{ssa}$, in two spellings.

Equation	Where it sits	What the pairing turns on
$\mathbf{ssa}(\theta) = \mathbf{atan2}(\sin\theta,\ \cos\theta)$	the `Fcn` block `atan2(sin(u),cos(u))` in `heading loop`, in the six models that close a loop	§6-3: the error is a signal the diagram carries, so the fold is a block and is visible on the canvas. `W06_3_thrust` has no `heading loop` and therefore no such block — it is the rung with no controller at all
the same, written out	`de = atan2(sin(psi - psi_berth), cos(psi - psi_berth));` in `phases`	§6-5: this error is a comparison internal to the state machine and never leaves it, so it is a line

Both spellings exist for the same reason the week keeps the controllers out of MATLAB Function blocks: a quantity that the diagram carries belongs on the diagram, and a quantity that never leaves a block belongs inside it. Writing the heading loop's fold as a line inside a function would hide the one place in the model where a $\mathbf{350}$-degree error becomes a $\mathbf{-10}$-degree one, and that is the place a student most needs to find.

Part 2 · Laboratory run order

The experiments of Part 1 are worked through in order; this table is the index of what was run, for repeating the week at home.

The seven models share one hull, one allocation block and one measurement block, and all seven are laid out in the course order — **command → reference → controller → allocation → plant → measurement** — with each stage a subsystem. Every value a controller uses is visible on the diagram: the target, the berth, the approach point, the stand-off, the speed caps and the arrival tolerances are **Constant blocks on the canvas**, and $K_{p,pos}$, K_p, K_i and K_b are the **values of Gain blocks**. No controller this week is one MATLAB Function. The only hand-written code is the allocation, the two rotations and the mission block, and each of those three is listed in full in the section that needs it — §6-2, §6-3 and §6-5 — taken out of the saved model rather than copied from the builder.

On the right of each model sit three things:

	What it shows	When
Scope `vessel u v r`	the three velocities — the six-column contract every week of this course logs	while it runs
Scope `W06 this week`	what Week 6 adds: the set points, the three demands X, Y and N, the shaft speeds n_L and n_R, the bow force F_B, the distance d and the phase number	while it runs
Animate block	the live view: quay, approach point, berth, track and hull with its heading on the left; d, the commanded and actual heading, u and v on one axis, the demands against their limits, the two shafts and the phase on the right	while it runs, when `animate = 1`

Setting `pace = 1` in the workspace runs the model **at wall-clock speed**, so the live view moves at the speed the vessel would; `pace = 0` runs it as fast as it can. The results are identical and only the waiting differs: at `pace = 0`, `W06_5_dock_run` opens its model, runs the whole 150 s berthing mission with the live window on, draws the `StopFcn` panel and prints its two tables in 13.2 s of wall clock; at `pace = 1` the mission alone takes its full 150 s, because that is what pacing means. `W06_0_setup` sets `animate = 1` and `pace = 0`, because a person who presses Run wants the window but not the wait; `W06_vars`, which the ladder uses, sets `animate = 0` so that a table is not paid for in redraws.

The Animate block is never deleted to switch the live view off. `animate = 0` leaves the block in place doing nothing, so the diagram is the same in every model and in every week.

When a run finishes, the model's `StopFcn` calls `W06_plot`, which draws the track beside six panels and prints the same numbers a section script prints. Its `InitFcn` runs `run_alone`, which adds `_tools` and the week folder to the path, fills in any workspace variable that is missing, and reads `pace`. **Pressing Run on a freshly opened model therefore works on its own**, with no setup script run first.

Experiment	Model	Script	What it shows
6-0	all seven	`W06_0_setup`, `W06_1_build_bow`, `verify_w06_actuation`	the constants, the seven models written from code, and 17 checks
6-1	`W06_3_thrust`	`W06_3_thrust_run`	the three columns of $\boldsymbol{B}$, commanded one axis at a time
6-2	`W06_2_sway`, with `W06_3_thrust` borrowed as an instrument	`W06_2_sway_run`	which actuator ceiling binds a sway demand, and what leaks past it
6-3	`W06_1_point`	`W06_1_point_run`	the three-loop cascade holding a point, with and without the bow unit
6-4	`W06_4_stop`, with `W06_1_point` borrowed for the contrast	`W06_4_stop_run`	the braking distance, the settling time, and the arrival test with its latch
6-5	`W06_5_dock`	`W06_5_dock_run`	the four latched phases, with the instant each is entered
6-6	`W06_5_dock` and `W06_7_front`	`W06_5_dock_run` and `W06_7_front_run`	alongside against bow-first, read from the same block printed by both scripts
6-7	`W06_1_point` and `W06_8_ned`	`W06_1_point_run` and `W06_8_ned_run`	the one rotation, in each of the two places it can sit
6-8	all seven	`W06_J_ladder`, with `W06_fig_rung` and `W06_fig_ladder`	the seven rungs in one table and on one chart, each rung differing from the one it was copied from in exactly one thing

- The scripts only repeat what Run already shows, for several values at once, and print the numbers quoted in Part 1.
- Each rung owns one model and one script, named alike, and the script opens that model and nothing else. A section that compares two rungs therefore runs **two** scripts and lays their output side by side; it does not reconfigure one model twice.
- §6-5 and §6-6 both read `W06_5_dock_run`. The first reads its phase table down the rows; the second reads the `across the BERTH leg` block against the identical block printed by `W06_7_front_run`. The two scripts measure that leg by the same definition, which is what makes the comparison legitimate — see §6-6.
- §6-3 and §6-7 stand in the same relation: `W06_1_point_run` and `W06_8_ned_run` each print a `two commanded headings` block of the same shape, and §6-7's four-row table is those two blocks stacked.
- Section 6-9 runs nothing. It is the index from the equations of Part 1 to the lines of code that carry them, and every line it names is already listed in §6-2, §6-3 or §6-5.
- Each experiment ends with an **In class** note: purpose, what to point at, a question with its answer, and the sentence to take away.

Summary

Week Summary

Step	What was done	How it was verified
1	wrote all seven models from one builder, one rung to a model	`check_overlaps` and `check_diagonals` both 0 in all seven, counted from the saved files on disk
2	added a third thruster and built B from the column rule alone	`verify_w06_actuation` checks 2–6: $\max\lvert B - B_{hand}\rvert = 0$, $\det B = 0.790$, rank 3, and the hand-solved law within 3.55×10^{-14} N of `B \ tau` over 200 random demands
3	commanded the three axes one at a time	Experiment 6-1: residual at most 1.4×10^{-14} N; sway alone gives $v = 0.344$ m/s with $r = 0.000$ deg/s, which is the third row of B^{-1} being $[0\ \ 1\ \ 0]$
4	found which actuator ceiling binds a sway demand	Experiment 6-2: 35.13 N from the stern astern limit, against 63.03 N from the ahead limit and 40.00 N from the bow unit
5	predicted the leak above that ceiling	Experiment 6-2: at $Y = 50$ N the clipped allocation delivers $X = 28.23$ N and $N = -3.85$ N·m, matching the hand prediction to two decimals
6	built the position cascade, then removed the bow unit	Experiment 6-3: 42.68 s to within 0.5 m with three thrusters; 14.827 m of standing error and 10.37 deg of heading error without
7	derived the braking distance and the settling time	Experiment 6-4: $u_{lim}/K_{p,pos} = 5.33/2.67/0.40$ m against $5.04/2.46/0.34$ measured, and $1/K_{p,pos} = 6.67/3.33/0.50$ s against $6.26/2.90/0.17$
8	made arrival a conjunction of four bounded terms with a latch	Experiment 6-4: the four terms true for 64.0, 100, 62.4 and 100 % of the run, all four together from 45.42 s, latch changes 1; in a 0.3 m/s current never satisfied, holding 2.153 m out with the sway demand pinned at 35.13 N
9	compared two berthing attitudes, one model each	Experiment 6-5/6-6: berth leg 22.98 s alongside against 11.44 s bow-first, with 35.13 N and 3.96 N of peak sway demand; both end within 0.35 m of the berth
10	moved the one rotation to the other side of the limits	Experiment 6-7: all four runs end at $d = 0.000$ m, so the choice is not about correctness; NED pays 51.00 s against 42.68 s and asks 43.68 N of sway from a hull that makes 35.13 N
11	ran the seven rungs back to back into one table	Experiment 6-8: rung 2 crosses 13.00 m of pure sway with $\lvert u\rvert$ held to 0.021 m/s and F_B pinned at 35.1 N, the ceiling of §6-2; rung 4 crosses on the reciprocal axis with $\lvert v\rvert = 0.000$ m/s and $F_B = 0.0$ N; rung 8 is the only rung to reach $F_B = 40.0$ N
12	indexed every equation of Part 1 to the line of code that carries it	§6-9: four tables against the three listings taken out of the saved models in §6-2, §6-3 and §6-5, with no line of code printed twice in the week

Progress Check

> ★ **Minimum condition for following Week 7**

Theory

- ☐ Able to state the rank test and apply it to the three Otter configurations: under-actuated, fully actuated, over-actuated.
- ☐ Able to write the three columns of B from the geometry alone, and to say what the lever $l_f/(2y_{pont}) = 1.8987$ is the ratio of.
- ☐ Able to say which ceiling binds a pure sway demand, and why it is the stern astern limit rather than the bow thruster's rating.

- [ ] Able to predict the surge force and yaw moment that leak out of a clipped allocation.
- [ ] Able to name the quantity each of the three loops owns, and to state the condition under which their time-scale separation fails.
- [ ] Able to derive the braking distance $u_{lim}/K_{p,pos}$ and the settling time $1/K_{p,pos}$, and to say where the derivation stops holding.
- [ ] Able to explain what the arrival latch prevents, and what it removes along with the chatter.
- [ ] Able to place the single rotation in both of its two possible positions, and to state what each placement costs.
- [ ] Able to name what is new on each rung of §6-8, and to say why rung 3 is the one rung on which this ladder and the undergraduate one move in opposite directions.
- [ ] Able to point, for each row of the four tables of §6-9, at the line of the saved model that carries it — and to say why the saturation is the one that is not a line.

Laboratory

- [ ] `W06_1_build_bow` ran and all seven models reported `overlapping lines: 0` and `diagonal lines: 0`.
- [ ] `verify_w06_actuation` printed `ALL CHECKS PASSED`.
- [ ] `W06_1_point` was opened and Run was pressed with nothing set up first, and the live view drew the track, the hull heading, both speeds and both demands against their limits.
- [ ] One demand Constant was changed on a canvas — `N_d` or `E_d` in `W06_2_sway`, or `psi_berth` in `W06_7_front` — and the new behaviour appeared in the figure `W06_plot` draws, with no script edited.
- [ ] One constant was changed in `W06_0_setup` — the berth, the stand-off, or `Kp_pos` — and the difference between that and the line above was stated: a canvas Constant is the command, a workspace name is the coefficient.
- [ ] `W06_5_dock_run` and `W06_7_front_run` were both run, and their two `across the BERTH leg` blocks differed in the one quantity §6-6 says they should.
- [ ] `W06_J_ladder` ran the seven rungs, printed its three tables, and left seven rung figures and `W06_result_ladder.png` in `img/`.

Assignment 6

- **Due**: before the Week 7 session
- **Submit**: the modified `W06_0_setup.m`, the numbers requested below, and two figures

① Requirements

Rotate the quay. Section 6-5 placed the approach point by adding `stand_off` along $\boldsymbol{E}$ alone, and `verify_w06_actuation` check 14 confirms it does exactly that: $(40.0,\ 1.2) + 7.0$ m $\rightarrow (40.0,\ 8.2)$. That is correct only for a quay running north–south. Give the quay a heading of 30 degrees, apply the offset along the quay's own normal — one rotation, the same $\boldsymbol{R}(\psi)$ of §6-3 — and berth against it both alongside and bow-first.

② Verification — mandatory

> **★ A claim is not a result. Numbers are required, with the averaging window stated**

1. The approach point, computed by hand from the quay heading and reported by the mission block, to three decimals.
2. The phase table of Experiment 6-5 for both attitudes against the rotated quay: entry instant and duration of all four phases.
3. The berth-leg maxima of $\boldsymbol{X}$ and $\boldsymbol{Y}$, and the percentage of that leg spent on the $\boldsymbol{Y}$ cap, for both attitudes.
4. The final distance to the berth, the drift and the heading error, for both attitudes.

③ Analysis (5–10 lines)

Against a north–south quay the alongside attitude spent 4.0 % of its berth leg on the sway cap and the bow-first attitude 0.0 %. State what those two percentages become on a quay rotated 30 degrees, and explain the direction of the change from where the sway demand comes from.

Grading

Criterion	Weight
The model runs and produces the requested output	25%
Verification performed and numbers reported	40%
Correctness of the analysis	25%
Readability of the code	10%

Troubleshooting

Symptom	Cause	Fix
`W06_0_setup` stops in `mss_path`	the MSS toolbox is not at `Tools\MSS`	place MSS there; see the first page
a section script appears to run and do nothing	an `.slx` shares its name, so MATLAB opens the model instead of running the script	the runners carry `_run` in their names; never give a script a model's stem
the model stops at `Undefined function or variable 'h'`	the workspace is empty and every Constant holds a **name**	nothing is broken. The model's `InitFcn` fills missing variables itself, so this appears only if the `.slx` was copied out of the vault and `_tools` can no longer be found from it
adding the measured distance as an input to `mission` is reported as an algebraic loop	the mission block sets the target, the loops compute $\boldsymbol{d}$ from that target, and $\boldsymbol{d}$ would then set the target	the block computes its own distance from $\boldsymbol{x}$; it must not receive one
a heading on a Scope is about $\mathbf{57}$ times the expected number	`psi` and `psi_set` are both logged in **degrees**, not radians	subtract them as they are; do not convert either
the vessel holds station a few metres out and never declares arrival	a current pushes it back outside $\boldsymbol{R_{app}}$, and the latch does not re-enter phase 1	expected, and measured: §6-4 gives $\mathbf{2.153}$ m in a $\mathbf{0.3}$ m/s current. Guarded re-entry would repair it, and is not built in this course
`W06_plot` draws a figure that does not match these notes	a changed variable is still in the workspace	run `W06_0_setup` again

References

Primary

- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed., Wiley, 2021, §15.3 — dynamic positioning: the outer position loop and the inner velocity loop, in both the body-fixed and the NED formulation of §6-7.
- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, 1st ed., Wiley, 2011, §12.3.4 — the extended thrust vector, from which `_tools/otter_B.m` builds the three columns of $\boldsymbol{B}$ used in §6-1.
- MSS toolbox, `VESSELS/otter.m` — the hull, its mass matrix and the two stern propellers; `demoOtterUSVPathFollowingHeadingControl` — the autopilot structure the heading loop of §6-3 keeps unchanged. The copy this course runs against has no berthing example, and the phase machine of §6-5 is accordingly derived in place rather than adapted.
- Bang, J., *et al.* Development and experimental validation of an autonomous USV berthing system with transition logic for multiple berthing manoeuvre strategies. *Ocean Engineering* **331** (2025) 121333. DOI 10.1016/j.oceaneng.2025.121333 — the two-strategy arrangement of §6-6, with the transition logic of which the `bow_on` Constant is the degenerate, chosen-once case.

In this course

- Week 2 §2-6 to §2-8: P, I and the tuning order, used unchanged by the velocity loop of §6-3.
- Week 4 §4-4: heading is not course. It is why the heading loop stays separate from the position loop rather than being folded into it.
- Week 5 §5-7: the yaw moment a surge force leaves behind. Section 6-2 asks the same question of sway, and the answer is harsher.
- Appendix A1 §A1-3 and §A1-6: the rank of $\mathbf{B}$ and the attainable control set, which is where the ceiling of §6-2 is stated in general form.

Next Week

Week 7 — Four Tilting Thrusters, and the Open-Loop Check. The allocation of this week was a square solve: three demands, three thrusters, exactly one answer, nothing to choose. Section 6-2 clipped that answer when it exceeded a ceiling and measured what leaked out. Week 7 replaces the two fixed thrusters with four tilting ones, so B becomes 3×8 with rank 3 and a five-dimensional null space: the answer is no longer unique, and clipping is no longer the best attainable approximation of the demand. Its first half gives the eight numbers by hand and checks that the motion model is the equations; its second half reverses the arithmetic — three demanded forces into eight components, under a weight matrix, a servo limit, a thrust cap and a 50 Hz budget.